icpcBoard

ICPC Algorithm Reference

目录

1. 竞赛常用内容 4

1.1 通用代码骨架与多测清空 4

1.2 数据范围与复杂度速判 5

1.3 快速输入输出 5

1.4 编译、运行与 Sanitizer 6

1.5 调试宏与断言 7

1.6 随机数与数据生成 8

1.7 暴力程序与正确性基准 9

1.8 自动对拍与失败保留 10

1.9 文件与标准流切换 11

1.10 WA、RE、TLE、MLE 与 UB 排查 11

1.11 边界数据检查清单 12

1.12 整数溢出与浮点精度 13

1.13 赛中提交与三人协作流程 14

1.14 Python 调试与随机对拍 14

2. STL 库使用 17

2.1 顺序容器：`vector` 与 `array` 17

2.2 `string` 17

2.3 `pair`、`tuple` 与结构化绑定 18

2.4 两端序列与适配器：`deque`、`stack`、`queue` 18

2.5 堆：`priority_queue` 19

2.6 有序集合：`set`、`multiset` 19

2.7 有序映射：`map` 20

2.8 哈希容器：`unordered_set`、`unordered_map` 20

2.9 `bitset` 21

2.10 排序、二分与边界 21

2.11 去重、排列与区间算法 22

2.12 数值算法与 Lambda 22

2.13 迭代器与半开区间 23

3. 基础算法与搜索 24

3.1 复杂度与枚举 24

3.2 前缀和与差分 25

3.3 二分查找与二分答案 27

3.4 双指针与滑动窗口 28

3.5 离散化 29

3.6 分治与归并计数 30

3.7 贪心与区间调度 31

3.8 DFS 深度优先搜索 32

3.9 BFS 广度优先搜索 34

3.10 回溯与剪枝 35

3.11 折半搜索 36
3.12 0-1 BFS 37
3.13 常用构造题思路 38
4. 动态规划 40
4.1 DP 入门：状态、转移与顺序 40
4.2 0-1 背包 41
4.3 完全背包与多重背包 42
4.4 最长上升子序列（LIS） 44
4.5 最长公共子序列（LCS） 45
4.6 区间 DP 46
4.7 树形 DP 47
4.8 状态压缩 DP 48
4.9 数位 DP 49
4.10 DAG 上的 DP 51
4.11 编辑距离 52
4.12 树形 DP 与换根 53
4.13 概率与期望 DP 54
5. 数据结构 56
5.1 栈与队列 56
5.2 单调栈 57
5.3 单调队列 58
5.4 堆与优先队列 58
5.5 并查集 59
5.6 树状数组 60
5.7 线段树（区间加、区间和） 62
5.8 ST 表（静态 RMQ） 64
5.9 分块（区间加、区间和） 65
5.10 树状数组：区间加与区间和 67
5.11 DFS 序与子树展平 67
5.12 树链剖分 68
5.13 线段树：区间加、赋值与求和 69
5.14 DSU on Tree（树上启发式合并） 71
5.15 单点修改线段树与自定义合并 73
6. 图论 76
6.1 拓扑排序 76
6.2 Dijkstra 单源最短路 77
6.3 Bellman–Ford 与负环判断 78
6.4 Floyd 全源最短路 80
6.5 Kruskal 最小生成树 81
6.6 二分图判定 82
6.7 二分图最大匹配（增广路） 83
6.8 强连通分量（Tarjan） 85
6.9 割点与桥（Tarjan） 86
6.10 最近公共祖先（倍增 LCA） 88
6.11 树的直径 89

6.12 最大流（Dinic） 91
6.13 欧拉路径与回路 93
6.14 2-SAT 94
6.15 最小费用最大流 95
7. 数学 97
7.1 GCD 与扩展 GCD 97
7.2 快速幂 98
7.3 素数筛 99
7.4 质因数分解 100
7.5 欧拉函数 101
7.6 模逆元 102
7.7 线性同余方程 103
7.8 中国剩余定理（非互质模数） 104
7.9 组合数 105
7.10 容斥原理 106
7.11 矩阵快速幂 108
7.12 高斯消元 109
7.13 公平组合游戏与 SG 函数 110
7.14 质因数分解、约数个数与约数和 111
8. 字符串 113
8.1 字符串哈希 113
8.2 前缀函数与 KMP 114
8.3 Z 函数（扩展 KMP） 115
8.4 Manacher 116
8.5 Trie 字典树 117
8.6 AC 自动机 118
8.7 后缀数组 120
9. 计算几何 122
9.1 点、向量与基本运算 122
9.2 叉积与方向判断 123
9.3 直线交点与线段相交 124
9.4 距离与投影 125
9.5 多边形面积 127
9.6 点在多边形内 128
9.7 凸包 129
9.8 旋转卡壳：凸包直径 130
9.9 圆与直线、圆与圆 132
9.10 整数几何方向模板 132
10. 题意关键词索引 134
10.1 上机、语言与 STL 134
10.2 序列、搜索、DP 与数据结构 134
10.3 图与树 135
10.4 数学、字符串与几何 136
10.5 五秒模型核对 137

1. 竞赛常用内容

1.1 通用代码骨架与多测清空

问题： 将单组逻辑放进 `solve()`，入口只负责读取测试组数并循环调用，便于复用和隔离状态。多测总成本是各组复杂度之和；优先使用局部容器，谨慎清空大型全局数组。

思路：

- 单测时保留 `int testCases = 1`，不要读取不存在的测试组数。
- 多测时取消读取 `testCases` 的注释；先确认题面是否给出 T。
- 优先把数组、图、DP 表定义为 `solve()` 内局部变量，每组自动重新构造。
- 大型全局数组只清空本组访问过的范围；时间戳法适合频繁清空布尔或编号状态。
- 每组输出格式统一在 `solve()` 内处理，题目要求时加 `Case #x:`。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// 参数：无；从标准输入读取一组数据。返回：无。效果：计算并写出当前测试用例答案。
void solve() {
    int n;
    cin >> n;

    // 状态尽量放在 solve 内：每组测试结束后自动销毁，不会串组。
    vector<ll> a(n + 1); // 本模板约定输入下标为 1..n
    for (int i = 1; i <= n; ++i) cin >> a[i];

    // 在这里编写单组答案。
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int testCases = 1;
    // cin >> testCases; // 仅当题面明确给出测试组数时启用
    while (testCases--) solve();
    return 0;
}
```

易错点：

1. 把样例中的多组数据误认为题目输入含 T，会从第一项开始读错。
2. `vector::clear()` 只把大小变为 0；重新 `resize` 后不能假设旧内存内容有意义。
3. 全局邻接表、答案、计数器、时间戳和静态局部变量都可能跨组残留。
4. 对最大数组每组完整 `memset`，可能把本可通过的算法拖成 TLE。

参考：

- OI Wiki：C++ 语法基础
- OI Wiki：常见错误

1.2 数据范围与复杂度速判

问题： 依据 n, m, W, T 和总和约束估算时间与峰值内存，再选择算法。清空、建图、排序、递归栈和多测开销都要计算；常数和评测机差异使经验表只能初筛。

思路：

- 多测必须估算 $\sum n$、$\sum m$，不能只看单组上限。
- 图算法用 $n + m$ 估算；背包看 nW；状压看 $n2^n$ 或 $n^2 2^n$。
- 内存按“元素个数 × 单元素字节数”计算，并加上容器、递归栈和多份数组。
- 约 10^8 次简单整数操作已可能接近常见时限；哈希、字符串复制、堆操作常数更大。
- 用最坏数据本地计时只能辅助判断，不能替代理论复杂度。

公式与速查：

```
常见候选（只作一秒级 C++ 的粗略起点）：
n <= 20          O(2^n)、O(n * 2^n)
n <= 40          O(2^(n/2)) 折半搜索
n <= 500         O(n^3) 需关注常数
n <= 5,000       O(n^2) 需关注常数和内存
n <= 200,000     O(n log n)、O(n sqrt(n)) 视常数而定
n,m <= 1,000,000 O(n)、O(n+m)

内存 MiB = 元素个数 * sizeof(元素) / 1024 / 1024
例如：2^20 * 20 个 long long 约为 160 MiB。
```

易错点：

1. $O(n^2)$ 的二维 `long long` 数组常先 MLE，再谈不上 TLE。
2. 把取模、平衡树、哈希或大整数运算都当成一个同常数的“基本操作”。
3. 忽略清空数组、建图、排序或输入输出，它们也计入总复杂度。
4. 只根据样例规模选算法；评测按题目给出的最大范围构造数据。

参考：

- OI Wiki：复杂度简介
- OI Wiki：常见错误

1.3 快速输入输出

问题： 非交互题在首次 IO 前关闭同步并解绑 `cin`，统一使用 C++ 流、以 `\n` 换行。交互题要及时刷新；`getline` 与格式化读取混用时需处理残留换行。

思路：

- 在任何 IO 前调用 `ios::sync_with_stdio(false); cin.tie(nullptr);`。
- 关闭同步后统一使用 `cin/cout`，不要混用 `scanf/printf`。
- 普通换行用 `'\n'`；`endl` 会额外刷新缓冲区。
- 交互题必须在每次询问后 `cout << flush`，不能照搬非交互写法。
- 输出大量元素时先拼接或顺序写入，不要在循环中无必要地刷新。

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false); // 必须在第一次 cin/cout 操作前调用
    cin.tie(nullptr);            // 非交互题解除 cin 对 cout 的自动刷新

    int n;
    cin >> n;
    vector<long long> a(n);
    for (long long& value : a) cin >> value;

    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << a[i];
    }
    cout << '\n'; // 不需要立即显示时，用换行而不是 endl
    return 0;
}
```

易错点：

1. 关闭同步后混用 C 与 C++ 输入或混用两套输出，顺序可能异常。
2. 交互题忘记刷新会等待超时；普通题频繁 `endl` 又可能拖慢输出。
3. `cin >> s` 跳过空白；要读取含空格整行时使用 `getline`，并处理残留换行。
4. 输入失败后变量不会得到有效新值；读到 EOF 的题应以 `while (cin >> x)` 控制。

参考：

- OI Wiki：读入、输出优化
- OI Wiki：C++ 输入输出

1.4 编译、运行与 Sanitizer

问题： 用 `-O2` 版模拟提交，用带警告和 Sanitizer 的调试版查越界与未定义行为。先修警告，再测样例、边界和随机数据；检查版较慢，不用于时限评估。

思路：

- 普通版开启 C++17、优化和警告；让本地标准尽量接近评测机。
- 调试版加入 `-g`、`-DDEBUG`、`-fsanitize=address,undefined` 和栈帧选项。
- 用输入重定向复现测试，必要时用 `time` 粗测最坏数据。
- 先修编译警告，再运行样例、边界数据和随机对拍。
- Sanitizer 报告从第一处错误和调用栈开始看；后续错误可能只是连锁反应。

命令：

```
# 接近提交环境的优化版
g++ main.cpp -std=c++17 -O2 -Wall -Wextra -Wshadow -o main

# 本地查越界和未定义行为；不要用它测正式时限
g++ main.cpp -std=c++17 -O1 -g -DDEBUG \
  -fsanitize=address,undefined -fno-omit-frame-pointer -o main_dbg
```

```
./main < sample.in > sample.out
./main_dbg < fail.in
time ./main < maximal.in > /dev/null
```

易错点：

1. 只在 `-O0` 下测试；未定义行为可能在 `-O2` 下表现不同。
2. 把 Sanitizer 的额外时间、内存开销当成正式程序复杂度。
3. 忽略警告中的窄化、符号比较、变量遮蔽和未初始化使用。
4. 本地编译器版本、标准或整数宽度与评测机不同，造成“本地过、提交错”。

参考：

- OI Wiki：命令行编译与调试
- OI Wiki：编译优化

1.5 调试宏与断言

问题： 把日志集中写到 `cerr`，用 `DEBUG` 控制是否编译；用 `assert` 检查下标、区间和内部不变量。断言不能代替合法输入分支，也不要把必要操作写进断言。

思路：

- 本地用 `-DDEBUG` 开启日志，提交代码不定义 `DEBUG`。
- 在二分边界、图下标、DP 状态范围和数据结构不变量处放断言。
- 错误诊断信息写 `cerr`；正式答案只写 `cout`。
- 先打印最小必要状态：位置、关键变量、容器大小，不要倾倒全部大数组。
- `assert` 失败表示实现或推理错误；若条件来自不可信输入，应正常分支处理。

```
#include <bits/stdc++.h>
using namespace std;

#ifdef DEBUG
template <class... Args>
// 参数：任意可流输出的调试值。返回：无。效果：按顺序写入标准错误。
void debugLog(Args&&... args) {
    // 折叠表达式按参数顺序输出，调试格式集中在一个入口维护。
    ((cerr << forward<Args>(args) << ' '), ...);
    cerr << '\n';
}
#define DBG(...) debugLog("[", #__VA_ARGS__, "] =", __VA_ARGS__)
#else
#define DBG(...) ((void)0)
#endif

// 参数：prefix[0..n] 与 1-based 闭区间 [left,right]。返回：区间和。效果：断言边界并写调试日志。
long long rangeSum(const vector<long long>& prefix, int left, int right) {
    // 调用约定：prefix[0..n]，查询原数组闭区间 [left, right]。
    assert(1 <= left && left <= right);
    assert(right < static_cast<int>(prefix.size()));
    DBG(left, right, prefix[right], prefix[left - 1]);
    return prefix[right] - prefix[left - 1];
}
```

易错点：

1. 调试输出写入 `cout` 会造成 WA；提交前也不要靠手工搜索删除日志。
2. 宏参数含逗号表达式或副作用时容易产生意外；`DBG(++x)` 会真的修改 `x`。
3. 定义 `NDEBUG` 后标准 `assert` 会被移除，不能把必要计算写进断言表达式。
4. 日志本身可能使 TLE 或改变时序，只保留能缩小错误范围的信息。

参考：

- OI Wiki：常见技巧
- OI Wiki：命令行编译与调试

1.6 随机数与数据生成

问题： 用 `<random>` 的引擎和分布生成可控测试，并保存种子以复现失败。均匀整数范围应由分布指定；随机化不能替代正确性证明或复杂度分析。

思路：

- 从命令行读取固定种子；未提供时才使用时钟种子，并把种子输出到 `cerr`。
- 用 `uniform_int_distribution<ll>(left,right)`，不要用 `rng() % range` 追求严格均匀。
- 随机排列使用 `shuffle`；C++17 已移除 `random_shuffle`。
- 生成图时明确是否允许自环、重边、断开；树可令每个 `v` 随机连接 `[1,v-1]`。
- 纯随机以外，固定加入最小值、最大值、全相等、单调、星形和链形等结构。

```
#include <bits/stdc++.h>
using namespace std;

using ull = unsigned long long;

int main(int argc, char** argv) {
    // 对拍时由脚本传入 seed；同一 seed 必须生成同一测试。
    ull seed = argc >= 2 ? stoull(argv[1])
                         : chrono::steady_clock::now().time_since_epoch().count();
    mt19937_64 rng(seed);
    cerr << "seed = " << seed << '\n';

    // 参数：闭整数区间 [left,right]。返回：均匀随机整数。效果：推进 mt19937_64 状态。
    auto randomInt = [&](long long left, long long right) {
        return uniform_int_distribution<long long>(left, right)(rng);
    };

    int n = static_cast<int>(randomInt(1, 12)); // 小规模便于暴力校验
    vector<int> permutation(n);
    iota(permutation.begin(), permutation.end(), 1);
    shuffle(permutation.begin(), permutation.end(), rng); // 原地打乱，元素仍是 1..n

    cout << n << '\n';
    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << permutation[i];
    }
    cout << '\n';
    return 0;
}
```

易错点：

1. 失败后不记录种子，导致错误数据无法再次生成。
2. `rand() * rand()` 可能溢出且分布差；`rng() % k` 也可能有模偏差。
3. 只造平均随机数据，覆盖不到空集、重复值、极值和退化结构。
4. 生成器自身不满足题面约束，会让对拍结论失去意义。

参考：

- OI Wiki：随机函数
- OI Wiki：出题·造数据

1.7 暴力程序与正确性基准

问题： 对小规模输入编写结构简单、容易确认的暴力解，作为优化算法的差分基准。暴力可指数级，但必须覆盖合法边界并与被测程序使用相同输入定义。

思路：

- 先写数学定义最直接对应的判定，再写优化程序；两者避免复用核心函数。
- 暴力只需支持生成器的小范围，不必支持题面最大数据。
- 返回完整答案；多解或浮点题需写等价比较器，不能简单文本 `diff`。
- 对优化程序的每个可疑性质，增加能打破该性质的构造数据。
- 发现差异后先保存输入，再分别检查暴力与正解，不能默认暴力必对。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// 示例基准：枚举所有子集，求不超过 capacity 的最大价值。
// 仅用于 n 很小的对拍；下标统一为 0..n-1。
// 参数：0-based 平行重量/价值数组及容量。返回：子集枚举得到的最大价值。效果：只适用于 n<=24 的小数据，O(2^n n)。
ll bruteKnapsack(const vector<int>& weight,
                 const vector<ll>& value,
                 int capacity) {
    int n = static_cast<int>(weight.size());
    assert(n <= 24); // 防止误拿暴力跑正式上限
    ll answer = 0;
    for (unsigned long long mask = 0; mask < (1ULL << n); ++mask) {
        // 每个二进制位独立表示一件物品是否选中。
        int totalWeight = 0;
        ll totalValue = 0;
        for (int i = 0; i < n; ++i) {
            if (mask & (1ULL << i)) {
                totalWeight += weight[i];
                totalValue += value[i];
            }
        }
        if (totalWeight <= capacity) answer = max(answer, totalValue);
    }
    return answer;
}
```

易错点：

1. 暴力与正解复制了同一个转移或辅助函数，二者可能稳定地产生同一错误。
2. 暴力数组越界、溢出或误解题意，会把正确程序判成错误。
3. 枚举上界用 `1 << n` 可能溢出，应使用 `1ULL << n` 并限制 n。
4. 只比较答案值而忽略无解、多解、顺序无关或误差判定规则。

参考：

- OI Wiki：枚举
- OI Wiki：常见技巧·对拍

1.8 自动对拍与失败保留

问题： 生成器、暴力程序和正式解在同一输入上运行并比较输出，可快速发现反例。对拍成本取决于单轮三程序耗时；保存种子和失败输入，先缩小反例再定位。

思路：

- 先分别编译 `gen.cpp`、`brute.cpp`、`main.cpp`，保证三者无编译警告。
- 脚本给生成器传递递增种子，任一步骤非零退出都视为失败。
- 用 `diff -Z -B` 只在题目允许忽略空白差异时比较；严格格式题直接用 `diff`。
- 失败时保留 `fail.in`、双方输出和 seed，再用 Sanitizer 单独复现。
- 多解、集合输出、浮点误差题应改写 checker，把两个输出都按题意验证。

命令：

```
#!/usr/bin/env bash
set -u

# 用法：先编译出 ./gen ./brute ./main，再执行 bash stress.sh。
for seed in $(seq 1 100000); do
  ./gen "$seed" > stress.in || exit 1
  ./brute < stress.in > brute.out || exit 1
  ./main < stress.in > main.out || exit 1

  if ! diff -Z -B brute.out main.out > /dev/null; then
    cp stress.in fail.in
    echo "Wrong answer, seed = $seed"
    diff -u brute.out main.out
    exit 1
  fi
done

echo "Accepted all generated tests"
```

易错点：

1. 程序 RE 后仍比较旧输出文件，可能误报一致；脚本必须检查退出码。
2. `diff` 不理解“任意合法方案”；此类题必须按题意写 checker。
3. 对拍全是同一种随机分布，跑再多轮也未必覆盖退化边界。
4. 并行对拍共用同名临时文件会互相覆盖；每个进程需独立目录或文件名。

参考：

- OI Wiki：常见技巧·对拍
- OI Wiki：出题·造数据

1.9 文件与标准流切换

问题： 本地可在宏控制下重定向输入输出文件，提交时仍使用标准流。检查运行目录和文件名；重定向应发生在任何 IO 之前，避免评测机找不到文件。

思路：

- 本地首选 `./main < sample.in > sample.out`，源码无需修改。
- 比赛明确要求文件 IO 时，严格使用题面给出的文件名和大小写。
- 若使用条件编译，只把本地文件切换放在 `#ifndef ONLINE_JUDGE` 中。
- 检查 `freopen` 返回值；本地文件不存在时立即报错。
- 调试信息仍写 `cerr`，它不会进入重定向到答案文件的 `stdout`。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
#ifndef ONLINE_JUDGE
    // 本地可用；若评测平台不定义 ONLINE_JUDGE，提交前应删除这段。
    if (freopen("input.txt", "r", stdin) == nullptr) return 1;
    if (freopen("output.txt", "w", stdout) == nullptr) return 1;
#endif

    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long value;
    while (cin >> value) cout << value << '\n';
    return 0;
}
```

易错点：

1. 假设所有 OJ 都定义 `ONLINE_JUDGE`；不确定时不要把 `freopen` 留在提交代码里。
2. 文件名大小写、扩展名或相对路径错误，导致读不到输入。
3. 把输入输出文件写反，或用写模式打开输入文件造成数据被截断。
4. 使用 shell 重定向时把 `2>` 与 `>` 混淆，调试信息进入答案文件。

参考：

- OI Wiki：常见技巧
- OI Wiki：C++ 输入输出

1.10 WA、RE、TLE、MLE 与 UB 排查

问题： 按现象分别检查建模/边界、非法访问、渐进复杂度/常数、峰值内存和未定义行为。先用最小失败输入复现，再结合编译警告、Sanitizer 与计时定位。

思路：

- CE：在与评测相同标准下编译，查看第一条错误，确认提交语言和入口函数。
- WA：手算最小反例，核对输入含义、开闭区间、无解输出、取模与浮点格式。
- RE：用失败输入跑 Sanitizer，检查数组下标、递归深度、迭代器和除数。
- TLE/OLE：确认循环会推进、输出量有限，再重新计算最坏复杂度和总测试规模。
- MLE：计算峰值而非单数组大小，关注 `vector<vector<...>>`、DP 多份表和按值传参。

排查命令。

```
# 1. 带警告编译；2. 用检查版复现；3. 查看峰值时间和内存（GNU/Linux）。
g++ main.cpp -std=c++17 -O2 -Wall -Wextra -Wshadow -o main
g++ main.cpp -std=c++17 -O1 -g \
  -fsanitize=address,undefined -fno-omit-frame-pointer -o main_dbg
./main_dbg < fail.in
/usr/bin/time -v ./main < maximal.in > /dev/null
```

易错点：

1. 看到 RE 就盲目加大数组；根因也可能是负下标、栈溢出或除零。
2. 随机修改代码后失败样例消失，就误以为问题已修复；UB 可能只改变表现。
3. 只测样例，未覆盖最大规模、不可达、重复边、负数和空答案。
4. 诊断 TLE 时只看主算法，忽略重复清空、容器复制和调试输出。

参考：

- OI Wiki：题型概述·评测结果
- OI Wiki：常见错误
- OI Wiki：编译优化·未定义行为

1.11 边界数据检查清单

问题： 根据约束逐项构造最小、最大、重复、单调、全相等、不可达和答案极值等输入。边界测试用于验证实现假设，复杂度仍需单独证明。

思路：

- 规模：$n = 0$（若允许）、$n = 1$、最小合法值、最大 n, m, T、总规模上限。
- 数值：全 0、全负、全相等、最小/最大值、答案刚好越过 `int`、乘法溢出。
- 序列：递增、递减、回文、重复集中、答案在首尾、空答案和唯一答案。
- 图树：孤点、断开、重边、自环（若允许）、链、星、完全图、源点即终点。
- 区间：单点、全区间、端点相接、左右边界、开闭含义；多测检查上一组残留。

检查清单。

```
[ ] 手算并通过最小合法输入
[ ] 测试空答案、无解、唯一解、多解
[ ] 测试全相等、严格递增、严格递减
[ ] 测试 0、负数、最大值与 long long 溢出边缘
[ ] 测试下标 0/1、n-1/n、区间端点和源点等于终点
[ ] 测试断开图、重边、自环、链、星（按题面允许范围）
[ ] 测试两组连续输入，第二组比第一组小
[ ] 最大数据下检查时间、内存和递归深度
[ ] 核对输出大小写、空格、换行、精度和 Case 编号
```

易错点：

1. 构造了题面不允许的数据并据此“修复”，反而破坏正确逻辑。
2. 只测 $n = 1$，未测答案在最后一个元素或最后一条边的情形。
3. 多测只用相同大小，未发现第二组变小时的残留数组内容。
4. 随机数据代替结构数据；链、星、全相等等退化形状出现概率极低。

参考：

- OI Wiki：常见错误
- OI Wiki：出题·数据与样例

1.12 整数溢出与浮点精度

问题： 根据最大中间结果选择整数类型，并在乘法前提升类型；累加、平方和及哨兵运算都要检查范围。浮点比较按误差模型选绝对/相对误差，避免随意固定 epsilon。

思路：

- 和、距离、计数优先用 `long long`；乘法写 `1LL * a * b`。
- 可能超过约 9×10^{18} 的整数中间值用 `__int128`，输出需自行转换。
- 无穷大取 `numeric_limits<ll>::max()/4`，加法前先排除不可达状态。
- 浮点比较用 `abs(a-b) <= eps * max(1,abs(a),abs(b))` 的混合误差。
- 输出按题意使用 `fixed << setprecision(k)`；几何 EPS 应结合坐标范围调整。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using i128 = __int128_t;
using u128 = __uint128_t;

// 参数：待比较实数及相对/绝对容差 eps。返回：混合误差下是否近似相等。效果：无副作用。
bool almostEqual(long double a, long double b,
                 long double eps = 1e-12L) {
    // 同时照顾接近 0 的绝对误差和大数的相对误差。
    return fabsl(a - b) <= eps * max({1.0L, fabsl(a), fabsl(b)});
}

// 参数：有符号 128 位整数。返回：十进制文本。效果：无副作用，支持最小负值。
string toString(i128 value) {
    if (value == 0) return "0";
    bool negative = value < 0;
    // 先加一再取负，避免直接对 i128 最小值取负导致溢出。
    u128 magnitude = negative ? static_cast<u128>(-(value + 1)) + 1
                              : static_cast<u128>(value);
    string result;
    while (magnitude > 0) {
        result.push_back(static_cast<char>('0' + magnitude % 10));
        magnitude /= 10;
    }
    if (negative) result.push_back('-');
    reverse(result.begin(), result.end());
    return result;
}

// 参数：两个 long long 因子。返回：精确的 128 位乘积。效果：先扩宽再相乘，避免 64 位乘法溢出。
i128 safeProduct(ll a, ll b) {
    return static_cast<i128>(a) * b; // 转换必须发生在乘法之前
}
```

易错点：

1. `long long product = a * b;` 中 `a,b` 为 `int` 时，乘法可能已溢出。
2. 有符号整数溢出是 UB，不能依赖“自然回绕”。
3. 用 `a == b` 判断计算所得浮点数，或所有问题固定使用同一个 EPS。
4. `NaN` 与任何数（包括自身）比较都不相等；异常值应用 `isfinite` 检查。

参考：

- OI Wiki：变量·整数与浮点类型
- OI Wiki：常见错误
- OI Wiki：编译优化·未定义行为

1.13 赛中提交与三人协作流程

问题： 三人协作时约定接口、变量/编号口径和提交负责人，避免多人同时改同一文件。提交前确认编译标准、样例、边界、多测清理与输出格式，并保留可回滚版本。

思路：

- 开场快速读题，记录题意确定度、预估难度、算法、实现者和卡点；优先稳题。
- 维护纸面或离线状态表：未读、在读、待证明、编码、测试、已交、待修。
- 上机前写清状态、关键公式和边界；机器优先给已证明、可快速 AC 的实现。
- 提交前由另一人核对题号、语言、复杂度、清空、整数范围、样例和文件内容。
- 收到 WA/RE/TLE 后先保留版本与失败假设，再决定调试、重写或暂时换题。
- 封榜期按“通过概率 × 题目收益 ÷ 预计时间”排序，不因 sunk cost 长时间单点卡题。

提交前检查。

```
[ ] 提交的是正确题号、正确文件、正确语言标准
[ ] 已删除本地 freopen，或确认评测定义 ONLINE_JUDGE
[ ] 调试信息不进入 stdout，交互题每次询问都 flush
[ ] 复杂度覆盖最大 n、m、T 和所有测试总规模
[ ] long long / __int128、INF 加法、取模和浮点精度正确
[ ] 多测状态已清空；数组、字符串、图的下标边界正确
[ ] 样例、最小数据、无解数据、手造边界均通过
[ ] 队友复述了算法并审过最危险的三处代码
```

易错点：

1. 三人同时围着一份代码调试，导致无人继续读题或准备下一题。
2. 未证明就抢机器编码，写完才发现模型或复杂度不成立。
3. WA 后立即重复提交同一份代码，产生无信息增益的罚时。
4. 只口头同步进度；长时间后队友不知道谁在做什么、哪题可接手。
5. 提交窗口选错题号、语言或旧文件，这是最不值得的罚时。

参考：

- OI Wiki：ICPC/CCPC 赛事与赛制
- OI Wiki：题型概述

1.14 Python 调试与随机对拍

适用范围： 本地验证一个已编译的 C++ 程序；适用于输入规模可由独立暴力程序求解的小数据。 **能做什么：** 固定种子生成用例、运行正解、检测超时/非零返回码、按空白切分比较输出，并保存可复现失败输

入。 **输入 / 输出：** 被测可执行文件读取 n 和一行 n 个整数，输出最大连续子数组和；脚本参数为已编译程序路径和可选迭代次数，失败时写入 failed.in。 **复杂度：** 每轮暴力 $O(n^2)$ 加一次子进程启动和运行时间；错误答案缩小最多额外运行 $O(n^2)$ 次子进程（数组上限为 12）。

独立暴力实现不能复用正解的核心逻辑。对拍答案不一致时，脚本逐项尝试删除并保留仍能复现答案差异的用例；超时或非零退出直接保存原始输入与错误，不继续缩小。失败输入可单独复跑。随机测试可发现错误，不能代替证明。

```
import random
import subprocess
import sys
from pathlib import Path

# 参数：整数列表。返回：枚举所有非空子数组所得最大和。效果：无副作用，O(n^2)。
def brute(a):
    best = None
    for left in range(len(a)):
        total = 0
        for right in range(left, len(a)):
            total += a[right]
            best = total if best is None else max(best, total)
    return best

# 参数：已编译程序路径和整数列表。返回：(完整输入文本，stdout tokens，错误描述)。效果：启动子进程，超时 1 秒。
def run(program, a):
    data = f"{len(a)}\n{' '.join(map(str, a))}\n"
    try:
        proc = subprocess.run([program], input=data, text=True, capture_output=True,
timeout=1)
    except subprocess.TimeoutExpired:
        return data, [], "timeout"
    if proc.returncode != 0:
        return data, proc.stdout.split(), f"return code {proc.returncode}: {proc.stderr}"
    return data, proc.stdout.split(), ""

# 参数：程序路径、固定随机种子和轮数。返回：通过/失败状态。效果：失败时写 failed.in 与
failed.txt。
def stress(program, seed=20261009, rounds=300):
    rng = random.Random(seed)
    for case_id in range(rounds):
        n = rng.randint(1, 12)
        a = [rng.randint(-20, 20) for _ in range(n)]
        data, got, error = run(program, a)
        expected = str(brute(a))
        if error or got != [expected]:
            # 答案不一致时逐项删除并保留仍不一致的近似最小样例；超时/非零退出直接保存原始输
入。
            small = a[:]
            changed = True
            while not error and changed and len(small) > 1:
                changed = False
                for i in range(len(small)):
                    trial = small[:i] + small[i + 1:]
                    trial_data, trial_got, trial_error = run(program, trial)
                    if trial_error or trial_got != [str(brute(trial))]:
                        small, data, got, error = trial, trial_data, trial_got, trial_error
                        changed = True
                        break
```

```
            Path("failed.in").write_text(data)
            Path("failed.txt").write_text(
                f"seed={seed} case={case_id} input={small} expected={brute(small)} "
                f"actual={got} error={error}\n"
            )
            print("FAIL: see failed.in and failed.txt")
            return False
    print(f"PASS: {rounds} cases, seed={seed}")
    return True

if __name__ == "__main__":
    if len(sys.argv) < 2:
        raise SystemExit("usage: python3 stress.py ./solution [rounds]")
    ok = stress(sys.argv[1], rounds=int(sys.argv[2]) if len(sys.argv) > 2 else 300)
    raise SystemExit(0 if ok else 1)
```

常用 Python 调试/枚举工具：

工具	用法与边界
`assert condition, message`	检查内部不变量；优化模式可能移除断言，不要用它校验不可信输入
`print(repr(x), file=sys.stderr)`	显示空格、换行等不可见字符；不会污染被测程序 stdout
`breakpoint()` / `pdb`	在本地交互检查变量、单步执行；提交前移除断点
`itertools.product/permutations/combinations`	小规模枚举笛卡尔积、排列、组合；规模按指数/阶乘增长，先估算数量
整数 token 比较	输出为整数时用 `stdout.split()` 再逐 token 比较，忽略空白差异
`math.isclose(a,b, rel_tol=..., abs_tol=...)`	浮点答案用容差核对；容差按题目精度要求设定
多解构造题	单独写 validator 检查格式、范围、互异性和约束，不比较与参考答案完全相同

本节脚本只示范整数最大子段和，且子段必须非空，因此全负数组的答案是最大元素。实际题目应替换生成器、独立暴力和输出验证器。

2. STL 库使用

本章按竞赛中最常用的 C++17 功能组织，供选型与查错。n 表示元素数，区间均为左闭右开 [first,last)。示例统一包含 <bits/stdc++.h>（GNU C++17）；若使用标准库实现，应按代码补齐对应头文件。容器操作后按其失效规则重新取得迭代器。

2.1 顺序容器：vector 与 array

vector<T> 是连续存储的动态数组，适合默认的序列存储、随机访问和尾部增删。下标、首尾访问 O(1)，尾插均摊 O(1)，中间插删 O(n)。扩容会使所有迭代器、指针和引用失效；未扩容的尾插使 end() 失效。reserve(n) 只预留容量，resize(n) 才改变元素数。array<T,N> 是固定长度数组，元素连续，大小编译期确定，不支持增删；适合小型固定表。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector<int> a{4, 1, 3};
    a.reserve(8);                  // 预留容量，不改变 size
    a.push_back(2);
    sort(a.begin(), a.end());
    array<int, 3> fixed{1, 2, 3};
    cout << a[0] + fixed[2] << '\n';
}
```

常用成员：size()、empty()、front()、back() 均为 O(1)；pop_back() 删除末元素 O(1)，空时不可调用；clear() 清空元素但容量通常保留；erase(it) 删除单个元素并返回新位置，后缀移动 O(n)；swap(other) 交换内容/容量，通常 O(1)。erase 会使删除点及之后的迭代器失效，扩容会使全部失效。

操作	作用与复杂度	常见用途/边界
vector<T> a(n) / a(n,x)	创建 n 个默认值 / x，O(n)	创建后下标有效；空 vector 不可写 a[0]
push_back(x) / emplace_back(args...)	末尾插入，均摊 O(1)	emplace 原位构造；扩容时全体迭代器失效
reserve(n)	预留容量，不改变 size	估计规模时减少扩容；不能因此访问 a[n]
resize(n,x)	调整 size，新元素填 x；最坏 O(n)	需要真正创建可下标访问的槽位
insert(pos,x) / erase(pos)	中间修改 O(n)	pos 必须来自本 vector；返回新的迭代位置

避免用 vector<bool> 需要真实 bool& 或连续 bool 存储的场景；它是位压缩特化。二维 vector 常用于不规则邻接表；规则网格也可用一维下标 r * width + c 改善局部性。

2.2 string

string 是可变长字符序列，提供随机访问，常用于文本、数字串和字符状态。size、下标访问 O(1)，查找和拼接通常与处理字符数成正比。getline(cin,s) 读取一整行（此前若用 >>，先处理残留换行）；find(x) 返回首匹配位置，找不到返回 string::npos；substr(pos,len) 返回新字符串，越界起点会抛出异常。

stoi/stoll 将前缀转换为整数，非法输入抛异常，超范围也抛异常。修改导致扩容时指针、引用和迭代器可能失效。字符串下标是字节位置，不是 Unicode 字符位置。

操作	行为
s += t / s.append(t)	追加字符或字符串；总成本与追加长度相关
s.find(t,pos)	从 pos 起找子串；找不到是 string::npos，不可当作普通负数位置用
s.substr(pos,len)	复制子串；pos 必须不大于 size，len 可超出尾部并自动截断
stoi / stoll	将十进制前缀转 int/long long；可用位置参数确认是否消费了全部输入

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s = "contest";
    s += "!";
    auto pos = s.find("test");
    if (pos != string::npos) cout << s.substr(pos, 4) << '\n';
}
```

2.3 pair、tuple 与结构化绑定

pair<A,B> 表示两个关联值，常用于坐标、边（邻点，权值）和排序键；比较按 first 再按 second 字典序进行。tuple 可装多个异质字段，但字段命名不直观；字段较多且语义稳定时优先自定义 struct。结构化绑定要求分量数量匹配。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector<pair<int,int>> edges{{2, 7}, {1, 4}};
    sort(edges.begin(), edges.end()); // 先按 first，再按 second
    auto [to, weight] = edges.front();
    auto record = make_tuple(3, string("Ada"), true);
    cout << to << ' ' << weight << ' ' << get<1>(record) << '\n';
}
```

2.4 两端序列与适配器：deque、stack、queue

deque<T> 支持两端均摊 O(1) 插入删除及 O(1) 随机访问；适合单调队列和两端操作。它不是连续内存，不能把 data() 当作数组接口。两端插入通常不使已有元素引用失效，但迭代器规则较严格，结构修改后宜重取。

stack<T> 提供后进先出，只用 push/pop/top；queue<T> 提供先进先出，只用 push/pop/front/back。二者是容器适配器，pop 返回 void，应先读取元素再弹出。读取空容器的顶部或队首是错误。

适配器	读取	插入/删除	典型用途
stack	top()	push() / pop()	括号匹配、DFS 显式栈、单调栈
queue	front()、back()	push() / pop()	BFS、按到达顺序处理任务
priority_queue	top()	push() / pop()	每步取当前最小/最大候选

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    deque<int> d{2, 3}; d.push_front(1); d.pop_back();
    stack<int> st; st.push(5); int x = st.top(); st.pop();
    queue<int> q; q.push(x); cout << q.front() << '\n';
}
```

2.5 堆：priority_queue

`priority_queue<T>` 默认是最大堆：`top()` 为最大值，插入和删除堆顶 O(log n)，读取 O(1)。最小堆可用 `greater<T>`。适合反复取当前最大/最小值；若要删除任意元素或遍历有序全体，选择其他结构。自定义比较器需满足严格弱序。

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    priority_queue<int> maxHeap;
    priority_queue<int, vector<int>, greater<int>> minHeap;
    for (int x : {4, 1, 7}) { maxHeap.push(x); minHeap.push(x); }
    cout << maxHeap.top() << ' ' << minHeap.top() << '\n';
}
```

2.6 有序集合：set、multiset

`set<T>` 保存互异键，`multiset<T>` 保存可重复键；二者按比较器维护有序，查找、插入 O(log n)，按键删除 multiset 的全部 k 个匹配项为 O(log n+k)。适合需要动态维护有序元素并查前驱/后继的场景。迭代器为双向迭代器，`lower_bound(x)` 返回首个不小于 `x` 的元素，`upper_bound(x)` 返回首个大于 `x` 的元素；不存在时返回 `end()`。插删通常只使被删元素迭代器失效。

`set::erase(x)` 删除键为 x 的元素，`multiset` 中会删除所有等于 x 的元素；若只删一个重复项，先 `find` 再按迭代器删除。不要对集合迭代器调用 `sort`。

操作	set / multiset 行为
`insert(x)`	O(log n) 插入；set 忽略重复键，multiset 保留
`find(x)` / `count(x)`	查键 O(log n)；multiset 统计重复数 O(log n + k)，k 为重复个数
`lower_bound(x)` / `upper_bound(x)`	O(log n) 取首个 >=x / >x 的迭代器
`erase(it)` / `erase(x)`	按迭代器删一个 O(log n)；按键删除 multiset 全部匹配项 O(log n+k)

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    multiset<int> s{2, 2, 5};
    auto it = s.find(2);
    if (it != s.end()) s.erase(it); // 只删除一个 2
    auto next = s.lower_bound(3);
    if (next != s.end()) cout << *next << '\n';
}
```

2.7 有序映射：map

map<K,V> 按键有序，每个键唯一；查找、插入和删除 O(log n)。适合键域稀疏、需要有序遍历或前驱后继查询的映射。m[key] 不存在时会插入默认值；只读查询用 find，避免意外插入。迭代器按键升序，插入通常不使其失效，删除只使被删项迭代器失效。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    map<string, int> count;
    ++count["AC"];
    auto it = count.find("WA");
    if (it == count.end()) cout << "missing\n";
    for (const auto& [key, value] : count) cout << key << ':' << value << '\n';
}
```

常用成员：

操作	说明
find(key)	O(log n)，缺失时返回 end()；只读查询优先用它
at(key)	查值但不插入；键不存在时抛出 out_of_range
m[key]	键不存在时插入默认构造的值；适合频次累加
insert({key,value}) / emplace(key,value)	插入键值；重复键不会覆盖既有值
erase(key) / erase(it)	按键删除或删除迭代器所指项；后者只删一项
lower_bound / upper_bound	O(log n) 返回首个 >= key / > key 的迭代器

2.8 哈希容器：unordered_set、unordered_map

哈希容器平均查找、插入和删除 O(1)，最坏 O(n)，不维护顺序；适合只需按键快速查找且不要求有序遍历的场景。扩容 rehash 会使所有迭代器失效（引用和指针仍有效）；插入可能触发 rehash。可用 reserve 预留元素数减少扩容。键类型必须可哈希且支持相等比较。

操作	说明
find(key)	返回迭代器，缺失时为 end()；只读检查优先用它
count(key) / contains(key)	判存在；contains 从 C++20 才提供，C++17 用 find
reserve(n)	预计元素总量，减少 rehash；参数是元素数而非桶数
load_factor() / max_load_factor(x)	查看/调整平均装载因子；调参不能保证对抗恶意碰撞

自定义键需要提供哈希函数并保持“相等的键哈希值相同”。哈希不是安全性边界；攻击性输入可能退化。整数键可用随机种子的 splitmix64 扰动降低被构造碰撞的风险，但这不是密码学保证。

```
#include <bits/stdc++.h>
using namespace std;
struct SplitMixHash {
    // 参数：64 位整数 x。返回：种子扰动后的哈希值。效果：只读进程级随机种子。
    size_t operator()(uint64_t x) const noexcept {
        static const uint64_t seed = chrono::steady_clock::now().time_since_epoch().count();
        x += seed + 0x9e3779b97f4a7c15ULL;
```

```
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return static_cast<size_t>(x ^ (x >> 31));
    }
};
int main() {
    unordered_map<uint64_t, int, SplitMixHash> id;
    id.reserve(1000);
    id[123456789ULL] = 7;
    cout << id.at(123456789ULL) << '\n';
}
```

2.9 bitset

`bitset<N>` 在编译期固定 N 位，适合小规模布尔状态、集合位运算和快速计数。与 `vector<bool>` 不同，长度不能运行时改变。按位与或异或、取反和移位按机器字批量处理；`count()` 返回 1 的位数。下标 0 是最低位，移位超出范围的位被丢弃。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    bitset<8> a(string("00101101"));
    bitset<8> b = a << 1;
    cout << (a & b).count() << ' ' << b << '\n';
}
```

2.10 排序、二分与边界

`sort(first,last)` 要求随机访问迭代器，原地升序排序 $O(n \log n)$，不稳定；降序可传 `greater<>`，自定义记录用严格弱序比较器。`stable_sort` 保留等价元素原相对顺序，通常使用 $O(n)$ 额外空间。`lower_bound/upper_bound` 要求区间按相同比较规则有序，返回迭代器，比较次数 $O(\log n)$；对 `vector` 总体 $O(\log n)$，对 `set` 应调用成员 `lower_bound` 才能得到 $O(\log n)$。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector<int> a{4, 1, 2, 2};
    sort(a.begin(), a.end());
    auto lo = lower_bound(a.begin(), a.end(), 2);
    auto hi = upper_bound(a.begin(), a.end(), 2);
    cout << (lo - a.begin()) << ' ' << (hi - a.begin()) << '\n';
}
```

常用补充：`nth_element(first,nth,last)` 平均 $O(n)$，把第 `nth` 元素放在排序后应处的位置，两侧各自满足分界但内部无序；`min_element/max_element` 返回极值迭代器，空区间返回 `last`；`binary_search` 只返回是否存在，`equal_range` 返回 `[lower_bound,upper_bound)`。`set/map` 需要二分时调用成员 `lower_bound/upper_bound`，复杂度 $O(\log n)$；对其迭代器使用泛型二分会因逐步移动而退化。

常用算法速查（`n` 为区间长度）：

算法	前提与结果	复杂度 / 副作用
`sort` / `stable_sort`	随机访问区间；稳定版保留等价项相对次序	$O(n \log n)$，原地重排 / 稳定排序有足够内存时 $O(n \log n)$，内存不足时最坏 $O(n \log^2 n)$

算法	前提与结果	复杂度 / 副作用
`nth_element(first,nth,last)`	`nth` 处成为全序第 `nth` 个元素，两边只保证分界	平均 O(n)，重排；若 `nth==last`，不解引用该迭代器
`lower_bound` / `upper_bound`	已按同一比较器排序；分别给出 `>=x` / `>x` 的首位置	O(log n) 比较；vector 随机访问总体 O(log n)
`binary_search` / `equal_range`	有序区间；前者测存在，后者给出等值段	O(log n) 比较
`min_element` / `max_element`	返回极值迭代器，空区间返回 `last`	O(n)，不修改区间
`reverse` / `fill`	分别反转、写入同一个值	O(n)，原地修改
`count` / `count_if`	统计等值项 / 满足谓词项	O(n)，不修改区间
`copy` / `transform`	输出区须足够大；`back_inserter(out)` 可追加	O(n)，写目标区间
`unique`	只压缩相邻重复项，返回新逻辑末尾	O(n)，重排但不改变容器 size；常与 `erase` 配合

`sort` 自定义比较器必须满足严格弱序，例如按键排序时比较 `a.key < b.key`，不能用 `<=`。对于 `set/map`，泛型 `lower_bound` 迭代器移动可能为 O(n)，应使用容器成员的 O(log n) 查找。

2.11 去重、排列与区间算法

`unique(first,last)` 把相邻重复元素压到前部并返回新逻辑末尾，不会缩短容器；先排序才能全局去重，再用 `erase` 真正缩短。`next_permutation` 把序列改成字典序下一个排列；到最大排列时会重置为最小排列并返回 `false`，故从排序序列开始可枚举所有不同排列。`reverse` 原地反转双向迭代器范围 O(n)；`fill(first,last,value)` 用同一值覆盖范围 O(n)；`count`/`count_if` 分别统计等值项/谓词为真的项，均 O(n)。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector<int> a{3, 1, 1, 2};
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    do { for (int x : a) cout << x; cout << '\n'; }
    while (next_permutation(a.begin(), a.end()));
}
```

2.12 数值算法与 Lambda

`accumulate(first,last,init)` 左折叠区间，结果类型由 `init` 决定；求 `long long` 和应写 `0LL`。`iota` 用递增值填充区间。

函数	结果	提醒
`accumulate(first,last,init)`	左折叠 / 累加	返回类型由 init 决定；求 long long 和用 `0LL`
`partial_sum(first,last,out)`	输出每个前缀累计值	输出空间须足够；可用 `back_inserter` 追加
`iota(first,last,start)`	依次填 start、start+1…	需 `<numeric>`；常用于生成下标序列

函数	结果	提醒
`gcd(a,b)` / `lcm(a,b)`	C++17 求最大公约数 / 最小公倍数	lcm 乘法仍可能溢出

`gcd(a,b)` 求最大公约数，`lcm(a,b)` 求最小公倍数（C++17 `<numeric>`）；结果类型取决于参数类型，乘法中间值仍可能溢出。Lambda 可捕获局部变量并作为比较器或局部逻辑；按值捕获复制，按引用捕获会修改原变量且不可超出其生命周期。`[&]` 捕获所有使用到的局部变量引用，需确认调用期间对象仍存活。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector<int> a(5); iota(a.begin(), a.end(), 1);
    long long sum = accumulate(a.begin(), a.end(), 0LL);
    int limit = 3;
    int count = count_if(a.begin(), a.end(), [limit](int x) { return x <= limit; });
    cout << sum << ' ' << count << '\n';
}
```

2.13 迭代器与半开区间

迭代器连接容器和泛型算法，范围统一写 `[first,last)`；`end()` 是尾后哨兵，不可解引用。`begin()`/`end()` 获取边界通常 O(1)，空范围满足 `begin()==end()`。`vector`、`array`、`deque`、`string` 迭代器支持随机访问；`set`/`map` 为双向迭代器。`sort`、`nth_element` 要求随机访问；`reverse` 要求双向；`find`、`count` 可用于前向区间。

`it + k`、`last-first` 只适用于随机访问迭代器且区间须来自同一容器。`next(it,k)` 返回移动后的副本，`advance(it,k)` 原地移动；随机访问容器 O(1)，双向/前向容器按步数 O(k)。不要保存跨越容器结构修改仍未确认有效性的迭代器。

常见边界：所有标准算法按半开区间操作；`end()` 不可解引用；`erase`、扩容和 rehash 后检查迭代器有效性；`lower_bound` 前先确认排序规则一致；注意整数累加类型是否足够。

3. 基础算法与搜索

3.1 复杂度与枚举

适用范围： 枚举有限状态/子集；子集和用 O(2^n) 空间时间。输入 vector，输出所有子集和；位掩码只适用于 n 小且移位不溢出的范围。 **能做什么：** 枚举所有子集和，或列出一个掩码的全部非空子掩码。 **输入 / 输出：** 输入 0-based long long 数组或非负掩码；返回子集和数组或非空子掩码列表。 **复杂度：** O(2^n).

用途： 先估算可执行次数，再选择枚举对象：约 10^8 次简单操作通常已接近上限。直接枚举为 $O(n^k)$，位掩码枚举全部子集为 $O(2^n)$、空间 $O(1)$；只枚举固定大小组合可减少无效状态。适用于数据范围较小或作为对拍暴力。

思路：

- `mask` 的第 `i` 位表示是否选择下标 `i`，适合 `n <= 25` 左右的全子集枚举。
- 用 `mask & (1 << i)` 判断元素是否被选择；位数较多时改用 `1LL << i`。
- 枚举某个集合的非空子集时，使用 `(sub - 1) & mask`。
- 若只需固定大小的集合，可用 `__builtin_popcount(mask)` 筛选。

代码：

```
#include <bits/stdc++.h>
using namespace std;

// 枚举 n 个元素的全部子集，并计算每个子集的元素和。
// 参数：a 为 0-based 整数数组。返回：每个掩码对应的子集和，空集和为 0。效果：无副作用；要求
a.size() 可安全作为位掩码宽度。
vector<long long> enumerateSubsetSums(const vector<long long>& a) {
    int n = static_cast<int>(a.size());
    // a 使用 0-based；sums[mask] 表示恰好选择 mask 中元素的总和。
    vector<long long> sums(1ULL << n, 0);
    for (unsigned long long mask = 1; mask < (1ULL << n); ++mask) {
        // 取一个已选位，复用删去该位后的答案，避免每个子集重新扫描 a。
        int bit = __builtin_ctzll(mask);
        sums[mask] = sums[mask ^ (1ULL << bit)] + a[bit];
    }
    return sums;
}

// 枚举 mask 的所有非空子集。
// 参数：非负集合掩码 mask。返回：mask 的所有非空子掩码。效果：无副作用。
vector<int> nonemptySubmasks(int mask) {
    vector<int> result;
    // 每次删去 sub 的最低位并只保留 mask 中的位；0 不在返回结果中。
    for (int sub = mask; sub; sub = (sub - 1) & mask) {
        result.push_back(sub);
    }
    return result;
}
```

易错点：

1. `1 << n` 在 `n >= 31` 时可能溢出，应使用 `1ULL << n`。
2. 全部 `mask` 再枚举其全部子集的总复杂度是 $O(3^n)$，不是 $O(2^n)$。
3. 不要只看大 O；字符串复制、哈希和递归的常数明显更大。

参考：

- OI Wiki：枚举
- OI Wiki：复杂度

3.2 前缀和与差分

适用范围： 静态区间和与区间加差分；前缀查询 O(1)，差分恢复 O(n)。输入按约定下标数组及合法闭区间，输出区间和或恢复后的数组。 **能做什么：** 计算静态区间和、批量区间加后恢复数组、查询二维矩形和。 **输入 / 输出：** 1D 使用 1-based 闭区间；2D 使用 1-based 闭矩形；返回区间和或恢复数组。 **复杂度：** O(n) build / O(1) query.

用途： 前缀和把静态区间和查询化为两个前缀之差，预处理 $O(n)$、单次查询 $O(1)$、空间 $O(n)$。差分把离线区间加转为两个端点修改，单次修改 $O(1)$，最后 $O(n)$ 还原。若修改与查询交错，应改用树状数组或线段树。

思路：

- 一维区间 `[l,r]` 的和为 `prefix[r] - prefix[l - 1]`。
- 对 `[l,r]` 全部加 `value`：`diff[l] += value`，`diff[r+1] -= value`。
- 二维矩形使用左上、右下坐标，容斥四个二维前缀。
- 数值总和可能超过 `int`，数组统一使用 `long long`。

代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct PrefixSum1D {
    // 输入与查询均为 1-based；prefix[0] 是空前缀哨兵。
    vector<ll> prefix;

    // 参数：含 0 下标哨兵的 1-based 数组 a[1..n]。返回：构造前缀和对象。效果：O(n) 初始化，仅保存汇总。
    explicit PrefixSum1D(const vector<ll>& a) {
        int n = static_cast<int>(a.size()) - 1; // a[1..n]
        prefix.assign(n + 1, 0);
        for (int i = 1; i <= n; ++i) prefix[i] = prefix[i - 1] + a[i];
    }

    // 参数：1-based 闭区间 [left,right]。返回：该区间元素和。效果：只读；要求区间在构造数组范围内。
    ll query(int left, int right) const {
        // 返回闭区间 [left, right]，调用者需保证 1 <= left <= right <= n。
        return prefix[right] - prefix[left - 1];
    }
};

struct Difference1D {
    int n;
    vector<ll> diff;

    // 参数：含哨兵的 1-based 初始数组。返回：差分对象。效果：O(n) 初始化，后续 add 记录闭区间加法。
    explicit Difference1D(const vector<ll>& a)
```

```
        : n(static_cast<int>(a.size()) - 1), diff(n + 2, 0) {
        for (int i = 1; i <= n; ++i) diff[i] = a[i] - a[i - 1];
    }

    // 参数：1-based 闭区间及增量 value。返回：无。效果：记录区间加法，须在 build 前调用。
    void add(int left, int right, ll value) {
        // 闭区间 [left, right] 加 value；diff[n+1] 承接右端撤销标记。
        diff[left] += value;
        diff[right + 1] -= value;
    }

    // 参数：无。返回：应用所有已记录区间加法后的 1-based 数组。效果：只读，不清空差分状态。
    vector<ll> build() const {
        // 所有离线修改完成后再还原；result 仍为 1-based。
        vector<ll> result(n + 1, 0);
        for (int i = 1; i <= n; ++i) result[i] = result[i - 1] + diff[i];
        return result;
    }
};

struct PrefixSum2D {
    // a 的有效区域为 [1..n] x [1..m]，第 0 行和第 0 列是容斥哨兵。
    vector<vector<ll>> prefix;

    // 参数：1-based 矩阵，0 行/列作为哨兵。返回：构造二维前缀和对象。效果：O(nm) 初始化。
    explicit PrefixSum2D(const vector<vector<ll>>& a) {
        int n = static_cast<int>(a.size()) - 1;
        int m = static_cast<int>(a[0].size()) - 1;
        prefix.assign(n + 1, vector<ll>(m + 1, 0));
        for (int i = 1; i <= n; ++i) {
            for (int j = 1; j <= m; ++j) {
                prefix[i][j] = a[i][j] + prefix[i - 1][j]
                    + prefix[i][j - 1] - prefix[i - 1][j - 1];
            }
        }
    }

    // 参数：1-based 闭矩形 [x1..x2]×[y1..y2]。返回：矩形元素和。效果：只读。
    ll query(int x1, int y1, int x2, int y2) const {
        // 返回闭矩形 [x1..x2] x [y1..y2]，调用者需保证坐标顺序合法。
        return prefix[x2][y2] - prefix[x1 - 1][y2]
            - prefix[x2][y1 - 1] + prefix[x1 - 1][y1 - 1];
    }
};
```

易错点：

1. 预留 `0` 行、`0` 列以及 `n+1` 差分哨兵，避免边界特判。
2. 二维容斥最后要把重复减去的左上区域加回来。
3. 差分适用于“全部修改后统一还原”，不能直接回答中途查询。

参考：

- OI Wiki：前缀和与差分

3.3 二分查找与二分答案

适用范围： 已排序查找或单调判定函数的整数答案；二分 O(log 范围) 次判定。输入有效边界与单调谓词，输出首个/末个满足边界；无解须由调用方预先界定。 **能做什么：** 在排序序列找边界，或对单调谓词求首个/最后一个真值。 **输入 / 输出：** 输入有效边界和假转真/真转假的单调谓词；无解分别返回 right+1/left-1。 **复杂度：** O(log n) predicate checks.

用途： 在有序序列查找边界，或答案存在“左侧均可行、右侧均不可行”等单调性时使用。维护一真一假的边界，每次检查中点并舍弃一半区间。整数二分需 $O(\log R)$ 次检查、空间 $O(1)$；总复杂度还要乘 `check` 的复杂度。

思路：

- `firstTrue` 返回闭区间 `[left,right]` 中第一个满足 `check` 的整数。
- 调用前必须保证至少存在一个可行答案；否则自行增加无解判断。
- 求“最后一个可行值”可把判定取反，或使用对应的 `lastTrue`。
- 中点写成 `left + (right - left) / 2`，避免两端相加溢出。

代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

template <class Check>
// 参数：闭区间边界及单调谓词（假到真）。返回：首个为真的值；无解返回 right+1。效果：调用谓词 O(log范围) 次。
ll firstTrue(ll left, ll right, Check check) {
    // 闭区间内要求答案存在，且 check 呈 false...false,true...true。
    while (left < right) {
        // 向下取整；check(mid) 为真时 mid 仍可能是首个真值，不能丢弃。
        ll mid = left + (right - left) / 2;
        if (check(mid)) right = mid;
        else left = mid + 1;
    }
    return left;
}

template <class Check>
// 参数：闭区间边界及单调谓词（真到假）。返回：最后一个为真的值；无解返回 left-1。效果：调用谓词 O(log范围) 次。
ll lastTrue(ll left, ll right, Check check) {
    // 闭区间内要求答案存在，且 check 呈 true...true,false...false。
    while (left < right) {
        // 向上取整保证只剩两个数时 mid != left，否则真分支会死循环。
        ll mid = left + (right - left + 1) / 2;
        if (check(mid)) left = mid;
        else right = mid - 1;
    }
    return left;
}
```

易错点：

1. 二分答案的关键是判定函数单调，不能只因答案是数字就二分。
2. 求最后一个可行值时中点必须向上取整，否则可能死循环。

3. 浮点二分通常固定迭代次数；不要直接套整数的 `+1/-1`。

参考：

- OI Wiki：二分

3.4 双指针与滑动窗口

适用范围： 窗口可由双指针单向维护时适用；和阈值模板要求数组元素非负且 `target>0`。输入数组及目标，输出最长长度或最短长度；不存在时返回 0。 **能做什么：** 求最长互异子数组，或非负数组中和至少为目标的最短子数组。 **输入 / 输出：** 输入整数数组及 target；返回长度；最短窗口不存在时返回 0，`target<=0` 时非空数组返回 1、空数组返回 0。 **复杂度：** O(n).

用途： 当区间随右端扩张后，合法性关于左端具有单调性时，用左右指针维护连续区间；每个元素至多进出窗口一次，时间 $O(n)$、空间由维护信息决定。常用于非负数组和、至多含若干种元素、去重窗口；有负数时“和越长越大”不成立。

思路：

- 右指针每次加入一个元素，再不断移动左指针直到窗口合法。
- 求最长合法区间时，在收缩完成后更新答案。
- 求最短满足区间时，应在条件成立期间更新答案并收缩。
- 计数类窗口常用 `unordered_map` 或定长频次数组维护。

代码：

```
#include <bits/stdc++.h>
using namespace std;

// 最长无重复连续子数组长度，元素值无需预先离散化。
// 参数：整数数组。返回：元素互异的最长连续子数组长度。效果：无副作用，O(n)。
int longestDistinctSubarray(const vector<int>& a) {
    // a 使用 0-based；每轮收缩后窗口 [left, right] 始终不含重复值。
    unordered_map<int, int> count;
    int answer = 0;
    for (int left = 0, right = 0; right < static_cast<int>(a.size()); ++right) {
        ++count[a[right]];
        // 新加入值重复时持续移走左端，直到窗口重新合法。
        while (count[a[right]] > 1) {
            if (--count[a[left]] == 0) count.erase(a[left]);
            ++left;
        }
        answer = max(answer, right - left + 1);
    }
    return answer;
}

// 非负数组中，和至少为 target 的最短非空连续子数组长度；不存在返回 0。
// 参数：非负数组 a 与目标和 target。返回：和至少为 target 的最短非空子数组长度；不存在返回
0。效果：无副作用，O(n)。
int minLengthAtLeast(const vector<long long>& a, long long target) {
    if (target <= 0) return a.empty() ? 0 : 1;
    // 仅适用于非负数组；非负性保证收缩左端不会漏过更短可行窗口。
    long long sum = 0;
    int answer = static_cast<int>(a.size()) + 1;
    for (int left = 0, right = 0; right < static_cast<int>(a.size()); ++right) {
        sum += a[right];
```

```
        // 条件成立时先记录当前闭区间，再移走左端继续尝试缩短。
        while (sum >= target) {
            answer = min(answer, right - left + 1);
            sum -= a[left++];
        }
    }
    return answer == static_cast<int>(a.size()) + 1 ? 0 : answer;
}
```

易错点：

1. 必须证明移动某一端不会错过答案，即窗口条件具有单调性。
2. `while` 收缩和 `if` 收缩含义不同，多数窗口需要连续收缩。
3. 用哈希表计数时，计数归零后及时删除，避免种类数判断错误。

参考：

- OI Wiki：双指针

3.5 离散化

适用范围： 把可比较值映射为保持大小次序的紧凑整数编号；排序 O(n log n)。输入原值，输出 0-based 编号/去重值表；仅适用于查询构造时收录过的值；对未收录值，`id` 返回插入位置而非查找失败信号。 **能做什么：** 把离线值映射为排序去重后的紧凑编号。 **输入 / 输出：** 构造时输入全体候选值；id 仅要求返回 0-based 插入位置，value(id) 要求有效编号，size 返回不同值数。 **复杂度：** O(n log n).

用途： 只关心数值的大小、相等关系，而原值范围很大时，将出现过的值排序去重后映射到连续编号。建表 $O(n\log n)$，每次查询 $O(\log n)$，空间 $O(n)$。适用于树状数组、线段树和扫描线；若数值间距离有意义，不能直接用名次替代。

思路：

- 收集所有可能参与查询、修改的坐标，再统一排序去重。
- `lower_bound` 得到从 `0` 开始的编号；需要 1-based 时再加一。
- 可用 `values[id]` 从编号恢复原值。
- 动态出现且无法预收集的值，应考虑平衡树或动态线段树。

代码：

```
#include <bits/stdc++.h>
using namespace std;

template <class T>
struct Discretizer {
    // values[id] 保存第 id 小的不同原值；编号统一为 0-based。
    vector<T> values;

    // 参数：所有将参与查询的原值。返回：初始化后的离散化表。效果：排序并去重，O(n log n)。
    // 参数：所有后续查询可能出现的值。返回：初始化对象。效果：排序去重，O(n log n)。
    explicit Discretizer(vector<T> allValues) : values(move(allValues)) {
        sort(values.begin(), values.end());
        // unique 只压缩有效前缀；erase 同步缩小编号域。
        values.erase(unique(values.begin(), values.end()), values.end());
    }

    // 参数：原值。返回：排序去重后的 0-based 插入位置；调用者应保证该值已收录。效果：只读。
```

```
    int id(const T& value) const {
        // value 应在建表输入中出现；否则返回的只是它的插入位置。
        return static_cast<int>(lower_bound(values.begin(), values.end(), value)
                                - values.begin());
    }

    // 参数：有效 0-based 编号。返回：对应原值常量引用。效果：只读。
    const T& value(int id) const {
        return values[id];
    }

    // 参数：无。返回：去重值数量。效果：只读，O(1)。
    int size() const {
        return static_cast<int>(values.size());
    }
};
```

易错点：

1. 忘记 `unique` 后的 `erase` 会使重复值获得错误的空间规模。
2. 线上读到新坐标后再加入，会导致已有编号整体变化。
3. 区间长度不能用离散编号之差代替，应使用原坐标之差。

参考：

- OI Wiki：离散化

3.6 分治与归并计数

适用范围： 可将问题拆为两个子区间并合并时适用；归并 O(n log n)。输入可随机访问数组，输出排序结果或逆序对数。 **能做什么：** 归并排序并统计逆序对。 **输入 / 输出：** 输入数组及半开区间；返回逆序对数并原地排序相应范围。 **复杂度：** O(n log n).

用途： 分治把问题拆成互不重叠的同类子问题，递归求解后合并。典型归并排序与逆序对计数：左右两半已排序时，若右元素更小，它与左侧剩余元素都构成逆序对。时间 $O(n\log n)$，辅助空间 $O(n)$，递归栈 $O(\log n)$。

思路：

- 调用 `countInversions(a)` 后，数组会被排序并返回逆序对数量。
- 逆序对可能达到 $n(n-1)/2$，返回值使用 `long long`。
- 合并阶段维持左右区间已排序这一不变量。
- 其他分治题需明确“如何拆分”和“如何在线性或较低复杂度内合并”。

代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// 参数：数组、同长临时区及半开下标区间 [left,right)。返回：该区间逆序对数。效果：原地排序
a[left..right)，递归 O(n log n)。
ll mergeCount(vector<ll>& a, vector<ll>& temp, int left, int right) {
    // 全函数统一使用左闭右开区间 [left, right)。
    if (right - left <= 1) return 0;
```

```
    int mid = left + (right - left) / 2;
    ll answer = mergeCount(a, temp, left, mid)
              + mergeCount(a, temp, mid, right);

    int i = left, j = mid, k = left;
    // 不变量：两半已排序，temp[left..k) 已是合并后的最小前缀。
    while (i < mid && j < right) {
        if (a[i] <= a[j]) {
            temp[k++] = a[i++];
        } else {
            temp[k++] = a[j++];
            // 当前右值小于左半当前值，也小于左半剩余的全部 mid-i 个值。
            answer += mid - i;
        }
    }
    while (i < mid) temp[k++] = a[i++];
    while (j < right) temp[k++] = a[j++];
    copy(temp.begin() + left, temp.begin() + right, a.begin() + left);
    return answer;
}

// 参数：可修改整数数组。返回：逆序对数量。效果：将 a 升序重排，O(n log n)。
ll countInversions(vector<ll>& a) {
    vector<ll> temp(a.size());
    return mergeCount(a, temp, 0, static_cast<int>(a.size()));
}
```

易错点：

1. 本模板采用左闭右开区间 `[left,right)`，递归和合并边界必须一致。
2. 相等元素不是逆序对，比较应写 `<=`。
3. 若每层合并都新建大数组，额外分配会增大常数。

参考：

- OI Wiki：递归与分治

3.7 贪心与区间调度

适用范围： 半开区间 `[left,right)` 的无权最大不相交选择；相接端点允许兼选。输入区间及调用方提供的原始 id，输出所选原始 id。复杂度 O(n log n)。 **能做什么：** 按结束位置贪心选择最多个互不重叠区间。 **输入 / 输出：** 输入 `Interval{left,right,id}` 列表；返回被选区间原始 `id`，空输入返回空向量。 **复杂度：** O(n log n).

用途： 贪心每步选择当前看来最优且可证明不损失全局最优的决策。典型最多不相交区间：按右端点升序，依次选择起点不早于上个终点的区间。排序 $O(n\log n)$、扫描 $O(n)$、空间 $O(n)$。套用前需用交换论证或反证证明选择安全。

思路：

- 本模板把区间视为半开区间 `[left,right)`，端点相接允许同时选择。
- 返回被选区间在原输入中的下标，便于恢复方案。
- 若题意为闭区间且端点重合算相交，把条件改成 `left > lastRight`。
- 带权区间不能用此贪心，应使用按终点排序的动态规划。

代码：

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Interval {
    long long left, right;
    int id;
};

// 参数：半开区间列表，id 由调用者提供。返回：所选区间的原始 id。效果：复制并排序输入，O(n log n)。
vector<int> maximumNonOverlapping(vector<Interval> intervals) {
    // 输入按半开区间 [left, right)；返回原输入 id，而不是排序后位置。
    // 比较器参数：两个区间。返回：先按 right、再按 left 升序的严格次序；无外部副作用。
    sort(intervals.begin(), intervals.end(), [](const Interval& a, const Interval& b) {
        if (a.right != b.right) return a.right < b.right;
        return a.left < b.left; // 同终点时固定次序，便于复现与检查
    });

    vector<int> chosen;
    long long lastRight = numeric_limits<long long>::lowest();
    for (const auto& interval : intervals) {
        // 选择右端最早的可行区间，为后续选择留下最大空间。
        if (interval.left >= lastRight) {
            chosen.push_back(interval.id);
            lastRight = interval.right;
        }
    }
    return chosen;
}
```

易错点：

1. “看起来合理”不是证明；不同目标可能需要按左端点、长度或其他关键字排序。
2. 明确区间开闭，否则端点相等时是否冲突会写反。
3. 最多数量与最大权值是不同问题，后者通常需要 DP。

参考：

- OI Wiki：贪心

3.8 DFS 深度优先搜索

适用范围： 图遍历、可达性和连通分量；邻接表 DFS O(V+E)。输入 1..n 编号图和起点，输出访问序或分量；无向图分量代码按无向邻接表使用。 **能做什么：** 从起点 DFS 遍历及无向图连通分量划分。 **输入 / 输出：** 输入 1..n 邻接表和起点；返回访问序或分量列表。 **复杂度：** O(V+E).

用途： DFS 沿一条分支走到底再回退，适合连通块、可达性、环检测及需要回溯的状态搜索。邻接表图遍历时间 $O(n+m)$、空间 $O(n+m)$，递归栈最坏 $O(n)$。一般图必须记录访问状态；深链可能爆栈，可改显式栈。

思路：

- `dfsFrom(start)` 返回起点所在连通块的访问顺序。
- 无向图应加入双向边；有向图只加入题目给出的方向。
- 搜整张图时，对所有未访问点分别调用 DFS，可得到连通块或 DFS 森林。
- 需要退出时刻、子树信息时，在递归调用返回后处理。

代码：

```
#include <bits/stdc++.h>
using namespace std;

// 参数：1..n 编号图且 1<=start<=n。返回：从起点可达点的 DFS 顺序。效果：只读图，O(V+E)。
vector<int> dfsFrom(int start, const vector<vector<int>>& graph) {
    // graph 与节点均为 1-based；环、重边与边方向由建图方式决定。
    int n = static_cast<int>(graph.size()) - 1;
    vector<char> visited(n + 1, false);
    vector<int> order;

    // 参数：当前顶点 u。返回：无。效果：递归标记可达点并追加到外层 order。
    function<void(int)> dfs = [&](int u) {
        // 必须在递归邻点前标记，防止沿环或重边再次进入 u。
        visited[u] = true;
        order.push_back(u);
        for (int v : graph[u]) {
            if (!visited[v]) dfs(v);
        }
    };

    dfs(start);
    return order;
}

// 参数：无向图邻接表，点编号 1..n。返回：各连通分量的点集。效果：只读图，O(V+E)。
vector<vector<int>> connectedComponents(const vector<vector<int>>& graph) {
    // 仅用于无向图连通分量；有向图不能据此求强连通或弱连通。
    int n = static_cast<int>(graph.size()) - 1;
    vector<char> visited(n + 1, false);
    vector<vector<int>> components;
    for (int start = 1; start <= n; ++start) {
        if (visited[start]) continue;
        components.push_back({});
        stack<int> st;
        st.push(start);
        // 显式栈同样在入栈时标记，保证每个点最多入栈一次。
        visited[start] = true;
        while (!st.empty()) {
            int u = st.top();
            st.pop();
            components.back().push_back(u);
            for (int v : graph[u]) {
                if (!visited[v]) {
                    visited[v] = true;
                    st.push(v);
                }
            }
        }
    }
    return components;
}
```

易错点：

1. 一般图进入节点时就要标记，返回时才标记会在环上无限递归。
2. 递归深度接近 10^5 时有爆栈风险。
3. 有向图的“弱连通”不能直接用原图 DFS 得到。

参考：

- OI Wiki：DFS（图论）
- OI Wiki：DFS（搜索）

3.9 BFS 广度优先搜索

适用范围： 无权图最短路；BFS O(V+E)。输入邻接表和起点，输出距离/父节点；不可达距离为 -1，路径恢复对不可达点返回空。 **能做什么：** 求无权图单源最短距离并恢复路径。 **输入 / 输出：** 输入 1..n 邻接表和源/目标；不可达距离为 -1，路径恢复返回空向量。 **复杂度：** O(V+E).

用途： BFS 用队列按层扩展状态，在每条边代价相同的图中可求起点到各点的最少边数，并记录前驱还原路径。邻接表实现时间 $O(n+m)$、空间 $O(n+m)$。边权不全相等时不能直接使用；权值仅为 0/1 时可用双端队列优化。

思路：

- `distance[v] == -1` 表示不可达，首次入队时记录距离和前驱。
- 无向边加入两次，有向边只加入一次。
- 从多个起点求最近距离时，把所有起点以距离 0 同时入队。
- 用 `restorePath` 根据 `parent` 还原起点到终点的最短路径。

代码：

```
#include <bits/stdc++.h>
using namespace std;

struct BFSResult {
    vector<int> distance;
    vector<int> parent;
};

// 参数：1-based 单位边权图及一个或多个源点。返回：每点最短边数与父节点；不可达距离为 -1。效果：O(V+E)，只读图。
BFSResult bfs(const vector<vector<int>>& graph, const vector<int>& sources) {
    // graph 与节点为 1-based；仅保证单位边权图上的最少边数。
    int n = static_cast<int>(graph.size()) - 1;
    vector<int> distance(n + 1, -1), parent(n + 1, -1);
    queue<int> q;
    for (int source : sources) {
        if (distance[source] != -1) continue;
        // 所有源点同时处于第 0 层；重复源点只入队一次。
        distance[source] = 0;
        q.push(source);
    }

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : graph[u]) {
            // 首次入队即得到最短距离并标记，避免同层其他点重复入队。
            if (distance[v] != -1) continue;
            distance[v] = distance[u] + 1;
            parent[v] = u;
            q.push(v);
        }
    }
```

```
    return {distance, parent};
}

// 参数：目标点及 BFS 的 parent/distance 结果。返回：起点到目标的点序列；不可达返回空。效果：只读结果，O(路径长度)。
vector<int> restorePath(int target, const BFSResult& result) {
    // 不可达返回空数组；否则沿 parent 回到某个源点并反转为正向路径。
    if (result.distance[target] == -1) return {};
    vector<int> path;
    for (int v = target; v != -1; v = result.parent[v]) path.push_back(v);
    reverse(path.begin(), path.end());
    return path;
}
```

易错点：

1. 节点应在入队时标记；出队才标记会导致重复入队。
2. 普通 BFS 只保证“边数最少”，不保证任意带权图的距离最短。
3. 多源 BFS 的所有源点都应初始化为距离 0。

参考：

- OI Wiki：BFS（搜索）
- OI Wiki：BFS（图论）

3.10 回溯与剪枝

适用范围： 解空间可逐步构造且能安全剪枝时适用；最坏指数时间。输入元素与目标，输出一个可行子集或空结果；剪枝条件必须不排除有效解。 **能做什么：** 用回溯搜索一个满足目标和的子集。 **输入 / 输出：** 输入非负数组与目标；返回一个解的元素值，未找到返回空；剪枝基于非负性。 **复杂度：** 指数级最坏情况.

用途： 回溯按决策树枚举方案，尝试一个选择、递归、再撤销；剪枝提前排除不可能或不优的分支。最坏仍为指数或阶乘复杂度，空间通常为递归深度。适合规模小的排列、组合和约束满足；应优先安排限制强的决策并设计可证明安全的剪枝。

思路：

- 模板示例从输入元素中选取一个子集使和为 `target`，每个位置至多选一次，找到一组即停止。
- 仅当所有数非负时，`sum > target` 和“剩余和不足”剪枝才成立。
- 先排序，使相同数字相邻，从而跳过同层重复选择。
- 若要枚举全部方案，找到答案后保存并继续搜索，不要立即返回。

代码：

```
#include <bits/stdc++.h>
using namespace std;

// 找到一个和恰为 target 的子集，返回其元素；要求 a 中元素非负。
// 参数：整数数组与目标和。返回：一组和为 target 的元素值；无解返回空。效果：复制输入并指数搜索；适用非负数剪枝条件。
vector<int> subsetSumBacktracking(vector<int> a, long long target) {
    // a 必须全为非负数；返回任意可行子集，空数组也可能是合法答案。
    sort(a.rbegin(), a.rend()); // 大数优先，通常更早触发剪枝
    int n = static_cast<int>(a.size());
    vector<long long> suffix(n + 1, 0);
    // suffix[i] 是尚未考虑的 a[i..n-1] 总和，用于剩余和不足剪枝。
```

```
    for (int i = n - 1; i >= 0; --i) suffix[i] = suffix[i + 1] + a[i];

    vector<int> path, answer;
    // 参数：下一候选下标与当前和。返回：是否找到解。效果：修改外层 path/answer，成功时保留答案。
    function<bool(int, long long)> dfs = [&](int index, long long sum) {
        if (sum == target) {
            answer = path;
            return true;
        }
        // 后两个剪枝依赖元素非负；允许负数时必须删除。
        if (index == n || sum > target || sum + suffix[index] < target) return false;

        for (int i = index; i < n; ++i) {
            // 只跳过同层重复值，不影响下一层选择另一个相同元素。
            if (i > index && a[i] == a[i - 1]) continue;
            path.push_back(a[i]);
            if (dfs(i + 1, sum + a[i])) return true;
            // 当前选择失败，必须恢复 path 后才能尝试同层下一个选择。
            path.pop_back();
        }
        return false;
    };

    dfs(0, 0);
    return answer;
}
```

易错点：

1. 递归返回后必须完整撤销对路径、计数器和标记数组的修改。
2. 去重条件是“同一递归层跳过相同选择”，不能误删不同位置的合法使用。
3. 不满足前提的剪枝会漏解，例如数组含负数时不能用当前和超界剪枝。

参考：

- OI Wiki：回溯法
- OI Wiki：剪枝

3.11 折半搜索

适用范围： n 约 40、元素非负且上限 `limit>=0` 的子集和最大化；两半枚举约 O(2^(n/2)) 空间。 **能做什么：** 两半枚举合并，求不超过上限的最大子集和。 **输入 / 输出：** 输入 long long 数组及非负上限；返回不超过上限的最大子集和，允许空集，故总有解且答案至少为 0。 **复杂度：** 时间 O(n·2^(n/2))，空间 O(2^(n/2))。

用途： 折半搜索把 n 个决策分成两半，各自枚举约 $2^{n/2}$ 个状态，再排序、二分或双指针合并。适合 $n \approx 40$ 的子集和、计数和最优化；时间通常 $O(2^{n/2}n)$，空间 $O(2^{n/2})$。它以额外内存换取指数规模的平方根级下降。

思路：

- `maximumSubsetSumNotExceeding` 求不超过 `limit` 的最大子集和。
- 模板要求元素非负且 `limit>=0`；因此空集总是可行。若允许负数，仍可枚举，但剪枝和返回约定需调整。
- 两半分别生成子集和，右半排序后对每个左半和做 `upper_bound`。

- 若要求方案，子集和需同时保存对应掩码。

代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// 参数：数组与半开下标区间 [left,right)。返回：该半区间所有子集和。效果：无副作用，空间 O(2^k)。
vector<ll> allSubsetSums(const vector<ll>& a, int left, int right) {
    // a 使用 0-based，枚举左闭右开区间 [left, right) 的全部子集和。
    vector<ll> sums{0};
    for (int i = left; i < right; ++i) {
        int oldSize = static_cast<int>(sums.size());
        // 只遍历扩展前的旧状态，否则本轮元素会被重复选择。
        for (int j = 0; j < oldSize; ++j) sums.push_back(sums[j] + a[i]);
    }
    return sums;
}

// 参数：非负整数数组及 limit>=0。返回：不超过 limit 的最大子集和，允许空集故结果至少为 0。效果：O(2^(n/2) log 2^(n/2))。
ll maximumSubsetSumNotExceeding(const vector<ll>& a, ll limit) {
    // 元素要求非负；返回不超过 limit 的最大子集和，默认空集给出 0。
    int mid = static_cast<int>(a.size()) / 2;
    vector<ll> leftSums = allSubsetSums(a, 0, mid);
    vector<ll> rightSums = allSubsetSums(a, mid, static_cast<int>(a.size()));
    sort(rightSums.begin(), rightSums.end());

    ll answer = 0;
    for (ll sum : leftSums) {
        if (sum > limit) continue;
        // upper_bound 指向首个过大值，其前驱才是可配对的最大右半和。
        auto it = upper_bound(rightSums.begin(), rightSums.end(), limit - sum);
        if (it != rightSums.begin()) answer = max(answer, sum + *prev(it));
    }
    return answer;
}
```

易错点：

1. `2^(n/2)` 个和可能占用大量内存，元素和也可能溢出 `int`。
2. `upper_bound(x)` 指向第一个大于 `x` 的值，取可行值前必须先判断是否为 `begin()`。
3. 分组范围统一使用左闭右开，避免漏掉或重复中点元素。

参考：

- OI Wiki：双向搜索 / Meet in the middle

3.12 0-1 BFS

适用范围： 边权只为 0 或 1 的最短路；双端队列 O(V+E)。输入 1..n 编号图和起点，输出距离；不可达值为 INF。 **能做什么：** 在 0/1 权图求单源最短距离。 **输入 / 输出：** 输入 n、1..n 编号 0/1 权图及源点；返回距离数组，不可达为 INF。 **复杂度：** O(V+E).

用途与复杂度。 边权只取 0 或 1 的单源最短路可用双端队列在线处理：松弛到 0 边的新点放队首，1 边放队尾。时间 $O(n+m)$，空间 $O(n)$；一般非负边权应使用 Dijkstra。

思路与用法。 邻接表按 1-based 点编号，权值必须是 0 或 1。队列允许同一顶点多次入队；每次弹出时用当前最短距离松弛邻边。

代码：

```
#include <bits/stdc++.h>
using namespace std;

struct Edge01 {
    int to;
    int weight; // 必须为 0 或 1
};

// 参数：1..n 点的 0/1 权有向图与起点。返回：最短距离，未达点为 INF。效果：只读图，O(V+E)。
vector<int> zeroOneBfs(int n, const vector<vector<Edge01>>& graph, int source) {
    const int INF = numeric_limits<int>::max() / 4;
    vector<int> dist(n + 1, INF); // 点编号 1..n；不可达保持 INF
    deque<int> work;
    dist[source] = 0;
    work.push_front(source);

    while (!work.empty()) {
        int u = work.front();
        work.pop_front();
        for (const Edge01& edge : graph[u]) {
            int candidate = dist[u] + edge.weight;
            if (candidate >= dist[edge.to]) continue;
            dist[edge.to] = candidate;
            // 0 权边不增加距起点的层数，优先处理；1 权边排在后面。
            if (edge.weight == 0) work.push_front(edge.to);
            else work.push_back(edge.to);
        }
    }
    return dist;
}
```

易错点。 权值出现 2 或负数时该策略不再保证最短路；只在松弛成功后入队；不要在入队时永久标记 visited，因为后续可能找到更短距离。

3.13 常用构造题思路

适用范围： 输出一个满足条件的对象，且可从不变量、奇偶性、极值或递推关系直接拼出候选解。 **能做什么：** 用奇偶/模类分类、排序交错、分组配对、反向构造或归纳递推设计答案；图构造先核对度数和连通性。 **输入 / 输出：** 先明确实例参数和可行条件，再输出构造对象；本节代码构造 n 个互异正整数，使其和为 S。 **复杂度：** 示例 O(n) 时间、O(n) 空间。

常见构造策略应对应可验证的题面信号：

思路	适用信号与小例	限制与检查
不变量（奇偶/模类）	每步同时将两个数各加 1，总和增加 2，所以总和奇偶不变；若初始总和为偶数，目标奇数不可达	只给出不可能性判定，需从操作定义证明不变量确实保持

思路	适用信号与小例	限制与检查
奇偶分组	需排列 1..n 且避免相邻差 1；n>=4 时先放偶数再放奇数并检查两个拼接边界	n<4 单独处理；仍须逐项检查相邻差
分组配对	需让配对和相同；将 1..2k 配为 (i, 2k+1-i)，每对和为 2k+1	检查每个数恰用一次；额外范围/配对限制需另证
反向构造	目标最后一步容易确定；从目标状态删除一个元素倒推前态	倒推必须终止且每步可逆/保持可行
归纳递推	构造树可从单点开始，每次添加一个新顶点并连一条边；连通且无环保持，边数始终为顶点数减一	该扩展构造只覆盖树；额外度数限制需单独验证
图的度数/连通性	题目要求给定度数序列或连通图；先检查度数和为偶数	度数和偶数仅为必要条件；还需验证度数上界及连通性

这些方法依赖具体限制，不能只凭模式猜测；给出不变量证明，并对小规模枚举核验。

例：构造 n>=1 个互不相同的正整数且总和为 S。可行当且仅当 `S >= n(n+1)/2`（假设 n>=1）。从 `[1,2,...,n]` 开始，把多余量全部加到最大元素；它仍大于其余元素，因此互异性保持。下面的验算枚举小 n、S，检查可行性和输出性质。

```cpp
#include <bits/stdc++.h>
using namespace std;

// 参数：正整数个数 n 与目标和 S。返回：构造数组；不可行返回空。效果：无副作用，O(n)。
vector<long long> constructDistinctPositive(int n, long long sum) {
    if (n <= 0 || sum < 1LL * n * (n + 1LL) / 2) return {};
    vector<long long> a(n);
    iota(a.begin(), a.end(), 1LL);
    a.back() += sum - 1LL * n * (n + 1LL) / 2;
    return a;
}

// 参数：无。返回：进程状态 0 表示所有小规模构造验算通过。效果：断言检查可行性、互异、正数和总和。
int main() {
    for (int n = 1; n <= 8; ++n) {
        for (long long sum = 1; sum <= 80; ++sum) {
            auto a = constructDistinctPositive(n, sum);
            bool possible = sum >= 1LL * n * (n + 1LL) / 2;
            assert(static_cast<int>(a.size()) == (possible ? n : 0));
            if (possible) {
                set<long long> unique(a.begin(), a.end());
                assert(static_cast<int>(unique.size()) == n);
                assert(*min_element(a.begin(), a.end()) > 0);
                assert(accumulate(a.begin(), a.end(), 0LL) == sum);
            }
        }
    }
}
```

4. 动态规划

4.1 DP 入门：状态、转移与顺序

适用范围： 存在按依赖顺序计算的状态时适用；复杂度为状态数乘转移数。输入状态定义与初值，输出目标状态值；必须说明不可达初值及计算顺序。 **能做什么：** 计算不相邻选数的最大和并恢复一组下标。 **输入 / 输出：** 输入 value[1..n]；返回 maximumSum 与 1-based chosenIndices，允许空选，故全负答案为 0。 **复杂度：** 状态数 × 转移数.

用途： 动态规划适合具有最优子结构和重复子问题的计数、可行性或最优化问题。先定义状态含义，再写转移、初值和计算顺序；每个状态只算一次。若状态数为 S、每个状态枚举 T 个转移，时间 $O(ST)$、空间 $O(S)$，可按依赖关系滚动压缩。

思路：

- 明确写出 `dp[i]` 表示什么，未定义状态不要参与转移。
- 初值对应最小子问题；求最小值时其余状态常设为无穷大。
- 递推顺序必须保证依赖状态已算出；不易确定时可先写记忆化搜索。
- 模板示例为线上选数：选出的下标不能相邻，允许一个都不选。

代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct LineDPResult {
    ll maximumSum;
    vector<int> chosenIndices; // 1-based
};

// 参数：value[1..n] 的 long long 权值。返回：最大和与一组 1-based 互不相邻下标；允许空选。
效果：O(n) 时间、空间。
LineDPResult maximumNonAdjacentSum(const vector<ll>& value) {
    // value 使用 1-based；dp[i] 表示前 i 个位置允许空选时的最大和。
    int n = static_cast<int>(value.size()) - 1; // value[1..n]
    vector<ll> dp(n + 1, 0);
    if (n >= 1) dp[1] = max(0LL, value[1]);
    for (int i = 2; i <= n; ++i) {
        // 不选 i 继承 dp[i-1]；选 i 时前一个可选前缀只能到 i-2。
        dp[i] = max(dp[i - 1], dp[i - 2] + value[i]);
    }

    vector<int> chosen;
    // 从 dp[n] 逆推一组最优下标；相等时固定选择“不取 i”。
    for (int i = n; i >= 1;) {
        if (i == 1) {
            if (dp[i] > 0) chosen.push_back(1);
            break;
        }
        if (dp[i] == dp[i - 1]) {
            --i;
        } else {
            chosen.push_back(i);
            i -= 2;
```

```
        }
    }
    reverse(chosen.begin(), chosen.end());
    return {dp[n], chosen};
}
```

易错点：

1. 状态含义必须固定，例如“前 `i` 个”与“恰好以 `i` 结尾”不可混用。
2. 求最小值时未到达状态不能初始化为 0，否则会产生虚假转移。
3. 空间压缩后往往无法直接恢复方案，需要保留前驱或完整 DP 表。

参考：

- OI Wiki：动态规划基础
- OI Wiki：记忆化搜索

4.2 0-1 背包

适用范围： 每件物品最多取一次的容量优化；时间 O(nC)。输入重量/价值与容量 C，输出最大价值；非负重量且 C 可分配，压缩数组时容量倒序。 **能做什么：** 求容量不超过 C 的 0-1 背包值，或可选 exact 版本求恰好装满。 **输入 / 输出：** 输入平行的 0-based 重量/价值数组及容量；返回最大价值；exact 版无法恰好装满时返回空 optional。 **复杂度：** O(nC).

用途： 有 n 件物品，每件至多选一次，求容量不超过 W 时的最大价值。令 `dp[j]` 为容量上限为 `j` 的最大价值；处理物品时容量必须倒序，避免同一物品重复使用。时间 $O(nW)$、空间 $O(W)$，适合容量不大且重量为非负整数的题目。

思路：

- `weight[i]`、`value[i]` 均按 `0..n-1` 存储。
- 本模板求“总重量不超过容量”的最大价值，允许不装满。
- 若要求恰好装满，除 `dp[0]=0` 外应初始化为负无穷，并跳过不可达状态。
- 只问可行性时可用 `vector<char>`，转移为逻辑或。

代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// 参数：平行的重量/价值数组与容量；每件至多取一次。返回：不超过容量的最大价值。效果：
O(nC)，要求容量非负。
ll zeroOneKnapsack(const vector<int>& weight,
                   const vector<ll>& value,
                   int capacity) {
    // 物品为 0-based；dp[j] 表示总重量不超过 j 的最大价值。
    vector<ll> dp(capacity + 1, 0);
    for (int i = 0; i < static_cast<int>(weight.size()); ++i) {
        // 容量倒序保证右侧 dp[j-weight[i]] 尚未使用当前物品。
        for (int j = capacity; j >= weight[i]; --j) {
            dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
        }
    }
    return dp[capacity];
```

```
}

// 恰好装满时的最大价值；不可达返回 nullopt。
// 参数：平行的重量/价值数组及非负容量；每件至多取一次。返回：恰好装满的最大价值，无解为空。
效果：O(nC)。
optional<ll> exactZeroOneKnapsack(const vector<int>& weight,
                                  const vector<ll>& value,
                                  int capacity) {
    const ll NEG_INF = numeric_limits<ll>::lowest() / 4;
    // 除 dp[0] 外均不可达，因而该版本要求最终重量恰好为 capacity。
    vector<ll> dp(capacity + 1, NEG_INF);
    dp[0] = 0;
    for (int i = 0; i < static_cast<int>(weight.size()); ++i) {
        for (int j = capacity; j >= weight[i]; --j) {
            // 不可达状态不能参与加法，否则会制造不存在的方案。
            if (dp[j - weight[i]] != NEG_INF) {
                dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
            }
        }
    }
    if (dp[capacity] == NEG_INF) return nullopt;
    return dp[capacity];
}
```

易错点：

1. 0-1 背包容量必须倒序；正序会把一件物品使用多次。
2. “不超过容量”与“恰好装满”的初始化不同。
3. 重量为 0 且价值为正时仍只能选一次，倒序模板可正确处理。

参考：

- OI Wiki：背包 DP

4.3 完全背包与多重背包

适用范围： 完全背包允许每件任取次数，多重背包有数量上限；前者 O(nC)，后者二进制拆分 O(CΣlog count)。输入物品与容量，输出最优值；循环方向决定取用次数语义。 **能做什么：** 求完全背包或有限数量多重背包最大价值。 **输入 / 输出：** 输入平行重量/价值/数量数组及容量；返回最大价值；完全背包每件可无限取。 **复杂度：** O(nC) 或 O(CΣlog count).

用途： 完全背包中每件物品可选无限次，容量正序转移；多重背包限制第 i 件最多选 c_i 次，可按二进制拆成 $O(\log c_i)$ 件 0-1 物品。完全背包时间 $O(nW)$；二进制多重背包时间 $O(W\sum \log c_i)$，空间均为 $O(W)$。

思路：

- 两个模板都求容量不超过 `capacity` 的最大价值，允许留空。
- 完全背包要求正重量；零重量正价值且可无限选会导致答案无界。
- 多重背包将数量拆为 `1,2,4,...,剩余`，再按 0-1 背包倒序。
- 容量很大而总价值较小时，可改用“价值为状态、重量为代价”的 DP。

代码：

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// 参数：平行重量/价值数组与容量；每件可无限取。返回：不超过容量的最大价值。效果：O(nC)，重量须为正。
ll completeKnapsack(const vector<int>& weight,
                    const vector<ll>& value,
                    int capacity) {
    // weight 必须为正；dp[j] 表示容量上限为 j 的最大价值。
    vector<ll> dp(capacity + 1, 0);
    for (int i = 0; i < static_cast<int>(weight.size()); ++i) {
        // 正序使 dp[j-weight[i]] 可含当前物品，从而允许重复选择。
        for (int j = weight[i]; j <= capacity; ++j) {
            dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
        }
    }
    return dp[capacity];
}

// 参数：重量、价值、数量上限数组与容量。返回：不超过容量的最大价值。效果：二进制拆分，O(CΣlog count)。
ll multipleKnapsack(const vector<int>& weight,
                    const vector<ll>& value,
                    const vector<int>& count,
                    int capacity) {
    // 每个二进制组视为一件只能选择一次的 0-1 物品。
    vector<pair<int, ll>> items;
    for (int i = 0; i < static_cast<int>(weight.size()); ++i) {
        int remaining = count[i];
        for (int group = 1; remaining > 0; group <<= 1) {
            // 最后一组取全部 remaining，确保拆分数量之和恰为 count[i]。
            int take = min(group, remaining);
            items.push_back({weight[i] * take, value[i] * take});
            remaining -= take;
        }
    }

    vector<ll> dp(capacity + 1, 0);
    for (auto [groupWeight, groupValue] : items) {
        // 拆分后的每组只能使用一次，因此容量必须倒序。
        for (int j = capacity; j >= groupWeight; --j) {
            dp[j] = max(dp[j], dp[j - groupWeight] + groupValue);
        }
    }
    return dp[capacity];
}
```

易错点：

1. 完全背包正序、0-1 背包倒序；顺序写反会改变物品可用次数。
2. 二进制拆分最后一组应取剩余数量，不能遗漏非二次幂部分。
3. `weight * count` 和 `value * count` 可能溢出，价值使用 `long long`。

参考：

- OI Wiki：背包 DP

4.4 最长上升子序列（LIS）

适用范围： 求一个最长严格递增子序列及其下标；O(n log n)。输入任意可比较序列，输出 0-based 下标序列，空输入返回空序列。 **能做什么：** 恢复一个最长严格递增子序列的下标。 **输入 / 输出：** 输入可严格比较序列；返回递增的 0-based 下标向量，空输入返回空。 **复杂度：** O(n log n).

用途： 求序列中下标递增且值严格递增的最长子序列。维护 `tail[len]`：长度为 `len+1` 的上升子序列可取得的最小末尾值；对每个数用 `lower_bound` 替换首个不小于它的位置。时间 $O(n\log n)$、空间 $O(n)$，并可记录前驱恢复方案。

思路：

- 模板返回一条严格上升子序列的下标，原数组使用 `0-based`。
- 若求非下降子序列，把 `lower_bound` 改成 `upper_bound`。
- `tailValue` 只用于优化转移，它本身通常不是最终子序列。
- 最终长度为 `tailValue.size()`，根据 `previous` 逆向恢复下标。

代码：

```
#include <bits/stdc++.h>
using namespace std;

template <class T>
// 参数：可严格比较序列。返回：一个最长严格递增子序列的 0-based 下标。效果：无副作用，O(n log n)。
vector<int> longestIncreasingSubsequenceIndices(const vector<T>& a) {
    // a 与返回下标均为 0-based；本模板恢复严格上升子序列。
    int n = static_cast<int>(a.size());
    // tailValue[len] 是长度 len+1 的严格上升子序列的最小末值。
    vector<T> tailValue;
    // tailIndex 与 tailValue 同步，保存该末值来自的原数组下标。
    vector<int> tailIndex;
    vector<int> previous(n, -1);

    for (int i = 0; i < n; ++i) {
        // lower_bound 找首个 >= a[i] 的位置；改成 upper_bound 即求非下降。
        int position = static_cast<int>(lower_bound(tailValue.begin(),
                                                    tailValue.end(), a[i])
                                        - tailValue.begin());
        if (position == static_cast<int>(tailValue.size())) {
            tailValue.push_back(a[i]);
            tailIndex.push_back(i);
        } else {
            tailValue[position] = a[i];
            tailIndex[position] = i;
        }
        // 记录长度少一的最优末尾下标，供最终方案沿前驱恢复。
        if (position > 0) previous[i] = tailIndex[position - 1];
    }

    vector<int> indices;
    if (!tailIndex.empty()) {
        for (int i = tailIndex.back(); i != -1; i = previous[i]) {
            indices.push_back(i);
        }
```

```
        reverse(indices.begin(), indices.end());
    }
    return indices;
}
```

易错点：

1. 子序列不要求连续；要求连续的是子数组或子串问题。
2. 严格上升用 `lower_bound`，非下降用 `upper_bound`。
3. 不能直接输出 `tailValue` 作为方案，应通过前驱恢复。

参考：

- OI Wiki：最长上升子序列

4.5 最长公共子序列（LCS）

适用范围： 求两个序列的最长公共子序列；O(nm) 时间与空间。输入任意可相等比较的序列，输出一个最长公共子序列；并列答案不保证唯一。 **能做什么：** 求两个序列的一个最长公共子序列。 **输入 / 输出：** 输入支持相等比较的 a、b；返回一个最长公共子序列元素序列。 **复杂度：** O(nm).

用途： 求两个序列都能按原顺序取出的最长公共子序列。令 `dp[i][j]` 表示两序列前 i, j 项的 LCS 长度；末项相等则从左上加一，否则取上方和左方最大值。时间 $O(nm)$、空间 $O(nm)$；只求长度可滚动为 $O(m)$ 空间。

思路：

- 模板输入任意可比较元素序列，返回一条 LCS。
- `dp` 多开一行一列作为空前缀，初值全为 0。
- 恢复时相等元素加入答案；否则走向 DP 值较大的一侧。
- 最长公共子串要求连续，状态和答案更新方式不同，不能套用此模板。

代码：

```
#include <bits/stdc++.h>
using namespace std;

template <class T>
// 参数：支持相等比较的两序列。返回：一个最长公共子序列。效果：无副作用，O(nm) 时间与空间。
vector<T> longestCommonSubsequence(const vector<T>& a, const vector<T>& b) {
    // a、b 为 0-based；dp[i][j] 表示两序列前 i、j 个元素的 LCS 长度。
    int n = static_cast<int>(a.size());
    int m = static_cast<int>(b.size());
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    // i、j 递增，保证左、上、左上三个依赖状态已经计算。
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (a[i - 1] == b[j - 1]) {
                // 最优公共子序列可把这对相同末字符一起接到左上状态。
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                // 末字符不同时，至少舍弃一侧末字符，取两种可能的较优值。
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }
```

```
    vector<T> result;
    // 从右下角逆推；值相等走左上，否则走不减小 dp 的一侧。
    for (int i = n, j = m; i > 0 && j > 0;) {
        if (a[i - 1] == b[j - 1]) {
            result.push_back(a[i - 1]);
            --i;
            --j;
        } else if (dp[i - 1][j] >= dp[i][j - 1]) {
            --i;
        } else {
            --j;
        }
    }
    reverse(result.begin(), result.end());
    return result;
}
```

易错点：

1. 元素相等时使用左上状态加一，不是上方或左方加一。
2. LCS 可以不连续，不要与最长公共子串混淆。
3. 完整二维表占 $O(nm)$ 内存，`n,m` 很大时可能超限。

参考：

- OI Wiki：动态规划基础

4.6 区间 DP

适用范围： 答案由相邻子区间合并得到时适用；常见 O(n^3)，空间 O(n^2)。输入序列，输出区间最优值；按区间长度递增，需定义单元素初值。 **能做什么：** 最小化相邻石子反复合并的总代价。 **输入 / 输出：** 输入石子权值；返回每次合并成本为区间和时的最小总费用，少于两项返回 0。 **复杂度：** 通常 O(n^3).

用途： 当答案由连续区间的子区间合并而来时，令 `dp[l][r]` 表示区间 `[l,r]` 的最优值，按区间长度递增，枚举分割点 `k` 合并左右答案。典型石子合并时间 $O(n^3)$、空间 $O(n^2)$；环形问题常复制一倍转成链。

思路：

- 模板求链式相邻石子合并的最小总代价，`stones` 使用 `0-based`。
- `dp[i][i]=0`，因为一堆石子无需合并。
- 前缀和在 $O(1)$ 时间得到当前区间总重量。
- 若求最大值，修改初值和 `min`；代价公式需按具体题意替换。

代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = numeric_limits<ll>::max() / 4;

// 参数：相邻合并权值序列。返回：每次合并相邻段、代价为段和时的最小总费用。效果：O(n^3)，空/
单元素费用为 0。
ll minimumMergeCost(const vector<ll>& stones) {
    // stones 使用 0-based；dp[left][right] 表示闭区间合成一堆的最小代价。
    int n = static_cast<int>(stones.size());
```

```
    if (n == 0) return 0;
    vector<ll> prefix(n + 1, 0);
    for (int i = 0; i < n; ++i) prefix[i + 1] = prefix[i] + stones[i];

    // dp[i][i]=0；按长度递增保证转移所需的两个子区间已经算完。
    vector<vector<ll>> dp(n, vector<ll>(n, 0));
    for (int length = 2; length <= n; ++length) {
        // 所有长度更短的区间都已完成，当前转移可直接读取子区间。
        for (int left = 0; left + length <= n; ++left) {
            int right = left + length - 1;
            dp[left][right] = INF;
            ll intervalSum = prefix[right + 1] - prefix[left];
            // split 将 [left,right] 分成 [left,split] 与 [split+1,right]。
            for (int split = left; split < right; ++split) {
                dp[left][right] = min(dp[left][right],
                    dp[left][split] + dp[split + 1][right] + intervalSum);
            }
        }
    }
    return dp[0][n - 1];
}
```

易错点：

1. 必须按区间长度从小到大计算，确保子区间已经完成。
2. 分割点范围为 `left <= split < right`。
3. 环形区间复制后通常只枚举长度不超过原长度的区间。

参考：

- OI Wiki：区间 DP

4.7 树形 DP

适用范围： 父子依赖可分解的树上最优值；O(n) 状态/边处理。输入以 1..n 编号的树，输出根状态或全点结果；递归深度可能达 n。 **能做什么：** 求树上最大权独立集。 **输入 / 输出：** 输入 1..n 无向树与节点权值；返回不选相邻点的最大权和，允许空集。 **复杂度：** O(n).

用途： 树形 DP 先选根，用 DFS 汇总子树信息；状态通常描述节点选或不选、子树内代价等。树上最大权独立集令 `dp[u][1]` 为选 u，则孩子不能选；`dp[u][0]` 为不选，可取孩子两状态较大者。时间 $O(n)$、空间 $O(n)$。

思路：

- 模板输入无根树邻接表和 `weight[1..n]`，返回最大权值，允许空集。
- `parent` 用于阻止 DFS 沿无向边返回父节点。
- 选中节点后，与它直接相邻的节点均不能选。
- 若需恢复方案，保存每个状态对孩子所选状态并再次遍历。

代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
```

```
// 参数：1..n 编号无向树。返回：不选相邻点时的最大权值和。效果：O(n)，权值按本函数参数约定给出。
ll maximumWeightIndependentSet(const vector<vector<int>>& tree,
                               const vector<ll>& weight,
                               int root = 1) {
    // tree 与 weight 均为 1-based；输入应为一棵连通无向树。
    int n = static_cast<int>(tree.size()) - 1;
    vector<array<ll, 2>> dp(n + 1, {0, 0});

    // 参数：当前点与父点。返回：无。效果：递归聚合子树状态到 dp[u]。
    function<void(int, int)> dfs = [&](int u, int parent) {
        // dp[u][0/1]：u 不选/选时，其整棵子树内可取得的最大权值。
        dp[u][0] = 0;
        dp[u][1] = weight[u];
        for (int v : tree[u]) {
            if (v == parent) continue;
            dfs(v, u); // 后序处理保证子树状态先于父状态完成
            // 不选 u 可自由选孩子最优状态；选 u 时所有孩子都必须不选。
            dp[u][0] += max(dp[v][0], dp[v][1]);
            dp[u][1] += dp[v][0];
        }
    };

    dfs(root, 0);
    return max(dp[root][0], dp[root][1]);
}
```

易错点：

1. 无向树必须跳过父节点，否则递归会来回循环。
2. 合并孩子前先写清各维状态含义，不能在同一轮覆盖仍需使用的旧值。
3. 输入若不是连通树，需要对每个连通块分别计算并合并答案。

参考：

- OI Wiki：树形 DP

4.8 状态压缩 DP

适用范围： n 较小且状态由元素子集决定；常见 O(2^n n^2) 时间、O(2^n n) 空间。输入 n 个点及代价矩阵，输出最优闭合路线代价；要求 n 能安全位移且内存可承受。 **能做什么：** 求访问所有点并回到起点的最小旅行商回路。 **输入 / 输出：** 输入 n×n cost 方阵及 0-based 起点；返回最小回路费用，无解为 INF。 **复杂度：** O(2^n n^2).

用途： 当元素数较小且状态仅表示“哪些元素已选”时，用二进制掩码压缩集合。旅行商路径令 `dp[mask][u]` 为从起点出发、访问集合 `mask` 且停在 u 的最小代价，枚举新节点转移。时间 $O(n^2 2^n)$、空间 $O(n 2^n)$，通常适合 $n \le 20$。

思路：

- 模板求从 `start` 出发、访问全部节点并回到起点的最短回路。
- `cost[u][v]` 为边权；无边设为 `INF`，允许有向图。
- `mask` 必须包含 `start` 和当前终点 `u`。
- 只求访问全部点的最短路径时，最后取各终点最小值，不再加回起点的边。

代码：

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = numeric_limits<ll>::max() / 4;

// 参数：方阵 cost 与起点下标。返回：从起点出发访问每点一次并回起点的最小费用；无可行环返回 INF。效果：O(2^n n^2)，n 须较小。
ll travelingSalesmanCycle(const vector<vector<ll>>& cost, int start = 0) {
    // 节点为 0-based；cost[u][v] 可表示有向边，无边必须设为 INF。
    int n = static_cast<int>(cost.size());
    int all = 1 << n;
    // dp[mask][u]：从 start 出发，恰好访问 mask 中节点并停在 u 的最小代价。
    vector<vector<ll>> dp(all, vector<ll>(n, INF));
    dp[1 << start][start] = 0;

    for (int mask = 0; mask < all; ++mask) {
        if (!(mask & (1 << start))) continue;
        for (int u = 0; u < n; ++u) {
            // u 必须已访问且状态可达；INF 状态不能参与加法。
            if (!(mask & (1 << u)) || dp[mask][u] == INF) continue;
            for (int v = 0; v < n; ++v) {
                if (mask & (1 << v)) continue;
                if (cost[u][v] == INF) continue;
                // 只向未访问 v 扩展，因此每个节点在路径中恰好出现一次。
                int nextMask = mask | (1 << v);
                dp[nextMask][v] = min(dp[nextMask][v],
                                      dp[mask][u] + cost[u][v]);
            }
        }
    }

    ll answer = INF;
    // 最后补上 u -> start；若仍为 INF，表示不存在哈密顿回路。
    for (int u = 0; u < n; ++u) {
        if (dp[all - 1][u] != INF && cost[u][start] != INF) {
            answer = min(answer, dp[all - 1][u] + cost[u][start]);
        }
    }
    return answer;
}
```

易错点：

1. `1 << n` 要求 `n` 较小；位数超过 `int` 时应改用 64 位掩码，但状态数通常早已不可承受。
2. 无穷大状态不能直接参与加法，避免溢出形成伪最短路。
3. 估算内存：`2^20 * 20 * 8` 字节约为 160 MiB。

参考：

- OI Wiki：状压 DP

4.9 数位 DP

适用范围： 按数字位逐位构造且约束可由有限状态描述；复杂度 O(L·S)，S 为状态数。输入十进制上界与目标和，输出 [0,N] 中符合条件数量；前导零语义由实现固定。 **能做什么：** 统计数位和等于目标的非负整数个

数。 **输入 / 输出：** 输入非负上界与目标数位和；返回 [0,n] 内符合条件个数，前导零不计数。 **复杂度：** O(L·S).

用途： 数位 DP 统计巨大整数区间内满足数位约束的数。按高位到低位搜索，状态记录位置、已用信息及是否受上界限制；不受限状态可记忆化。模板统计数位和等于目标值的非负整数个数，时间 $O(D \cdot S \cdot 10)$、空间 $O(D \cdot S)$。

思路：

- `countDigitSumUpTo(n,s)` 统计 `[0,n]` 中数位和为 `s` 的数，`n<0` 返回 0。
- 区间 `[left,right]` 的答案为两个前缀计数之差。
- 前导零在本题不改变数位和；涉及“数字长度、相邻位”时需增加 `started` 状态。
- 计数可能超过 `long long`，题目要求取模时应在转移中取模。

代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// 参数：非负上界 n 与目标数位和。返回：[0,n] 中数位和等于目标的整数个数。效果：O(位数
×targetSum)。
ll countDigitSumUpTo(ll n, int targetSum) {
    // 统计闭区间 [0,n]；n<0 或目标和为负时统一返回 0。
    if (n < 0 || targetSum < 0) return 0;
    string digits = to_string(n);
    int length = static_cast<int>(digits.size());
    vector<vector<ll>> memo(length, vector<ll>(targetSum + 1, -1));

    // position 从最高位向低位；tight 表示当前前缀是否仍贴住 n 的上界。
    // 参数：当前数位位置、已用数位和及是否受上界限制。返回：满足目标数位和的补全数。效果：
写入 memo，避免重复状态计算。
    function<ll(int, int, bool)> dfs = [&](int position, int sum, bool tight) -> ll {
        if (sum > targetSum) return 0;
        if (position == length) return sum == targetSum;
        // 仅非受限状态与更高位选择无关，可以跨路径复用记忆化值。
        if (!tight && memo[position][sum] != -1) return memo[position][sum];

        int upper = tight ? digits[position] - '0' : 9;
        ll answer = 0;
        for (int digit = 0; digit <= upper; ++digit) {
            // 只有原状态受限且本位取到当前上界，下一位才继续受限。
            answer += dfs(position + 1, sum + digit,
                          tight && digit == upper);
        }
        if (!tight) memo[position][sum] = answer;
        return answer;
    };

    return dfs(0, 0, true);
}

// 参数：非负闭区间 [left,right] 与目标数位和。返回：区间内符合条件的整数个数。效果：调用两
次 countDigitSumUpTo。
ll countDigitSumInRange(ll left, ll right, int targetSum) {
```

```
    // 用前缀计数相减得到闭区间 [left,right]; left-1 由前缀函数处理负数。
    return countDigitSumUpTo(right, targetSum)
         - countDigitSumUpTo(left - 1, targetSum);
}
```

易错点：

1. `tight` 的下一状态应为“原来受限且当前位取到上界”。
2. 只有不受上界限制的状态才能直接复用记忆化结果。
3. 数字 0 是否计入、前导零是否影响约束，必须按题意单独处理。

参考：

- OI Wiki：数位 DP

4.10 DAG 上的 DP

适用范围： 依赖图为 DAG 且转移沿边传播；O(V+E)。输入邻接表与源点，输出最长路值；不可达节点用 NEG_INF 标记。 **能做什么：** 计算 DAG 中从源点出发的最长路。 **输入 / 输出：** 输入 1..n DAG 邻接表及源点；返回各点最长路，不可达为 NEG_INF。 **复杂度：** O(V+E).

用途： 有向无环图天然给出状态依赖：按拓扑序处理节点，使所有前驱先于后继完成。模板求指定源点到各点的最长路，可有负边；拓扑排序与松弛总时间 $O(n+m)$、空间 $O(n+m)$。若图存在环则拓扑序不完整，不能直接使用 DAG DP。

思路：

- 图使用 `graph[u] = {v,w}`，点编号为 `1..n`。
- `distance[v] == NEG_INF` 表示从源点不可达。
- 求路径条数、最短路等，只需替换状态初值与松弛公式。
- 若拓扑序长度小于 `n`，函数返回空数组表示输入不是 DAG。

代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll NEG_INF = numeric_limits<ll>::lowest() / 4;

struct Edge {
    int to;
    ll weight;
};

// 参数：1..n 编号 DAG 邻接表与源点。返回：各点最长路权值；不可达为 NEG_INF。效果：O(V+E)，图必须无环。
vector<ll> longestPathInDAG(const vector<vector<Edge>>& graph, int source) {
    // graph 与节点为 1-based、有向、可含重边和负权，但必须无环。
    int n = static_cast<int>(graph.size()) - 1;
    vector<int> indegree(n + 1, 0);
    for (int u = 1; u <= n; ++u) {
        for (const auto& edge : graph[u]) ++indegree[edge.to];
    }

    queue<int> q;
```

```
    // 所有入度为 0 的点同时入队，保证断开的 DAG 也能完成拓扑排序。
    for (int u = 1; u <= n; ++u) {
        if (indegree[u] == 0) q.push(u);
    }
    vector<int> order;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        for (const auto& edge : graph[u]) {
            if (--indegree[edge.to] == 0) q.push(edge.to);
        }
    }
    // 拓扑序不含全部节点说明存在有向环；用空数组作为失败状态。
    if (static_cast<int>(order.size()) != n) return {};

    // distance[u] 为 source 到 u 的最长距离；NEG_INF 表示不可达。
    vector<ll> distance(n + 1, NEG_INF);
    distance[source] = 0;
    for (int u : order) {
        // 不可达点不能松弛出边，否则负无穷参与加法会产生伪状态。
        if (distance[u] == NEG_INF) continue;
        for (const auto& edge : graph[u]) {
            distance[edge.to] = max(distance[edge.to],
                                    distance[u] + edge.weight);
        }
    }
    return distance;
}
```

易错点：

1. 必须按拓扑序转移，按节点编号遍历通常不满足依赖关系。
2. 负权边存在时，不可达状态必须设为负无穷而非 0。
3. 有环图没有完整拓扑序；若需要处理环，应先缩点或改建模。

参考：

- OI Wiki：DAG 上的 DP

4.11 编辑距离

适用范围： 插入、删除、替换的单位代价字符串编辑距离；O(nm) 时间、O(m) 空间。输入两字符串，输出最少操作数。 **能做什么：** 计算单位编辑代价下的字符串编辑距离。 **输入 / 输出：** 输入字符串 a、b；返回插入/删除/替换代价均为 1 时的最少操作数。 **复杂度：** O(nm).

用途与复杂度。 求两个字符串通过插入、删除、替换变成相同字符串的最少操作数。状态转移枚举最后一步，时间 $O(nm)$；滚动一行后空间为 $O(m)$。

思路与用法。 `dp[j]` 表示已处理的 `a` 前缀与 `b[0..j)` 的编辑距离。每行开始时保存左上旧值；相等字符沿用左上，不等时在删除、插入、替换三者中取最小。

代码：

```cpp
#include <bits/stdc++.h>
using namespace std;

// 参数：两字符串。返回：插入、删除、替换代价均为 1 的最小编辑次数。效果：O(|a||b|) 时间、O(|b|) 空间。
int editDistance(const string& a, const string& b) {
    // dp[j] 表示当前 a 前缀与 b 的前 j 个字符之间的编辑距离。
    vector<int> dp(b.size() + 1);
    iota(dp.begin(), dp.end(), 0); // 空串变成 b 前 j 字符需要 j 次插入

    for (int i = 1; i <= static_cast<int>(a.size()); ++i) {
        int diagonal = dp[0]; // 覆盖前保存左上角的旧状态
        dp[0] = i; // b 为空时只能删除 a 的 i 个字符
        for (int j = 1; j <= static_cast<int>(b.size()); ++j) {
            int oldAbove = dp[j];
            if (a[i - 1] == b[j - 1]) {
                dp[j] = diagonal;
            } else {
                dp[j] = 1 + min({oldAbove, dp[j - 1], diagonal});
            }
            diagonal = oldAbove;
        }
    }
    return dp[b.size()];
}
```

易错点。 `dp[j]` 在更新前是上方旧值，`dp[j-1]` 是本行左侧新值，`diagonal` 才是左上旧值；字符串很长时 `O(nm)` 仍可能超时。

4.12 树形 DP 与换根

适用范围： 树上需要求每个点作根的答案且可从父答案增量转移；O(n)。输入树，输出每个根的答案；边权和/节点编号需符合代码约定。 **能做什么：** 计算树上每个点作为根时的距离和。 **输入 / 输出：** 输入 n 和 1-based 无向树边；solve 返回 n+1 向量，1..n 槽为各根答案；同一对象仅调用一次，重复调用会重复累加。 **复杂度：** O(n).

用途与复杂度。 子树信息可合并成父节点状态；换根 DP 复用相邻根之间的答案，避免每个根重新遍历整棵树。示例要求 $n \geq 1$，求无权树每个点到所有点的距离和，时间 $O(n)$、空间 $O(n)$。

思路与用法。 先以 1 为根求子树大小和根 1 的距离总和；第二遍从父点 `u` 移到子点 `v` 时，`v` 子树内的 `sz[v]` 个点距离减 1，其余 `n-sz[v]` 个距离加 1，因此 `ans[v]=ans[u]+n-2*sz[v]`。

代码：

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct RerootDistanceSum {
    int n;
    vector<vector<int>> graph;
    vector<int> subtreeSize;
    vector<ll> answer;

    // 参数：树的顶点数 n>=1。返回：清零的换根 DP 对象。效果：分配 1..n 邻接表及状态数组。
    explicit RerootDistanceSum(int n)
        : n(n), graph(n + 1), subtreeSize(n + 1), answer(n + 1, 0) {}
```

```
    // 参数：树中相邻顶点 u、v。返回：无。效果：向无向邻接表加入一条边。
    void addEdge(int u, int v) {
        graph[u].push_back(v);
        graph[v].push_back(u); // 无向树
    }

    // 参数：当前点、父点和深度。返回：无。效果：计算子树大小与初始根答案。
    void firstDfs(int u, int parent, int depth) {
        subtreeSize[u] = 1;
        answer[1] += depth; // 第一遍累加以 1 为根的距离和
        for (int v : graph[u]) {
            if (v == parent) continue;
            firstDfs(v, u, depth + 1);
            subtreeSize[u] += subtreeSize[v];
        }
    }

    // 参数：当前根及其父点。返回：无。效果：将父点答案转移到子点并遍历整棵树。
    void rerootDfs(int u, int parent) {
        for (int v : graph[u]) {
            if (v == parent) continue;
            answer[v] = answer[u] + n - 2LL * subtreeSize[v];
            rerootDfs(v, u);
        }
    }

    // 参数：无；构造器须已给出 n 与全部树边。返回：answer[1..n] 的各根距离和。效果：O(n)，
写入并返回内部答案。
    vector<ll> solve() {
        firstDfs(1, 0, 0);
        rerootDfs(1, 0);
        return answer; // answer[1..n] 有效
    }
};
```

易错点。 公式依赖树和单位边长；加权树需改用子树权重和总边权推导。递归链可能达到 n 层，超大树应改迭代 DFS；每个测试用例都要重建或清空结构。

4.13 概率与期望 DP

适用范围： 等概率、独立抽取不同种类卡片直到集齐；本例不覆盖一般状态图。复杂度 O(n)。 **能做什么：** 计算 n 种卡片等概率独立抽取直至集齐的期望次数。 **输入 / 输出：** 输入种类数 n>0；返回期望抽取次数。 **复杂度：** O(n) 时间与空间。

用途与复杂度。 已收集 i 种时，以 $(n-i)/n$ 概率抽到新种类，否则仍在状态 i。移项可得 `E[i]=(1 + pNew * E[i+1])/pNew`，总计 O(n) 时间与空间。

思路与用法。 收集 n 种等概率卡片，已有 i 种时下一步以 (n-i)/n 概率获得新种类，否则回到同一状态。把留在原状态的项移到等式左边，得到 `E[i]=(1 + pNew * E[i+1])/pNew`。

代码：

```
#include <bits/stdc++.h>
using namespace std;

// 参数：不同卡片种类数 n>0。返回：等概率独立抽取直到集齐的期望次数。效果：O(n) 时间、空间。
double couponCollectorExpectation(int n) {
```

```
    // e[i] 是已收集 i 种时还需抽取次数的期望，边界 e[n]=0。
    vector<double> e(n + 1, 0.0);
    for (int i = n - 1; i >= 0; --i) {
        double pNew = static_cast<double>(n - i) / n;
        // 一步必然消耗，再以 pNew 前进；重复抽到旧种的概率已移项。
        e[i] = (1.0 + pNew * e[i + 1]) / pNew;
    }
    return e[0];
}
```

易错点。期望的线性性允许对和取期望，但一般不能把“最优”随机决策直接替换为期望最大或最小；检查分母非零，输出精度按题目要求设置。

5. 数据结构

本章代码默认使用 C++17。除特别说明外，数组下标从 1 开始；调用查询、删除操作前应保证容器非空、区间合法。字典树统一收录在第 8 章字符串中。

5.1 栈与队列

适用范围： 适用于需要后进先出或先进先出处理的序列任务。 **能做什么：** 分别提供常数时间的栈顶/队首访问与两端操作。 **输入 / 输出：** 输入：可迭代元素序列；输出：按栈、队列或双端队列规则处理的元素。

问题与思路： 栈按“后进先出”处理括号、递归展开和撤销；队列按“先进先出”处理分层扩展与任务调度。`push`、`pop`、访问栈顶或队首均为 $O(1)$，存储 k 个元素需 $O(k)$ 空间。删除或读取前必须判空；需要两端操作时改用 `deque`。

用法：

- `stack` 只访问 `top()`；`queue` 访问 `front()`，也可查看 `back()`。
- `pop()` 不返回元素，应先读取再弹出。
- `empty()` 判空，`size()` 返回当前元素数。
- 需要从两端插入、删除时使用 `deque` 的 `push_front`、`push_back`、`pop_front`、`pop_back`。

```
#include <bits/stdc++.h>
using namespace std;

// 参数 a 是待演示的整数序列；返回 void，局部容器消费并展示栈/队列/双端队列顺序，不改 a。
    void stackQueueExample(const vector<int>& a) {
    // 适配器不暴露迭代器；以下循环都先读顶端/队首，再 pop。
    stack<int> st;
    for (int x : a) st.push(x); // 入栈顺序与 a 相同，弹出顺序因此相反
    while (!st.empty()) {
        int x = st.top();
        st.pop();
        (void)x;  // 在此处理：顺序与 a 相反
    }

    queue<int> q;
    for (int x : a) q.push(x); // 队列按到达顺序服务
    while (!q.empty()) {
        int x = q.front();
        q.pop();
        (void)x;  // 在此处理：顺序与 a 相同
    }

    deque<int> dq;
    dq.push_front(1);
    dq.push_back(2);
    if (!dq.empty()) dq.pop_front();
}
```

易错点：

1. 对空容器调用 `top()`、`front()`、`back()` 或 `pop()` 是未定义行为。
2. `stack`、`queue` 默认不支持随机访问和遍历。
3. 深度优先搜索改写为显式栈时，入栈顺序会影响访问顺序。

参考：

- OI Wiki：栈
- OI Wiki：队列

5.2 单调栈

适用范围： 适用于静态数组中寻找左右最近的更大/更小元素。 **能做什么：** 在线性扫描中定位每个位置右侧首个严格更小元素。 **输入 / 输出：** 输入：任意整数数组；输出：等长下标数组，不存在处为 -1。

问题与思路： 求每个位置左/右侧第一个更大或更小元素时使用。扫描数组，栈内保存仍未找到答案的下标，并保持对应值单调；新元素使栈顶确定答案时不断弹出。每个下标至多进出栈一次，时间 $O(n)$、空间 $O(n)$。等号是否弹出决定“严格”还是“不严格”。

用法：

- 模板返回每个位置右侧第一个严格更小元素的下标，不存在为 `-1`。
- 栈保存下标，既能取值也能计算距离。
- 改求右侧第一个严格更大：把弹出条件改为 `a[i] > a[st.top()]`。
- 若求“小于等于”，相应把 `<` 改为 `<=`。

```
#include <bits/stdc++.h>
using namespace std;

// 参数 a 为 0-based 数组；返回每项右侧首个严格更小位置的下标，不存在为 -1；不改 a。
    vector<int> firstSmallerOnRight(const vector<int>& a) {
    // a 与答案均为 0-based；answer[i] 是右侧首个严格更小值的下标。
    int n = (int)a.size();
    vector<int> answer(n, -1);
    stack<int> st;  // 栈中下标递增，且对应值从栈底到栈顶单调不减

    for (int i = 0; i < n; ++i) {
        // 严格更小才弹；改成 <= 即改为“右侧首个小于等于”。
        while (!st.empty() && a[i] < a[st.top()]) {
            answer[st.top()] = i;
            st.pop();
        }
        st.push(i);
    }
    return answer;
}
```

易错点：

1. 必须先明确答案是否允许相等，错误的等号会改变重复值的答案。
2. 求距离时使用 `answer[i] - i`，不存在的 `-1` 要单独处理。
3. 栈内通常保存下标而非数值，否则无法定位答案。

参考：

- OI Wiki：单调栈

5.3 单调队列

适用范围： 适用于固定窗口最值及可转化为候选区间最值的动态规划。 **能做什么：** 在线性时间计算每个合法滑动窗口的最小值。 **输入 / 输出：** 输入：整数数组和窗口长度 k；输出：各长度 k 窗口的最小值，k 非法时为空数组。

问题与思路： 用于固定长度滑动窗口最值，也可优化形如“某段范围内取最优”的 DP。双端队列保存候选下标：队首过期就弹出，队尾不可能成为最优就弹出。每个下标至多进出一次，时间 $O(n)$、空间 $O(k)$。模板求长度为 k 的窗口最小值。

用法：

- 返回数组长度为 `n-k+1`，第 `i` 项对应区间 `[i,i+k-1]`。
- 队列必须保存下标，才能判断元素是否离开窗口。
- 求最大值时把维护条件 `>=` 改为 `<=`。
- 本模板保留更靠右的相等元素，使其有效期更长。

```
#include <bits/stdc++.h>
using namespace std;

// 参数 a 为 0-based 数组、k 为窗口长度；返回各窗口最小值，k<=0 或 k>n 时返回空表；不改 a。
    vector<int> slidingWindowMinimum(const vector<int>& a, int k) {
    // a 为 0-based；每个答案对应闭区间 [i-k+1, i]。
    int n = (int)a.size();
    vector<int> answer;
    if (k <= 0 || k > n) return answer;

    // dq 保存候选下标：下标递增，对应值严格递增，队首永远是窗口最小值。
    deque<int> dq;
    for (int i = 0; i < n; ++i) {
        while (!dq.empty() && dq.front() <= i - k) dq.pop_front();  // 移出窗口
        // 相等也弹出旧下标，保留有效期更长的新下标；求最大值时反转不等号。
        while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
        dq.push_back(i);
        if (i >= k - 1) answer.push_back(a[dq.front()]);
    }
    return answer;
}
```

易错点：

1. 应先删除过期下标，再读取队首答案。
2. 队列单调的是对应值，队列中的下标始终递增。
3. 窗口长度不合法时不能直接访问队首。

参考：

- OI Wiki：单调队列

5.4 堆与优先队列

适用范围： 适用于反复取当前最大/最小值的在线任务。 **能做什么：** 支持插入及按优先级取出元素。 **输入 / 输出：** 输入：元素序列；输出：按优先级弹出的有序序列。

问题与思路： 堆动态维护当前最大或最小元素，适合 Dijkstra、贪心调度和 Top-K。C++ `priority_queue` 默认大根堆；插入、删除堆顶为 $O(\log n)$，读取堆顶 $O(1)$，建堆 $O(n)$，空间 $O(n)$。它不能高效删除任意元

素，常用“重复插入并跳过失效记录”。

用法：

- 大根堆：`priority_queue<int>`；小根堆见模板。
- 自定义状态常以 `pair<距离,点>` 配合 `greater`，按字典序取最小。
- `pop()` 不返回值，先用 `top()` 保存。
- 无法直接修改堆内元素；更新后重新 `push`，弹出时验证状态是否仍有效。

```
#include <bits/stdc++.h>
using namespace std;

// 参数 a 为待排序整数序列；返回按升序弹出的副本，不改 a。
    vector<int> heapOrder(const vector<int>& a) {
    // greater<int> 使 top() 为当前最小值；若去掉则默认是大根堆。
    priority_queue<int, vector<int>, greater<int>> minHeap;
    for (int x : a) minHeap.push(x);

    vector<int> sorted;
    while (!minHeap.empty()) {
        sorted.push_back(minHeap.top());
        minHeap.pop();
    }
    return sorted;
}
```

易错点：

1. `priority_queue` 默认取最大值，求最小时不要忘记 `greater<T>`。
2. 自定义类型需提供比较规则，比较器含义与 `sort` 容易混淆。
3. 使用懒删除时，堆的实际大小可能远大于有效元素数。

参考：

- OI Wiki：堆

5.5 并查集

适用范围： 适用于只合并、不拆分的动态连通分量与等价类。 **能做什么：** 查询代表元、判断连通并按大小合并集合。 **输入 / 输出：** 输入：元素编号 1..n 及查询/合并操作；输出：连通判断、代表元或集合大小。

问题与思路： 维护元素不断合并后的连通关系，适合 Kruskal、离线连通性和等价类。每个集合用根代表；路径压缩加按大小合并，使单次查询或合并均摊 $O(\alpha(n))$，初始化和空间均为 $O(n)$。普通并查集不支持拆分，也不能直接维护集合内的路径信息。

用法：

- 构造 `DSU dsu(n)`，管理编号 `1..n`。
- `find(x)` 返回集合代表元，同时压缩路径。
- `unite(a,b)` 合并成功返回 `true`；已经连通返回 `false`。
- `same(a,b)` 判断连通，`size(x)` 查询所在集合大小。

```cpp
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    // parent[x] 指向父节点；仅根满足 parent[x]==x，且只有根的 sizeOf 有效。
    vector<int> parent, sizeOf;

    // 参数 n 是元素上界；建立编号 1..n 的单元素集合，初始化内部 parent/sizeOf。
    explicit DSU(int n) : parent(n + 1), sizeOf(n + 1, 1) {
        iota(parent.begin(), parent.end(), 0);
    }

    // 参数 x 是 1..n 的元素；返回其集合根，并对访问路径做路径压缩。
    int find(int x) {
        if (parent[x] == x) return x;
        // 路径压缩保持集合不变，只把查询路径上的点直接挂到根。
        return parent[x] = find(parent[x]);
    }

    // 参数 a、b 是 1..n 的元素；合并两集合，成功返回 true、原已连通返回 false；修改根和集合大小。
    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sizeOf[a] < sizeOf[b]) swap(a, b);  // 始终把小树挂到大树
        parent[b] = a;
        sizeOf[a] += sizeOf[b];
        return true;
    }

    // 所有公开查询都先压到根，避免调用方误把非根当代表。
    // 参数 a、b 是 1..n 元素；返回是否同属一集合，查询会通过 find 压缩路径。
    bool same(int a, int b) { return find(a) == find(b); }
    // 参数 x 是 1..n 元素；返回其所在集合大小，查询会通过 find 压缩路径。
    int size(int x) { return sizeOf[find(x)]; }
};
```

易错点：

1. 按大小合并时，集合大小只在根节点处有效。
2. 若题目需要撤销合并，不能使用普通路径压缩并查集。
3. 合并前必须先取根，不能直接修改 `parent[a]`。

参考：

- OI Wiki：并查集

5.6 树状数组

适用范围： 适用于可加减的单点更新、前缀和及区间和。 **能做什么：** 以 O(log n) 执行单点增量和前缀/区间求和。 **输入 / 输出：** 输入：1-based 数组及合法下标更新/查询；输出：对应和或满足频次前缀条件的最小下标。

问题与思路： 维护可加减信息的单点修改与前缀查询，常用于动态区间和、逆序对。下标 `x` 通过 `x & -x` 跳到负责区间；单点加、前缀和及区间和均为 $O(\log n)$，建表通常 $O(n\log n)$，空间 $O(n)$。基础模板不直接支持区间最值或任意区间赋值。

用法：

- 构造 Fenwick fw(n)，有效下标为 1..n，不能向下标 0 更新。
- add(i,delta) 表示第 i 项增加 delta。
- prefixSum(i) 查询 [1,i]，rangeSum(l,r) 查询 [l,r]。
- 单点赋值需先知道旧值，再增加“新值减旧值”。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct Fenwick {
    int n;
    // 仅使用 1..n；tree[i] 保存 (i-lowbit(i), i] 这一段的和。
    vector<ll> tree;

    // 参数 n 是正的最大下标；创建全零 1-based 树状数组，初始化内部 tree。
    explicit Fenwick(int n) : n(n), tree(n + 1, 0) {}

    // 参数 index 属于 1..n、delta 为增量；返回 void，修改覆盖 index 的 Fenwick 节点。
    void add(int index, ll delta) {
        // index 必须 >=1；向父区间跳转，直到超过 n。
        for (int i = index; i <= n; i += i & -i) tree[i] += delta;
    }

    // 参数 index 属于 0..n；返回原数组闭区间 [1,index] 的和，不改结构。
    ll prefixSum(int index) const {
        // 查询闭区间 [1,index]；每次删掉当前下标最低位的 1。
        ll result = 0;
        for (int i = index; i > 0; i -= i & -i) result += tree[i];
        return result;
    }

    // 参数为合法闭区间；返回当前区间和，不修改分块状态。
    ll rangeSum(int left, int right) const {
        if (left > right) return 0;
        return prefixSum(right) - prefixSum(left - 1);
    }

    // 频次数组非负时，返回最小 i 使 prefixSum(i)>=target；无解返回 n+1。
    // 参数 target 为目标前缀频次；要求各点频次非负，返回最小满足前缀和>=target 的下标，无解
为 n+1。
    int lowerBound(ll target) const {
        int index = 0;
        ll accumulated = 0;
        int step = 1;
        while ((step << 1) <= n) step <<= 1;
        // 二进制提升：index 始终是“前缀和仍小于 target”的最大已知下标。
        for (; step; step >>= 1) {
            int next = index + step;
            if (next <= n && accumulated + tree[next] < target) {
                index = next;
                accumulated += tree[next];
            }
        }
        return index + 1;
    }
};
```

易错点：

1. 更新下标必须大于 0，否则 `i += i & -i` 会死循环。
2. 区间和是 `prefixSum(r)-prefixSum(l-1)`。
3. 值和答案可能超过 `int`，树数组优先使用 `long long`。
4. `lowerBound` 要求单点频次非负，目标大于总和时返回 `n+1`。

参考：

- OI Wiki：树状数组

5.7 线段树（区间加、区间和）

适用范围： 适用于数组区间加与区间和交错出现的场景。 **能做什么：** 维护闭区间加法和区间和，单次操作 O(log n)。 **输入 / 输出：** 输入：1-based 初始数组及合法闭区间操作；输出：区间和，更新直接修改结构。

问题与思路： 维护数组的区间修改与区间查询。节点保存区间和，整段增加时记录懒标记，访问子区间前再下传。建树 $O(n)$，每次区间加或区间和查询 $O(\log n)$，空间 $O(n)$（通常开 `4n`）。模板要求修改可组合；改维护最值时需同步修改合并与标记逻辑。

用法：

- 输入数组 `a` 使用下标 `1..n`；先构造新对象 `SegmentTree st(n)`，再调用一次 `st.build(a)` 初始化节点和。
- `rangeAdd(l,r,delta)` 令区间内每项增加 `delta`。
- `rangeSum(l,r)` 返回区间和，内部自动下传懒标记。
- 若只需单点修改，可调用 `rangeAdd(i,i,delta)`。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct SegmentTree {
    int n;
    // 节点 p 管理闭区间 [left,right]。
    // sum[p] 已含本节点全部修改；lazy[p] 表示尚未下传给子节点的“每项增量”。
    vector<ll> sum, lazy;

    // 参数 n 为正数组长度；分配区间和与懒标记空间，初始值均为 0。
    explicit SegmentTree(int n) : n(n), sum(4 * n + 4), lazy(4 * n + 4) {}

    // 参数 a 为 1-based 初值；p 管理闭区间 [left,right]，递归写入 sum、不触碰 lazy；区间合法且非空，只应对全零新对象构建。
    void build(const vector<ll>& a, int p, int left, int right) {
        // 输入 a 使用 1-based；递归区间始终是闭区间。
        if (left == right) {
            sum[p] = a[left];
            return;
        }
        int mid = (left + right) / 2;
        build(a, p * 2, left, mid);
        build(a, p * 2 + 1, mid + 1, right);
        sum[p] = sum[p * 2] + sum[p * 2 + 1];
    }
```

```
    // 参数 a[1..n] 是初值；返回 void，只初始化节点 sum，不清 lazy；须对刚构造的新对象调用一次，已有更新后重建需重新构造对象。
    void build(const vector<ll>& a) { build(a, 1, 1, n); }

    // 参数指定节点区间和增量 delta；返回 void，直接修改该节点 sum/lazy。
    void apply(int p, int left, int right, ll delta) {
        // 整段每项增加 delta，所以区间和增加 delta * 区间长度。
        sum[p] += delta * (right - left + 1);
        lazy[p] += delta;
    }

    // 参数 p 管理闭区间 [left,right]；返回 void，将未下传增量写到子节点并清父标记。
    void pushDown(int p, int left, int right) {
        if (lazy[p] == 0 || left == right) return;
        int mid = (left + right) / 2;
        // 下传必须使用子区间真实长度；完成后清空父标记，避免重复应用。
        apply(p * 2, left, mid, lazy[p]);
        apply(p * 2 + 1, mid + 1, right, lazy[p]);
        lazy[p] = 0;
    }

    // ql/qr 为待加闭区间，p,left,right 为当前节点；要求区间合法且相交，返回 void 并修改覆盖节点。
    void rangeAdd(int ql, int qr, ll delta, int p, int left, int right) {
        // ql,qr 是待修改闭区间；调用前保证它合法且与当前区间有交集。
        if (ql <= left && right <= qr) {
            apply(p, left, right, delta);
            return;
        }
        pushDown(p, left, right);
        int mid = (left + right) / 2;
        if (ql <= mid) rangeAdd(ql, qr, delta, p * 2, left, mid);
        if (qr > mid) rangeAdd(ql, qr, delta, p * 2 + 1, mid + 1, right);
        // 子树修改后回收信息，保持 sum[p] 等于整个 [left,right] 的和。
        sum[p] = sum[p * 2] + sum[p * 2 + 1];
    }

    // 参数为合法闭区间及每项增量；返回 void，更新散块元素/整块懒标记。
    void rangeAdd(int left, int right, ll delta) {
        rangeAdd(left, right, delta, 1, 1, n);
    }

    // ql/qr 为查询闭区间，p,left,right 为当前节点；返回两区间交集之和，并可能下传懒标记。
    ll rangeSum(int ql, int qr, int p, int left, int right) {
        // 完全覆盖直接返回；部分覆盖前先下传，保证子节点信息最新。
        if (ql <= left && right <= qr) return sum[p];
        pushDown(p, left, right);
        int mid = (left + right) / 2;
        ll result = 0;
        if (ql <= mid) result += rangeSum(ql, qr, p * 2, left, mid);
        if (qr > mid) result += rangeSum(ql, qr, p * 2 + 1, mid + 1, right);
        return result;
    }

    // 参数为合法 1-based 闭区间；返回区间和，查询可能下传内部懒标记。
    ll rangeSum(int left, int right) {
        return rangeSum(left, right, 1, 1, n);
    }
};
```

易错点：

1. 区间和增加量是 `delta * 区间长度`，不是只加 `delta`。
2. 访问子节点前必须下传标记；下传后清空父标记。
3. `n=0` 时不能建树；调用区间必须满足 `1<=l<=r<=n`。
4. 区间赋值与区间加法的标记组合不同，不能直接混用此模板。

参考：

- OI Wiki：线段树

5.8 ST 表（静态 RMQ）

适用范围： 适用于不可修改数组上的幂等区间运算，如最大值、最小值和 GCD。 **能做什么：** 预处理后 O(1) 查询区间最大值。 **输入 / 输出：** 输入：1-based 非空数组及合法闭区间；输出：区间最大值。

问题与思路： 静态数组上反复查询区间最大/最小值时使用。预处理每个起点、长度为 2^j 的答案；查询 `[l,r]` 用两个可重叠的同长度区间合并。预处理 $O(n\log n)$、查询 $O(1)$、空间 $O(n\log n)$。仅适合可重复贡献的运算，如最大、最小、GCD；数组不能修改。

用法：

- 模板维护最大值，输入数组下标为 `1..n`。
- `query(l,r)` 要求 `1<=l<=r<=n`。
- 改最小值：将两处 `max` 改为 `min`。
- 区间和不可用重叠区间合并，应使用前缀和、树状数组或线段树。

```
#include <bits/stdc++.h>
using namespace std;

struct SparseTableMax {
    int n, levels;
    vector<int> log2Floor;
    // st[j][i] 保存闭区间 [i, i+2^j-1] 的最大值；输入 a 为 1-based。
    vector<vector<int>> st;

    // 参数 a 为 1-based 非空静态数组；构造各级区间最大值表，不修改 a。
    explicit SparseTableMax(const vector<int>& a) {
        n = (int)a.size() - 1;
        log2Floor.assign(n + 1, 0);
        // log2Floor[i] = floor(log2(i))，查询时可 O(1) 找到覆盖长度。
        for (int i = 2; i <= n; ++i) log2Floor[i] = log2Floor[i / 2] + 1;

        levels = log2Floor[n] + 1;
        st.assign(levels, vector<int>(n + 1));
        for (int i = 1; i <= n; ++i) st[0][i] = a[i];

        for (int j = 1; j < levels; ++j) {
            int length = 1 << j;
            for (int i = 1; i + length - 1 <= n; ++i) {
                // 长为 2^j 的区间由两个相邻、长为 2^(j-1) 的区间组成。
                st[j][i] = max(st[j - 1][i], st[j - 1][i + length / 2]);
            }
        }
    }

    // 参数为合法闭区间 1<=left<=right<=n；返回该区间最大值，不修改表。
```

```
    int query(int left, int right) const {
        // max 可重复贡献：用两个可能重叠的 2^j 区间覆盖 [left,right]。
        int j = log2Floor[right - left + 1];
        return max(st[j][left], st[j][right - (1 << j) + 1]);
    }
};
```

易错点：

1. 构造函数要求至少有一个有效元素，且 `a[0]` 不使用。
2. 第二段起点是 `r-2^j+1`，容易漏掉 `+1`。
3. ST 表建成后不能修改原数组。

参考：

- OI Wiki：ST 表

5.9 分块（区间加、区间和）

适用范围： 适用于可分块维护、操作以区间加和区间和为主的数组。 **能做什么：** 支持区间加和区间和查询，单次 O(√n)。 **输入 / 输出：** 输入：1-based 非空数组及合法闭区间操作；输出：区间和，更新直接修改结构。

问题与思路： 将数组按约 $\sqrt{n}$ 个元素分块，整块用汇总值和懒标记处理，零散部分直接扫描。适合操作容易整块维护、又不想写线段树的题。模板区间加与区间和均为 $O(\sqrt{n})$，建表与空间 $O(n)$；常数较小但渐进复杂度弱于线段树。

用法：

- 输入数组下标为 `1..n`，构造 `SqrtDecomposition ds(a)`。
- `rangeAdd(l,r,delta)` 对闭区间加值。
- `rangeSum(l,r)` 查询闭区间和。
- `sum[b]` 保存原始数组在整块的和，`lazy[b]` 保存整块每项额外增加量。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct SqrtDecomposition {
    int n, blockSize, blockCount;
    // value[i] 与 sum[b] 均不含 lazy[b]；真实值为 value[i]+lazy[belong[i]]。
    vector<ll> value, sum, lazy;
    // belong 映射 1-based 下标到 0-based 块；块端点均为闭区间。
    vector<int> belong, leftEnd, rightEnd;

    // 参数 a[1..n] 为非空初始数组；构造块和懒标记副本，不修改 a。
    explicit SqrtDecomposition(const vector<ll>& a) {
        // 输入 a[1..n]，a[0] 不使用；要求 n>=1。
        n = (int)a.size() - 1;
        blockSize = max(1, (int)sqrt(n));
        blockCount = (n + blockSize - 1) / blockSize;
        value = a;
        sum.assign(blockCount, 0);
        lazy.assign(blockCount, 0);
        belong.resize(n + 1);
```

```
        leftEnd.resize(blockCount);
        rightEnd.resize(blockCount);

        for (int b = 0; b < blockCount; ++b) {
            leftEnd[b] = b * blockSize + 1;
            rightEnd[b] = min(n, (b + 1) * blockSize);
            for (int i = leftEnd[b]; i <= rightEnd[b]; ++i) {
                belong[i] = b;
                sum[b] += value[i];
            }
        }
    }

    // 参数为合法闭区间及每项增量；返回 void，更新散块元素/整块懒标记。
    void rangeAdd(int left, int right, ll delta) {
        // 修改闭区间 [left,right]: 散块写 value/sum，完整块只累加 lazy。
        int bl = belong[left], br = belong[right];
        if (bl == br) {
            for (int i = left; i <= right; ++i) value[i] += delta;
            sum[bl] += delta * (right - left + 1);
            return;
        }
        for (int i = left; i <= rightEnd[bl]; ++i) value[i] += delta;
        sum[bl] += delta * (rightEnd[bl] - left + 1);
        for (int b = bl + 1; b < br; ++b) lazy[b] += delta;  // 不改原始块和
        for (int i = leftEnd[br]; i <= right; ++i) value[i] += delta;
        sum[br] += delta * (right - leftEnd[br] + 1);
    }

    // 参数为合法闭区间；返回当前区间和，不修改分块状态。
    ll rangeSum(int left, int right) const {
        // 查询闭区间 [left,right]; 散块逐项补 lazy，整块一次补标记贡献。
        int bl = belong[left], br = belong[right];
        ll result = 0;
        if (bl == br) {
            for (int i = left; i <= right; ++i) result += value[i] + lazy[bl];
            return result;
        }
        for (int i = left; i <= rightEnd[bl]; ++i) result += value[i] + lazy[bl];
        for (int b = bl + 1; b < br; ++b) {
            result += sum[b] + lazy[b] * (rightEnd[b] - leftEnd[b] + 1);
        }
        for (int i = leftEnd[br]; i <= right; ++i) result += value[i] + lazy[br];
        return result;
    }
};
```

易错点：

1. 整块懒标记没有写回 `value`，扫描零散元素时必须额外加上本块标记。
2. `sum` 只保存不含 `lazy` 的块内原值和，查询整块时再计算标记贡献。
3. 最后一块长度可能不足 `blockSize`，应使用实际左右端点。

参考：

- OI Wiki：分块

5.10 树状数组：区间加与区间和

适用范围： 适用于数组区间加、区间和查询且可复用单点 Fenwick 的场景。 **能做什么：** 用两棵树状数组以 O(log n) 执行区间增量及求和。 **输入 / 输出：** 输入：1-based 长度 n 数组/区间操作；输出：区间和，更新修改两棵树。

用两棵树维护差分：前缀和为 `x*sum(B1,x)-sum(B2,x)`；区间修改/查询均 $O(\log n)$。本代码复用 5.6 节的 `Fenwick`（`add` 与 `prefixSum`）。

```
struct RangeFenwick {
    // 两棵树保存差分系数，prefix(x)=x*B1.prefix-B2.prefix。
    int n;
    Fenwick b1,b2;
    // 参数 size 为正数组长度；初始化两棵差分 Fenwick，逻辑数组初始为零。
    explicit RangeFenwick(int size) : n(size), b1(size), b2(size) {}
    // 参数 x 属于 1..n+1，v 是差分增量；返回 void，更新系数树，n+1 哨兵不写树。
    void addPoint(int x, ll v) {
        // 差分在 l 加 v、在 r+1 减 v；n+1 的哨兵不需要写入树中。
        if (x <= n) {
            b1.add(x, v);
            b2.add(x, v * (x - 1));
        }
    }
    // 参数为 1-based 闭区间及增量；返回 void，修改两棵树表示的逻辑数组，空区间无操作。
    void addRange(int l, int r, ll v) {
        if (l > r) {
            return;
        }
        addPoint(l, v);
        addPoint(r + 1, -v);
    }
    // 参数 x 属于 0..n；返回逻辑数组 [1,x] 之和，不修改树。
    ll prefix(int x) const {
        return b1.prefixSum(x) * x - b2.prefixSum(x);
    }
    // 参数为合法闭区间；返回逻辑数组区间和，空区间返回 0。
    ll query(int l, int r) const {
        return l > r ? 0 : prefix(r) - prefix(l - 1);
    }
};
```

坑：初始数组可逐点 `addRange(i,i,a[i])`；乘法必须用 `long long`；`r+1` 超界时不更新。

5.11 DFS 序与子树展平

适用范围： 适用于静态定根树的子树整体操作，可将子树查询转为数组区间操作。 **能做什么：** 把每个子树映射为 DFS 序上的连续区间。 **输入 / 输出：** 输入：1-based 连通无向树和根；输出：tin、tout 及 DFS 顺序。每个对象只 DFS 构建一次；需要换根或重算时重新创建对象。

用途 / 用法： 把有根树的子树映射为连续区间 `[tin[u],tout[u]]`，再接树状数组/线段树；预处理 $O(n)$。

```
struct DfsOrder {
    vector<vector<int>> g;
    vector<int> tin,tout,order;
    int timer=0;
    // 参数 n 为树点数；分配 1-based 图与 DFS 序数组，timer 初始为 0。
    explicit DfsOrder(int n)
```

```
        : g(n + 1), tin(n + 1), tout(n + 1), order(n + 1) {}
    // 参数 u、v 是 1..n 树点；返回 void，向 g 双向加入边；建 DFS 序前每条树边调用一次。
    void addEdge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    // 参数 u 是当前子树根、p 是父点（根传 0）；返回 void，写 tin/tout/order/timer；输入须为
树。
    void dfs(int u, int p) {
        // 进入时分配唯一时间戳，递归结束时的 timer 即子树最后一个时间戳。
        tin[u] = ++timer;
        order[timer] = u;
        for (int v : g[u]) {
            if (v != p) {
                dfs(v, u);
            }
        }
        tout[u] = timer;
    }
};
```

坑：这是每点只记录一次的 DFS 序，不是 LCA 欧拉序；换根后“子树”含义变化；深链注意栈深。

5.12 树链剖分

适用范围： 适用于树上路径拆段后交由线段树等结构处理。 **能做什么：** 把两点路径拆成 O(log n) 个连续编号区间。 **输入 / 输出：** 输入：1-based 连通无向树和路径端点；输出：通过回调交付路径区间，端点次序不保证。

用途 / 用法： 把树上路径拆成 $O(\log n)$ 个连续 DFS 区间；配合线段树后路径查询通常 $O(\log^2 n)$。

```
struct HLD {
    // heavy 子树最大者优先同链；任意树路径被拆成 O(log n) 个连续区间。
    int n,timer=0;
    vector<vector<int>> g;
    vector<int> parent,depth,sz,heavy,top,pos;
    // 参数 nodeCount 为点数；分配 1-based 邻接表和剖分数组，所有位置初始未构建。
    explicit HLD(int nodeCount)
        : n(nodeCount), g(nodeCount + 1), parent(nodeCount + 1),
          depth(nodeCount + 1), sz(nodeCount + 1),
          heavy(nodeCount + 1, -1), top(nodeCount + 1), pos(nodeCount + 1) {}
    // 参数 u 为当前子树根、p 为父点；返回子树大小，并写 parent/depth/sz/heavy；须在树上调
用。
    int dfs1(int u, int p) {
        // 后序汇总子树大小，并选出最大的孩子作为重儿子。
        parent[u] = p;
        sz[u] = 1;
        int best = 0;
        for (int v : g[u]) {
            if (v == p) {
                continue;
            }
            depth[v] = depth[u] + 1;
            int sub = dfs1(v, u);
            sz[u] += sub;
            if (sub > best) {
                best = sub;
                heavy[u] = v;
            }
```

```
        }
        return sz[u];
    }
    // 参数 u 为子树根、head 为其重链链头；返回 void，按重儿子优先写 top/pos 并递增 timer。
    void dfs2(int u, int head) {
        // 重儿子先访问以保证同一重链映射为连续位置。
        top[u] = head;
        pos[u] = ++timer;
        if (heavy[u] != -1) {
            dfs2(heavy[u], head);
        }
        for (int v : g[u]) {
            if (v != parent[u] && v != heavy[u]) {
                dfs2(v, v);
            }
        }
    }
    // 参数 root 是 1..n 的定根；返回 void，以当前 g 首次建立父深度、重儿子和重链位置。要求新对象且先加完所有边；重复 build 前须重新创建对象，因为 timer/heavy 不会清零。
    void build(int root = 1) {
        dfs1(root, 0);
        dfs2(root, root);
    }
    // 参数 u/v 为树上端点，use(l,r) 接收闭区间且返回值被忽略，可更新调用方维护的结构；返回 void，仅改局部端点副本，区间顺序不保证路径方向。
    template<class F>
    void forPath(int u, int v, F use) { // use(l,r): 点权区间
        while (top[u] != top[v]) {
            if (depth[top[u]] < depth[top[v]]) {
                swap(u, v);
            }
            use(pos[top[u]], pos[u]);
            u = parent[top[u]];
        }
        if (depth[u] > depth[v]) {
            swap(u, v);
        }
        use(pos[u], pos[v]);
    }
};
```

坑：边权映射到较深端点，最后同链区间应排除 LCA；非交换操作要保留路径方向；记得先向 g 加边再 build。

5.13 线段树：区间加、赋值与求和

适用范围： 适用于需要混合区间加、区间赋值和区间和的数组。 **能做什么：** 维护两类懒标记并以 O(log n) 支持更新和查询。 **输入 / 输出：** 输入：1-based 初始数组及合法闭区间操作；输出：区间和，更新直接修改结构。

用途 / 用法： 同时支持区间加、区间赋值与区间和，每次操作 $O(\log n)$。赋值覆盖旧加法；赋值之后的新加法累加到赋值值上。以下代码与 5.7 节的区间加树用途相近，但懒标记额外维护覆盖操作。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct AssignAddSegmentTree {
```

```
    struct Node {
        ll sum = 0;
        ll add = 0;
        ll setValue = 0;
        bool hasSet = false; // 赋值为 0 也有效，不能用 setValue==0 表示“无标记”
    };
    int n;
    vector<Node> tr;

    // 参数 size 为正数组长度；分配初始零值节点，标记均无效。
    explicit AssignAddSegmentTree(int size) : n(size), tr(4 * size + 4) {}

    // 参数 p 管理闭区间 [l,r]、value 为赋值；返回 void，覆盖该节点和并设置赋值标记。
    void applySet(int p, int l, int r, ll value) {
        // 覆盖整个节点：清除旧加法，并留下待下传的赋值。
        tr[p].sum = value * (r - l + 1);
        tr[p].setValue = value;
        tr[p].add = 0;
        tr[p].hasSet = true;
    }
    // 参数 p 管理闭区间 [l,r]、delta 为增量；返回 void，修改该节点和并合并加法/赋值标记。
    void applyAdd(int p, int l, int r, ll delta) {
        tr[p].sum += delta * (r - l + 1);
        // 赋值标记仍在时把增量合并到赋值值，否则累积为独立加法标记。
        if (tr[p].hasSet) tr[p].setValue += delta;
        else tr[p].add += delta;
    }
    // 参数 p 管理闭区间 [l,r]；返回 void，按赋值后加法顺序下传标记并清理父标记。
    void push(int p, int l, int r) {
        if (l == r) return;
        int m = (l + r) / 2;
        // 操作顺序必须是先赋值后加法，才能保持时间顺序。
        if (tr[p].hasSet) {
            applySet(p * 2, l, m, tr[p].setValue);
            applySet(p * 2 + 1, m + 1, r, tr[p].setValue);
            tr[p].hasSet = false;
        }
        if (tr[p].add != 0) {
            applyAdd(p * 2, l, m, tr[p].add);
            applyAdd(p * 2 + 1, m + 1, r, tr[p].add);
            tr[p].add = 0;
        }
    }
    // 参数 a 为 1-based 初值，p 管理闭区间 [l,r]；返回 void，递归建立节点和。
    void build(int p, int l, int r, const vector<ll>& a) {
        if (l == r) {
            tr[p].sum = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m + 1, r, a);
        tr[p].sum = tr[p * 2].sum + tr[p * 2 + 1].sum;
    }
    // 参数 a[1..n] 为初值；返回 void，重建树节点和及标记状态。
    void build(const vector<ll>& a) { build(1, 1, n, a); }

    // 参数 p/l/r 为当前节点，ql/qr 为合法修改区间，value 为操作值、assign 选择赋值或加法；
返回 void 并更新节点。
    void update(int p, int l, int r, int ql, int qr, ll value, bool assign) {
        if (ql <= l && r <= qr) {
```

```
            if (assign) applySet(p, l, r, value);
            else applyAdd(p, l, r, value);
            return;
        }
        push(p, l, r);
        int m = (l + r) / 2;
        if (ql <= m) update(p * 2, l, m, ql, qr, value, assign);
        if (qr > m) update(p * 2 + 1, m + 1, r, ql, qr, value, assign);
        tr[p].sum = tr[p * 2].sum + tr[p * 2 + 1].sum;
    }
    // 参数为合法闭区间及增量；返回 void，将区间每项增加 value 并修改树状态。
    void rangeAdd(int l, int r, ll value) { update(1, 1, n, l, r, value, false); }
    // 参数为合法闭区间及赋值；返回 void，将区间每项设为 value 并修改树状态。
    void rangeSet(int l, int r, ll value) { update(1, 1, n, l, r, value, true); }

    // 参数 p/l/r 为当前节点，ql/qr 为合法查询区间；返回交集和，并可能下传懒标记。
    ll query(int p, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return tr[p].sum;
        push(p, l, r);
        int m = (l + r) / 2;
        ll result = 0;
        if (ql <= m) result += query(p * 2, l, m, ql, qr);
        if (qr > m) result += query(p * 2 + 1, m + 1, r, ql, qr);
        return result;
    }
    // 参数为合法闭区间；返回区间和，查询可能下传内部标记。
    ll rangeSum(int l, int r) { return query(1, 1, n, l, r); }
};
```

构造 `AssignAddSegmentTree st(n)` 后，用 `build(a)` 初始化 `a[1..n]`；`rangeSet(l,r,v)` 赋值，`rangeAdd(l,r,v)` 增加，`rangeSum(l,r)` 求和。要求 `n>=1`、区间合法，且乘积和节点和均能放入 `long long`。

5.14 DSU on Tree（树上启发式合并）

适用范围： 静态有根树上需要逐点回答子树颜色种类数等可由频次集合维护的问题。 **能做什么：** 重儿子答案保留，轻儿子子树临时统计并清除，使每个点仅在轻边层级被重复加入，支持子树 distinct color 计数。 **输入 / 输出：** 输入：1-based 连通无向树、每点任意整数颜色及根；输出：每个点子树内不同颜色数。颜色压缩耗时 $O(n\log n)$，统计耗时 $O(n\log n)$，空间 $O(n)$。仅适用于离线静态子树问题，不支持在线颜色更新或路径查询。

模板的 `build()` 重新定根并计算子树区间和重儿子；`solve()` 每次先清空频次与答案再重算，因此同一对象重复调用结果一致。调用前须提供 `n>=1` 的合法树。实现使用递归，链状树深度可达 n，超大树需考虑栈限制或改为迭代实现。

```
#include <bits/stdc++.h>
using namespace std;

struct DSUOnTree {
    int n, root;
    const vector<vector<int>>& tree;
    const vector<int>& color;
    vector<int> compressed, parent, subtreeSize, heavy;
    vector<int> tin, tout, order, frequency, answer;
    int timer = 0, distinct = 0;

    // 输入 tree 为 1-based 连通无向树，color[1..n] 为任意整数颜色；引用对象须长于本对象。
```

```
    // 初始化数组；不修改 tree 或 color。要求 n>=1 且 root 在 1..n。
    DSUOnTree(const vector<vector<int>>& tree, const vector<int>& color, int root)
        : n((int)tree.size() - 1), root(root), tree(tree), color(color),
          compressed(n + 1), parent(n + 1), subtreeSize(n + 1), heavy(n + 1, 0),
          tin(n + 1), tout(n + 1), order(n + 1), answer(n + 1) {}

    // 对 u 子树做一次 DFS 序展平，计算 parent、子树大小和最大子树的重儿子；返回
subtreeSize[u]。
    // p 是 u 的父节点，根调用时 p=0；写入内部预处理数组，不改变输入树。
    int prepare(int u, int p) {
        parent[u] = p;
        subtreeSize[u] = 1;
        tin[u] = ++timer;
        order[timer] = u;
        int largest = 0;
        for (int v : tree[u]) {
            if (v == p) continue;
            int childSize = prepare(v, u);
            subtreeSize[u] += childSize;
            if (childSize > largest) {
                largest = childSize;
                heavy[u] = v;
            }
        }
        tout[u] = timer;
        return subtreeSize[u];
    }

    // 无参数；重新以 root 为根建立 DFS 序区间、父节点、子树大小和重儿子信息；清空旧预处理数
组，不修改输入树。
    // 仅完成结构预处理；颜色频次要由 solve() 初始化后才能用于 dfs() 统计。
    void build() {
        timer = 0;
        fill(parent.begin(), parent.end(), 0);
        fill(subtreeSize.begin(), subtreeSize.end(), 0);
        fill(heavy.begin(), heavy.end(), 0);
        fill(tin.begin(), tin.end(), 0);
        fill(tout.begin(), tout.end(), 0);
        prepare(root, 0);
    }

    // 将节点 x 的颜色频次增加 delta（+1 或 -1）；更新 distinct，保持频次非负。
    // x 是 1..n 点号；只修改内部 frequency/distinct。
    void changeColor(int x, int delta) {
        int id = compressed[x];
        if (frequency[id] == 0 && delta == 1) ++distinct;
        frequency[id] += delta;
        if (frequency[id] == 0 && delta == -1) --distinct;
    }

    // 将以 u 为根的整棵子树加入当前频次表或移除；add=true 表示加入。
    // u 必须已由 prepare 建立 DFS 区间；只修改内部频次状态。
    void addSubtree(int u, bool add) {
        int delta = add ? 1 : -1;
        for (int i = tin[u]; i <= tout[u]; ++i) changeColor(order[i], delta);
    }

    // 处理 u 子树；keep=true 时保留该子树颜色频次供父层合并，false 时清空。
    // p 为父节点；结束时 answer[u] 是子树 distinct 数，频次表依 keep 决定是否保留。
    void dfs(int u, int p, bool keep) {
        for (int v : tree[u]) {
```

```
            if (v != p && v != heavy[u]) dfs(v, u, false);
        }
        if (heavy[u] != 0) dfs(heavy[u], u, true);
        for (int v : tree[u]) {
            if (v != p && v != heavy[u]) addSubtree(v, true);
        }
        changeColor(u, 1);
        answer[u] = distinct;
        if (!keep) addSubtree(u, false);
    }

    // 重新预处理并计算所有子树答案；每次调用都会清空并重建结果，允许安全重复调用。
    // 返回 n+1 长度数组，answer[0] 不使用；不修改输入 tree/color。
    vector<int> solve() {
        build();
        fill(answer.begin(), answer.end(), 0);

        vector<int> values(color.begin() + 1, color.begin() + n + 1);
        sort(values.begin(), values.end());
        values.erase(unique(values.begin(), values.end()), values.end());
        for (int u = 1; u <= n; ++u) {
            compressed[u] = lower_bound(values.begin(), values.end(), color[u]) -
values.begin();
        }
        frequency.assign(values.size(), 0);
        distinct = 0;
        dfs(root, 0, true);
        return answer;
    }
};

// 示例：tree 为 1-based 无向邻接表，color[1..n] 可使用任意整数；答案按点号读取。
// DSUOnTree solver(tree, color, 1); vector<int> distinct = solver.solve();
```

5.15 单点修改线段树与自定义合并

适用范围： 长度固定的非空序列上的区间聚合与单点替换；区间合并须满足结合律，并提供左右两侧均为单位元的值。**能做什么：** 用调用者提供的 `merge` 维护满足上述代数条件的摘要，查询闭区间聚合；合并器须可在 `const` 查询中调用。示例存储仿射函数，演示可自定义合并规则与“维护函数值”是两件事。**输入 / 输出：** 构造器接收 1-based `initial[1..n]`、单位元和合并器；`pointSet(pos,value)` 替换一个位置，`rangeQuery(l,r)` 返回 `[l,r]` 按原序合并结果，要求 `1<=l<=r<=n`。时间：构造 $O(n)$、单点修改与区间查询 $O(\log n)$；空间 $O(n)$。不支持非结合合并或懒区间更新。

仿射示例以模 `1e9+7` 表示 `f(x)=ax+b`，区间中的函数按下标从左到右依次作用，因此 `merge(left,right)=right∘left`，单位元是恒等函数。`a,b` 先归一化到 `[0,MOD)`；乘积小于 `MOD^2`，`long long` 足以容纳中间乘积与相加。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

template<class T, class Merge>
struct PointSegmentTree {
    int n;
    T identity;
    Merge merge;
    vector<T> tree;
```

```
    // initial 为 1-based 非空序列，identity 是 merge 的双侧单位元，merge 满足结合律；复制到
内部树并建树，不修改入参。
    explicit PointSegmentTree(const vector<T>& initial, T identity, Merge merge)
        : n((int)initial.size() - 1), identity(identity), merge(merge), tree(2 * n,
identity) {
        assert(n >= 1);
        for (int i = 1; i <= n; ++i) tree[n + i - 1] = initial[i];
        for (int i = n - 1; i >= 1; --i) tree[i] = this->merge(tree[i << 1], tree[i << 1 |
1]);
    }

    // pos 为 1..n 的位置，value 是替换后的元素；返回 void，更新叶子及其祖先摘要，不改其他位
置。
    void pointSet(int pos, const T& value) {
        int p = n + pos - 1;
        tree[p] = value;
        while (p > 1) {
            p >>= 1;
            tree[p] = merge(tree[p << 1], tree[p << 1 | 1]);
        }
    }

    // l、r 为 1-based 闭区间且 1<=l<=r<=n；返回按原序聚合的摘要，不修改树。
    T rangeQuery(int l, int r) const {
        int left = l + n - 1, right = r + n;
        T leftResult = identity, rightResult = identity;
        while (left < right) {
            if (left & 1) leftResult = merge(leftResult, tree[left++]);
            if (right & 1) rightResult = merge(tree[--right], rightResult);
            left >>= 1;
            right >>= 1;
        }
        return merge(leftResult, rightResult);
    }
};

constexpr ll MOD = 1000000007;

// x 为任意 long long；返回模 MOD 的 [0,MOD) 代表元。
ll normalizeMod(ll x) {
    x %= MOD;
    if (x < 0) x += MOD;
    return x;
}

struct Affine {
    ll a, b;
};

struct ComposeAffine {
    // left、right 的系数均已归一化；返回先执行 left 再执行 right 的复合 right∘left。
    Affine operator()(const Affine& left, const Affine& right) const {
        return {right.a * left.a % MOD,
                (right.a * left.b + right.b) % MOD};
    }
};

// 原始系数可为任意 long long；返回系数归一化后的 f(x)=ax+b。
Affine makeAffine(ll a, ll b) {
    return {normalizeMod(a), normalizeMod(b)};
```

```
}

// 示范仿射函数的顺序组合与点替换；仅修改本地线段树，返回区间函数作用于 x 的结果。
ll affineExample() {
    vector<Affine> functions(4);
    functions[1] = makeAffine(2, 1);
    functions[2] = makeAffine(3, 4);
    functions[3] = makeAffine(5, 6);
    PointSegmentTree<Affine, ComposeAffine> st(functions, makeAffine(1, 0),
ComposeAffine{});
    st.pointSet(2, makeAffine(7, 2));
    Affine combined = st.rangeQuery(1, 3);
    ll x = normalizeMod(11);
    return (combined.a * x + combined.b) % MOD;
}
```

`merge` 是可替换的合并规则，存储类型 `T` 则决定区间摘要是什么。仿射例子可逐点替换函数，查询后得到的组合函数再作用于输入值；同一模板也可用整数与 `sum`/`min` 合并器维护区间和或最小值。

6. 图论

本章默认点编号为 `1..n`。无向边必须加入两个方向；权值和距离使用 `long long`。若题目允许重边，应保留全部边，或在矩阵建图时取最小权值。DFS 与 BFS 基础统一见第 3.8、3.9 节。

6.1 拓扑排序

适用范围： 适用于有向无环图的依赖排序和 DAG 动态规划。 **能做什么：** 返回满足每条有向边先后关系的一个拓扑序，并检测环。 **输入 / 输出：** 输入：1-based 有向邻接表及逐边入度；输出：拓扑点序，有环时为空。

问题与思路： 把有向无环图排成每条边都从前指向后的序列，适合先修关系、任务依赖与 DAG DP。统计入度，将所有入度为 0 的点入队，取出后删除其出边。邻接表实现时间 $O(n+m)$、空间 $O(n+m)$。结果不足 n 个点说明存在有向环，因而没有拓扑序。

用法：

- 建有向边 `u -> v`，并令 `indegree[v]++`。
- `topologicalSort` 返回任意一个拓扑序；有环返回空数组。
- 若需要字典序最小序列，把普通队列换成小根堆。
- 函数按值接收入度，不会破坏调用者保存的原入度。

```
#include <bits/stdc++.h>
using namespace std;

// 参数 graph 为 1-based 有向邻接表，indegree 是逐边统计的入度副本；返回拓扑序，有环返回空表。
vector<int> topologicalSort(
    const vector<vector<int>>& graph,
    vector<int> indegree
) {
    // graph[u] 只保存有向出边 u->v；indegree 必须逐条统计所有入边（含重边）。
    int n = (int)graph.size() - 1;
    queue<int> q;
    for (int u = 1; u <= n; ++u) {
        if (indegree[u] == 0) q.push(u);
    }

    // 不变量：队列中的点在“删去已输出点及其出边后”入度为 0。
    vector<int> order;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        for (int v : graph[u]) {
            if (--indegree[v] == 0) q.push(v);
        }
    }

    if ((int)order.size() != n) return {};  // 有向环导致剩余点无法降至入度 0
    return order;
}
```

易错点：

1. 计算的是入度而不是出度。
2. 重边需逐条计入入度，也会在遍历时逐条减少。
3. 返回空数组既可能表示有环，也可能是 `n=0`；本章默认 `n>=1`。

参考：

- OI Wiki：拓扑排序

6.2 Dijkstra 单源最短路

适用范围： 适用于所有边权非负的有向或无向加权图。 **能做什么：** 求单源最短距离并记录一棵最短路前驱树。 **输入 / 输出：** 输入：1-based 邻接表、非负边权和起点；输出：距离及前驱数组，不可达距离为 INF。

问题与思路： 求一个起点到所有点的最短距离，适用于有向或无向图，但边权必须非负。用小根堆反复取当前距离最小的点并松弛出边，旧堆记录直接跳过。复杂度 $O((n+m)\log n)$，空间 $O(n+m)$。负边会破坏正确性；不可达点保持 `INF`。

用法：

- 有向边只加入 `graph[u]`；无向边同时加入 `u->v` 与 `v->u`。
- 保留重边没有问题，算法会自动选更优边。
- `dist[v]==INF` 表示不可达；`previous` 可还原路径。
- 所有权值必须 `>=0`，距离与权值使用 `long long`；有限最短距离须小于 `INF` 且加法不越界。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = numeric_limits<ll>::max() / 4;

struct Edge {
    // 邻接表中的一项表示有向边“当前点 -> to”；无向边需手动加入反向项。
    int to;
    ll weight;
};

struct DijkstraResult {
    vector<ll> distance;  // INF 表示从 start 不可达
    vector<int> previous; // 最短路树中的前驱；start 和不可达点均为 -1
};

// 参数 graph 为 1-based 非负权邻接表、start 为合法起点；返回各点距离和前驱，不修改图。
DijkstraResult dijkstra(
    const vector<vector<Edge>>& graph,
    int start
) {
    // 仅适用于所有边权 >=0；允许重边。点编号为 1..n。
    int n = (int)graph.size() - 1;
    vector<ll> distance(n + 1, INF);
    vector<int> previous(n + 1, -1);
    // 小根堆保存“入堆时距离,点”；同一点可能有多条新旧记录。
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;

    distance[start] = 0;
    pq.push({0, start});
    while (!pq.empty()) {
        auto [currentDistance, u] = pq.top();
```

```
        pq.pop();
        if (currentDistance != distance[u]) continue;  // 跳过已经失效的旧记录

        for (const Edge& edge : graph[u]) {
            int v = edge.to;
            // u 被取出时 distance[u] 已是当前最优；这里松弛一条出边 u->v。
            if (distance[v] > distance[u] + edge.weight) {
                distance[v] = distance[u] + edge.weight;
                previous[v] = u;
                pq.push({distance[v], v});
            }
        }
    }
    return {distance, previous};
}

// 参数 start/target 为点号，previous 是最短路前驱表；返回路径点序，不可达时空表；不改输入。
    vector<int> restorePath(int start, int target, const vector<int>& previous) {
    // 返回 start 到 target 的点序列；前驱链无法到 start 时返回空数组。
    vector<int> path;
    for (int v = target; v != -1; v = previous[v]) {
        path.push_back(v);
        if (v == start) break;
    }
    if (path.empty() || path.back() != start) return {};
    reverse(path.begin(), path.end());
    return path;
}
```

易错点：

1. 存在负权边时不能使用 Dijkstra，即使负边不从起点直接发出。
2. 无向边必须加入两个方向；重边不应错误覆盖成较大权值。
3. 必须跳过小根堆中的旧距离记录。
4. 不可达时不能直接拿 `INF` 参与后续加法或输出为普通距离。

参考：

- OI Wiki：最短路

6.3 Bellman–Ford 与负环判断

适用范围： 适用于含负边的单源最短路及起点可达负环检测。 **能做什么：** 计算可达最短距离并报告起点可达负环。 **输入 / 输出：** 输入：点数、起点及有向边表；输出：距离数组与负环布尔值。

问题与思路： 单源最短路含负权边时使用。连续扫描所有边并松弛，最多做 $n-1$ 轮；第 n 轮仍可松弛说明存在“起点可达”的负环，最短路无定义。时间 $O(nm)$、空间 $O(n)$，适合规模较小或用于差分约束。不可达点不能参与松弛。

用法：

- 边表 `DirectedEdge{from,to,weight}` 表示一条有向边；无向负边会立即构成往返负环。
- 返回值中的布尔量只判断从 `start` 可达的负环。
- 无负环时，`distance[v]==INF` 表示不可达。
- 若某轮没有更新，可提前结束。

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = numeric_limits<ll>::max() / 4;

struct DirectedEdge {
    // 边方向固定为 from -> to；无向边若需要须显式加入两个方向。
    int from, to;
    ll weight;
};

// 参数 n/start/edges 为点数、起点和有向边表；返回距离及起点可达负环标志，不改边表。
pair<vector<ll>, bool> bellmanFord(
    int n,
    int start,
    const vector<DirectedEdge>& edges
) {
    // 用更宽的中间值避免 long long 松弛溢出；有限答案须能放入 ll 且小于 INF。
    const __int128_t BIG = (__int128_t)1 << 120;
    vector<__int128_t> work(n + 1, BIG);
    work[start] = 0;

    // 每轮扫描所有边；原地松弛可一次推进多条边，但 n 轮判环结论仍成立。
    for (int round = 1; round <= n; ++round) {
        bool changed = false;
        for (const auto& edge : edges) {
            if (work[edge.from] == BIG) continue; // 不可达点不能参与松弛
            __int128_t candidate = work[edge.from] + edge.weight;
            if (candidate < work[edge.to]) {
                work[edge.to] = candidate;
                changed = true;
                if (round == n) return {vector<ll>(n + 1, INF), true};
            }
        }
        if (!changed) break;
    }

    vector<ll> distance(n + 1, INF);
    for (int v = 1; v <= n; ++v) {
        if (work[v] != BIG) {
            // 调用方需保证所有可达最短距离均在 ll 范围内且严格小于 INF。
            distance[v] = (ll)work[v];
        }
    }
    return {distance, false};
}
```

易错点：

1. 不跳过不可达起点会发生 `INF + 负数` 的伪松弛。
2. 第 n 轮更新只说明存在起点可达负环；图中其他负环不一定被发现。
3. 有可达负环时，受其影响的点不存在有限最短距离；此实现只返回负环标志，不提供部分距离。
4. 所有有限距离必须能表示为 `long long` 且严格小于 `INF`；中间松弛用 `__int128` 避免常见边权累加溢出。

参考：

- OI Wiki：Bellman–Ford 算法

6.4 Floyd 全源最短路

适用范围： 适用于点数较小、需要所有点对距离的加权图，可含负边但不能有负环。 **能做什么：** 原地计算所有点对最短距离。 **输入 / 输出：** 输入：1-based 距离矩阵，INF 表示无边；输出：原地更新后的全源距离矩阵。

问题与思路： 求任意两点间最短路，适合点数较小的稠密图，并允许负边。依次允许编号 `1..k` 的点作为中转，更新 `dist[i][j]`。时间 $O(n^3)$、空间 $O(n^2)$。不能存在负环；完成后若 `dist[i][i]<0`，说明存在负环。不可达状态必须先判断再相加。

用法：

- 先构造 `(n+1)×(n+1)` 矩阵：对角线为 0，其余为 `INF`。
- 加边时用 `min(dist[u][v],w)` 处理重边；无向边同时更新两个方向。
- 调用后 `dist[u][v]==INF` 表示不可达。
- 模板原地修改距离矩阵。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = numeric_limits<ll>::max() / 4;

// 参数 n 为点数；返回 n+1 方阵，1..n 对角为 0、其余 INF；分配新矩阵。
    vector<vector<ll>> makeDistanceMatrix(int n) {
    // 矩阵使用 1..n；distance[u][v] 表示有向距离，初始仅自身到自身为 0。
    vector<vector<ll>> distance(n + 1, vector<ll>(n + 1, INF));
    for (int i = 1; i <= n; ++i) distance[i][i] = 0;
    return distance;
}

// 参数为距离矩阵、端点和权值；返回 void，以较小权值写 distance[u][v]；不自动添加反向边。
    void addDirectedEdge(vector<vector<ll>>& distance, int u, int v, ll weight) {
    // 保留同方向重边中的最小值；无向边需再调用一次 (v,u,weight)。
    distance[u][v] = min(distance[u][v], weight);
}

// 参数是 1-based 方阵；返回 void，原地覆盖为全源最短距离，须无负环且加法不溢出。
    void floydWarshall(vector<vector<ll>>& distance) {
    int n = (int)distance.size() - 1;
    // k 必须最外层：完成第 k 轮后，只允许 1..k 作为内部中转点。
    for (int k = 1; k <= n; ++k) {
        for (int i = 1; i <= n; ++i) {
            if (distance[i][k] == INF) continue;
            for (int j = 1; j <= n; ++j) {
                if (distance[k][j] == INF) continue;
                // 两段都可达才相加，避免 INF 参与运算产生伪路径。
                distance[i][j] = min(
                    distance[i][j],
                    distance[i][k] + distance[k][j]
                );
            }
        }
    }
}
```

易错点：

1. 循环顺序必须让中转点 `k` 在最外层。
2. 加边时需要对重边取最小值，不能直接覆盖。
3. 负边可以使用，但存在负环时普通最短距离无定义。
4. 矩阵空间为 $O(n^2)$，点数大时不可用。

参考：

- OI Wiki：Floyd 算法

6.5 Kruskal 最小生成树

适用范围： 适用于连通无向带权图的最小生成树，也能返回不连通图的最小生成森林。 **能做什么：** 按权贪心选边，给出总权和所选边。 **输入 / 输出：** 输入：点数及每条无向边仅出现一次的边表；输出：连通标记、总权和森林边集。

问题与思路： 在连通无向带权图中选 $n-1$ 条边，使总权最小。按边权升序扫描，用并查集加入连接两个不同连通块的边。排序主导时间 $O(m\log m)$，空间 $O(n+m)$；允许负权和重边。若最终不足 $n-1$ 条边，原图不连通，只得到最小生成森林。

用法：

- 每条无向边在边表中只放一次，不需要存两个方向。
- `kruskal(n,edges)` 返回是否连通、总权和、所选边。
- 重边、自环可保留；自环会因端点已连通而被忽略。
- 只求总权时可以删除 `chosen` 以节省少量空间。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct DSU {
    // 只有根的 sizeOf 有效；路径压缩和按大小合并保证近常数均摊复杂度。
    vector<int> parent, sizeOf;
    // 参数 n 是点编号上界；建立 1..n 单元素集合并初始化内部父节点和大小。
    explicit DSU(int n) : parent(n + 1), sizeOf(n + 1, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
    // 参数 x 是 1..n 元素；返回集合代表并压缩访问路径。
    int find(int x) {
        return parent[x] == x ? x : parent[x] = find(parent[x]);
    }
    // 参数 a/b 是 1..n 元素；成功合并返回 true，否则 false；修改代表和集合大小。
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sizeOf[a] < sizeOf[b]) swap(a, b);
        parent[b] = a;
        sizeOf[a] += sizeOf[b];
        return true;
    }
};

struct WeightedEdge {
```

```
    // 无向边在边表中只保存一次；允许负权、重边和自环。
    int u, v;
    ll weight;
};

struct KruskalResult {
    bool connected;
    ll totalWeight;
    vector<WeightedEdge> chosen;
};

// 参数 n 为点数、edges 为每条无向边只存一次的边表副本；返回生成森林信息，不改调用者边表。
    KruskalResult kruskal(int n, vector<WeightedEdge> edges) {
    // 按权从小到大扫描；不成环的最轻边一定可安全加入当前生成森林。
// 比较器参数为两条边；权值较小者排前，同权顺序任意；不修改边对象。
        sort(edges.begin(), edges.end(), [](const auto& a, const auto& b) {
        return a.weight < b.weight;
    });

    DSU dsu(n);
    ll totalWeight = 0;
    vector<WeightedEdge> chosen;
    for (const auto& edge : edges) {
        if (!dsu.unite(edge.u, edge.v)) continue;  // 同一连通块：加入会成环
        totalWeight += edge.weight;
        chosen.push_back(edge);
        if ((int)chosen.size() == n - 1) break;
    }
    return {(int)chosen.size() == n - 1, totalWeight, chosen};
}
```

易错点：

1. 最小生成树只定义在无向连通图上。
2. 图不连通时 `totalWeight` 是生成森林权值，不能当作生成树答案。
3. 总权可能超过 `int`；负边应照常参与排序。

参考：

- OI Wiki：最小生成树

6.6 二分图判定

适用范围： 适用于无向图二部性检查，包括非连通图。 **能做什么：** 染色并判断是否存在奇环。 **输入 / 输出：** 输入：1-based 无向邻接表；输出：是否为二分图及 0/1 染色（失败时染色可能不完整）。

问题与思路： 判断无向图能否把点分成两组，使每条边跨组；等价于图中没有奇环。对每个未染色点 BFS，令相邻点颜色相反，若相邻点同色则失败。时间 $O(n+m)$、空间 $O(n+m)$。非连通图必须逐个连通块染色；自环必然使图不是二分图。

用法：

- 无向边必须双向加入邻接表。
- 返回 `true` 时，`color[u]` 为 0 或 1，给出一组合法划分。
- `color[u]==-1` 表示尚未访问。
- 重边不影响结论；自环会被正确判为冲突。

```cpp
#include <bits/stdc++.h>
using namespace std;

// 参数 graph 为 1-based 无向邻接表；返回是否二分图，并覆盖 color 为染色结果（失败时可能不完整）。
    bool colorBipartite(const vector<vector<int>>& graph, vector<int>& color) {
    // graph 必须是无向图的双向邻接表；-1 未访问，0/1 表示两侧颜色。
    int n = (int)graph.size() - 1;
    color.assign(n + 1, -1);

    // 必须枚举所有起点，才能覆盖非连通图中的每个连通块。
    for (int start = 1; start <= n; ++start) {
        if (color[start] != -1) continue;
        queue<int> q;
        color[start] = 0;
        q.push(start);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : graph[u]) {
                if (color[v] == -1) {
                    // 每条无向边两端必须异色；异或 1 得到另一种颜色。
                    color[v] = color[u] ^ 1;
                    q.push(v);
                } else if (color[v] == color[u]) {
                    return false;
                }
            }
        }
    }
    return true;
}
```

易错点：

1. 只从点 1 开始会漏掉其他连通块中的奇环。
2. 判定模板针对无向图；有向边通常先忽略方向讨论底图。
3. `color` 的 0 是一种颜色，不等于“未访问”。

参考：

- OI Wiki：二分图

6.7 二分图最大匹配（增广路）

适用范围： 适用于已知左右部划分的中等规模二分图匹配。 **能做什么：** 求最大匹配数并维护右部匹配关系。 **输入 / 输出：** 输入：左右部点数及左到右的邻接关系；输出：最大匹配数，matchRight 保存匹配。

问题与思路： 在二分图中选最多条互不共享端点的边，适合任务分配和配对。逐个左部点 DFS 寻找增广路；遇到已匹配右点时，尝试为其原搭档改配。朴素复杂度 $O(VE)$、空间 $O(V+E)$，适合中等规模；更大规模可用 Hopcroft–Karp。

用法：

- 左部点编号 `1..leftCount`，右部点编号 `1..rightCount`。
- `graphLeft[u]` 只保存左点 `u` 能连接的右点。
- 返回最大匹配数；`matchRight[v]` 是右点 `v` 匹配的左点，0 表示未匹配。

- 每轮 DFS 必须使用新的右部访问标记。

```
#include <bits/stdc++.h>
using namespace std;

struct BipartiteMatcher {
    int leftCount, rightCount;
    // graphLeft[u] 只存“左部 u -> 右部 v”，左右编号属于两个独立空间。
    vector<vector<int>> graphLeft;
    vector<int> matchRight; // matchRight[v]=u; 0 表示右点 v 当前未匹配
    vector<int> seen;       // 时间戳：本轮增广中右点是否已尝试
    int visitToken = 0;

    // 参数 leftCount/rightCount 为左右部点数；创建空二分图并初始化匹配及访问时间戳。
    BipartiteMatcher(int leftCount, int rightCount)
        : leftCount(leftCount), rightCount(rightCount),
          graphLeft(leftCount + 1), matchRight(rightCount + 1, 0),
          seen(rightCount + 1, 0) {}

    // 参数 left/right 分别为左右部合法点号；返回 void，将兼容边加入 graphLeft。
    void addEdge(int left, int right) {
        graphLeft[left].push_back(right);
    }

    // 参数 u 是当前待匹配左点；返回是否找到增广路，成功时修改 matchRight 的匹配关系。
    bool augment(int u) {
        // 不变量：尝试为左点 u 找交错路；成功时沿递归链整体改配。
        for (int v : graphLeft[u]) {
            if (seen[v] == visitToken) continue;
            seen[v] = visitToken;
            // v 空闲，或能把 v 的旧搭档改配到别处，就可令 u 占用 v。
            if (matchRight[v] == 0 || augment(matchRight[v])) {
                matchRight[v] = u;
                return true;
            }
        }
        return false;
    }

    // 无参数；返回当前二分图最大匹配数，并在已有匹配上继续增广。
    int maximumMatching() {
        int matching = 0;
        for (int u = 1; u <= leftCount; ++u) {
            ++visitToken; // 每个左点必须拥有独立访问轮次，不能共用 seen 状态
            if (augment(u)) ++matching;
        }
        return matching;
    }
};
```

易错点：

1. 输入必须明确哪边是左部、哪边是右部，编号空间不要混淆。
2. 每次尝试新的左点时都要清空访问状态或增加时间戳。
3. 再次调用 `maximumMatching()` 会在已有匹配上继续运行，不是从空匹配重算。

参考：

- OI Wiki：二分图最大匹配

6.8 强连通分量（Tarjan）

适用范围： 适用于有向图强连通分量划分与缩点。 **能做什么：** 在线性时间为每个点标记 SCC 编号。 **输入 / 输出：** 输入：1-based 有向邻接表；输出：各点分量编号和分量数。

问题与思路： 把有向图分成最大强连通块，块内任意两点互相可达，常用于缩点成 DAG。DFS 记录访问次序 `dfn` 与能回到的最早次序 `low`；当 `low[u]==dfn[u]` 时弹栈得到一个分量。时间 $O(n+m)$、空间 $O(n+m)$。递归深度可达 n。

用法：

- 建有向图，只加入题目给出的方向。
- 构造 `TarjanSCC solver(graph)` 后调用 `run()`。
- `component[u]` 为点所属分量编号，范围 `0..componentCount-1`。
- 缩点时遍历原边；若两端分量不同，则加入分量间边并按需去重。

```
#include <bits/stdc++.h>
using namespace std;

struct TarjanSCC {
    // graph 是 1-based 有向邻接表；只按题目方向加边，重边不影响正确性。
    const vector<vector<int>>& graph;
    int n, timer = 0, componentCount = 0;
    // dfn: 首次访问序；low: 经 DFS 树边和栈内返边能到达的最小 dfn。
    vector<int> dfn, low, component, st;
    vector<char> inStack;

    // 参数 graph 是 1-based 有向邻接表；保存其引用并分配 SCC 工作数组，图须长于对象生命周
期。
    explicit TarjanSCC(const vector<vector<int>>& graph)
        : graph(graph), n((int)graph.size() - 1),
          dfn(n + 1, 0), low(n + 1, 0), component(n + 1, -1),
          inStack(n + 1, false) {}

    // 参数 u 是未访问顶点；返回 void，递归遍历其可达区域并更新 dfn/low/栈/分量编号。
    void dfs(int u) {
        // st 只保存尚未归入任何 SCC 的活跃点，inStack 与之同步。
        dfn[u] = low[u] = ++timer;
        st.push_back(u);
        inStack[u] = true;

        for (int v : graph[u]) {
            if (dfn[v] == 0) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (inStack[v]) {
                // 已访问且仍在栈中才可能属于当前 SCC；此处必须使用 dfn[v]。
                low[u] = min(low[u], dfn[v]);
            }
        }

        if (low[u] == dfn[u]) {
            // u 是一个 SCC 的 DFS 根，弹到 u 为止的点恰属于同一分量。
            while (true) {
                int v = st.back();
                st.pop_back();
                inStack[v] = false;
                component[v] = componentCount;
```

```
                if (v == u) break;
            }
            ++componentCount;
        }
    }

    // 无参数；从所有未访问点启动 DFS 并写 component/componentCount；对象仅应运行一次。
    // 无参数；遍历全部连通区域并填充 SCC 编号及数量；须在对象尚未运行时调用。
    void run() {
        for (int u = 1; u <= n; ++u) {
            if (dfn[u] == 0) dfs(u);
        }
    }
};
```

易错点：

1. 遇到仍在栈中的已访问点时，用 `dfn[v]` 更新，而不是无条件用 `low[v]`。
2. 已弹出栈的点不能再更新当前点的 `low`。
3. 大图递归可能栈溢出，需要增大栈限制或改迭代实现。

参考：

- OI Wiki：强连通分量

6.9 割点与桥（Tarjan）

适用范围： 适用于无向图的割点、桥及非连通图分析。 **能做什么：** 在线性时间识别删点/删边会增加连通块数的对象。 **输入 / 输出：** 输入：点数及逐条添加的无向边；输出：割点标记和桥端点列表。

问题与思路： 无向图中，删去某点使连通块增加则它是割点；删去某边使连通块增加则它是桥。DFS 计算 `dfn`、`low`：树边 `(u,v)` 满足 `low[v]>dfn[u]` 是桥；非根点存在 `low[v]>=dfn[u]` 则为割点，根需至少两个 DFS 子树。时间 $O(n+m)$。

用法：

- 用 `addEdge(u,v)` 添加一条无向边，内部共享同一边编号。
- 调用 `run()` 后，`isCutVertex[u]` 表示割点，`bridges` 保存桥的端点。
- 模板可正确处理重边：DFS 只跳过原树边的编号，不跳过所有父节点边。
- 非连通图会从每个未访问点重新 DFS。

```
#include <bits/stdc++.h>
using namespace std;

struct UndirectedLowlink {
    // 同一条无向边的两个邻接项共享 id；这是正确处理父子重边的关键。
    struct Edge { int to, id; };

    int n, timer = 0, edgeCount = 0;
    vector<vector<Edge>> graph;
    vector<int> dfn, low;
    vector<char> isCutVertex;
    vector<pair<int, int>> bridges;

    // 参数 n 为点数；初始化无向邻接表及割点/桥工作数组。
    explicit UndirectedLowlink(int n)
        : n(n), graph(n + 1), dfn(n + 1), low(n + 1),
```

```
          isCutVertex(n + 1, false) {}

    // 参数 u/v 是 1..n 端点；返回 void，加入一条带共享边编号的无向边。
    void addEdge(int u, int v) {
        // 每条无向边只调用一次，内部自动加入 u->v 与 v->u 两个方向。
        graph[u].push_back({v, edgeCount});
        graph[v].push_back({u, edgeCount});
        ++edgeCount;
    }

    // 参数 u 为当前点、parentEdge 为进入该点的边 id（根传 -1）；返回 void，写 lowlink 并追
加割点/桥。
    void dfs(int u, int parentEdge) {
        // low[u]: 从 u 的 DFS 子树经至多一条非树边能到达的最小 dfn。
        dfn[u] = low[u] = ++timer;
        int childCount = 0;

        for (const Edge& edge : graph[u]) {
            int v = edge.to;
            if (edge.id == parentEdge) continue; // 只跳过来时那一条边，不跳所有父点边
            if (dfn[v] == 0) {
                ++childCount;
                dfs(v, edge.id);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u]) bridges.push_back({u, v}); // 子树无法绕回 u 或更高
                // 非根 u: v 子树无法越过 u 时，删除 u 会把 v 子树分离。
                if (parentEdge != -1 && low[v] >= dfn[u]) isCutVertex[u] = true;
            } else {
                low[u] = min(low[u], dfn[v]);
            }
        }

        // DFS 根没有父侧，只有至少两个独立 DFS 子树时才是割点。
        if (parentEdge == -1 && childCount >= 2) isCutVertex[u] = true;
    }

    // 无参数；遍历无向图所有连通块并追加割点/桥结果；请在 addEdge 完成后调用一次。
    void run() {
        for (int u = 1; u <= n; ++u) {
            if (dfn[u] == 0) dfs(u, -1);
        }
    }
};
```

易错点：

1. 割点条件是 `>=`，桥条件是严格 `>`。
2. DFS 根的割点条件特殊，需至少两个 DFS 子树。
3. 有重边时不能只用 `v==parent` 跳边，必须记录边编号。
4. 该算法针对无向图；有向图应使用强连通分量等概念。

参考：

- OI Wiki：割点和桥

6.10 最近公共祖先（倍增 LCA）

适用范围： 适用于静态连通树上反复查询祖先关系。 **能做什么：** O(log n) 查询两点最近公共祖先，并支持祖先提升。 **输入 / 输出：** 输入：1-based 无向树和根；输出：指定两点的 LCA 点编号。

问题与思路： 在树上反复查询两点最深的公共祖先，也可据此求路径长度。预处理每个点的深度与第 2^j 个祖先；查询时先把深点提升，再同步提升两点。预处理 $O(n\log n)$，单次查询 $O(\log n)$，空间 $O(n\log n)$。模板要求输入为一棵树。

用法：

- 无向树边需双向加入，构造 `BinaryLiftingLCA(graph,root)`。
- `lca(u,v)` 返回最近公共祖先。
- 无权路径边数为 `depth[u]+depth[v]-2*depth[lca(u,v)]`。
- 根可任意选择；不同根会改变祖先关系，但不会改变无权路径长度。

```
#include <bits/stdc++.h>
using namespace std;

struct BinaryLiftingLCA {
    int n, levels;
    vector<int> depth;       // depth[root]=0; 模板只接受一棵连通树
    // up[j][u] 是 u 的第 2^j 个祖先；越过根后统一落到虚拟点 0。
    vector<vector<int>> up;

    // 参数 tree 为 1-based 连通无向树、root 为合法根；预处理祖先表/深度，不修改 tree。
    BinaryLiftingLCA(const vector<vector<int>>& tree, int root) {
        // tree 是 1-based 无向树，必须双向加边且不能含环、重边或自环。
        n = (int)tree.size() - 1;
        levels = 1;
        while ((1 << levels) <= n) ++levels;
        depth.assign(n + 1, -1);
        up.assign(levels, vector<int>(n + 1, 0));

        queue<int> q;
        depth[root] = 0;
        q.push(root);
        // BFS 定根：up[0][v] 是父节点，同时也充当“已访问”的判断依据。
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : tree[u]) {
                if (v == up[0][u]) continue; // 仅跳父边；因此输入必须真的是树
                up[0][v] = u;
                depth[v] = depth[u] + 1;
                q.push(v);
            }
        }

        for (int j = 1; j < levels; ++j) {
            for (int u = 1; u <= n; ++u) {
                // 先跳 2^(j-1)，再从中间祖先跳同样距离。
                up[j][u] = up[j - 1][up[j - 1][u]];
            }
        }
    }

    // 参数 u 为树点、steps 在 0..depth[u]；返回向上 steps 步的祖先，不改预处理表。
```

```
    int lift(int u, int steps) const {
        // 按 steps 的二进制位拆分向上跳；调用者应保证 steps<=depth[u]。
        for (int j = 0; j < levels; ++j) {
            if ((steps >> j) & 1) u = up[j][u];
        }
        return u;
    }

    // 参数 u/v 为同一棵树内的点；返回最近公共祖先点号，不修改预处理表。
    int lca(int u, int v) const {
        // 先对齐深度，再从最高位向下保持“两点祖先仍不同”。
        if (depth[u] < depth[v]) swap(u, v);
        u = lift(u, depth[u] - depth[v]);
        if (u == v) return u;
        for (int j = levels - 1; j >= 0; --j) {
            if (up[j][u] != up[j][v]) {
                u = up[j][u];
                v = up[j][v];
            }
        }
        // 此时 u、v 不同但父节点相同，该父节点就是最近公共祖先。
        return up[0][u];
    }
};
```

易错点：

1. 输入若有环，模板只跳过父边，会重复访问；必须保证它是一棵树。
2. `up` 的 0 号虚拟点各级祖先应保持为 0。
3. 查询点必须和根处于同一棵树，本模板不处理森林跨树查询。
4. `1 << levels` 在极大 `n` 时应改用 `1LL << levels`。

参考：

- OI Wiki：最近公共祖先

6.11 树的直径

适用范围： 适用于边权非负的树；无权树将每条边权设为 1。 **能做什么：** 求树中最长简单路径长度及两个端点。 **输入 / 输出：** 输入：1-based 连通带权树；输出：直径长度与端点。

问题与思路： 树的直径是任意两点间最长简单路径。任选点出发找距离最远点 `a`，再从 `a` 找最远点 `b`，`dist[b]` 即直径；树上路径唯一，所以一次遍历即可算距离。时间 $O(n)$、空间 $O(n)$。两次遍历法适用于无负权树；无权树令边权为 1。

用法：

- `tree[u]` 保存 `{v,w}`，无向边加入两个方向。
- `treeDiameter(tree)` 返回直径长度及两个端点。
- 要还原直径路径，可在第二次遍历中额外记录父节点。
- 模板默认 `n>=1`，且所有边权非负、图连通并有 `n-1` 条边。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
```

```
struct TreeEdge {
    // 无向树边需在两端邻接表各存一份；两次最远点法要求 weight>=0。
    int to;
    ll weight;
};

// 参数 tree 为 1-based 非负边权树、start 为树点；返回最远点及距离，使用局部遍历数组。
    // 参数 tree 为 1-based 非负权连通树、start 为合法点；返回最远点和距离，仅分配局部数组。
    // 参数 tree 为 1-based 非负权连通树、start 为合法点；返回最远点和距离，仅分配局部数组。
pair<int, ll> farthestVertex(
    const vector<vector<TreeEdge>>& tree,
    int start
) {
    // 点编号为 1..n；distance=-1 同时作为未访问标记，因此边权必须非负。
    int n = (int)tree.size() - 1;
    vector<ll> distance(n + 1, -1);
    stack<int> st; // 树上起点到每点路径唯一，无需 Dijkstra 即可累计带权距离
    st.push(start);
    distance[start] = 0;

    while (!st.empty()) {
        int u = st.top();
        st.pop();
        for (const auto& edge : tree[u]) {
            if (distance[edge.to] != -1) continue; // 跳过父节点，避免沿无向边返回
            distance[edge.to] = distance[u] + edge.weight;
            st.push(edge.to);
        }
    }

    int vertex = start;
    for (int u = 1; u <= n; ++u) {
        if (distance[u] > distance[vertex]) vertex = u;
    }
    return {vertex, distance[vertex]};
}

// 参数为 1-based 连通非负带权树且 n>=1；返回 {直径长度,端点a,端点b}，不改树。
    tuple<ll, int, int> treeDiameter(const vector<vector<TreeEdge>>& tree) {
    // 第一次找直径某端点 a，第二次从 a 找另一端 b；返回 {长度,a,b}。
    auto [a, ignored] = farthestVertex(tree, 1);
    auto [b, diameter] = farthestVertex(tree, a);
    return {diameter, a, b};
}
```

易错点：

1. 该模板依赖树上路径唯一，不能直接用于一般带权图。
2. 有负边时“两次找最远点”结论不再可靠。
3. 距离可能超过 `int`，应使用 `long long`。

参考：

- OI Wiki：树的直径

6.12 最大流（Dinic）

适用范围： 适用于非负容量有向网络的最大流/最小割建模。 **能做什么：** 在残量网络上增广并返回最大可发送流量。 **输入 / 输出：** 输入：点数、容量边、不同的源点和汇点；输出：至多 limit 的流量，残量结构会被修改。

问题与思路： 求有向容量网络从源点到汇点的最大可发送流量。残量网络中先 BFS 建分层图，再 DFS 沿层次递增边反复增广；当前弧避免重复扫描。一般复杂度 $O(V^2E)$，空间 $O(V+E)$，竞赛中常用于二分图匹配、资源分配和割模型。容量必须非负。

用法：

- 构造 `Dinic flow(n)`，点编号 `1..n`。
- `addEdge(u,v,cap)` 加一条有向容量边，并自动加容量 0 的反向残量边。
- 无向容量边通常要分别调用两次 `addEdge`，含义需按题目确认。
- `maxFlow(source,sink)` 返回最大流；若只需不超过某上限，可传 `limit`。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = numeric_limits<ll>::max() / 4;

struct Dinic {
    struct Edge {
        int to, reverseIndex;
        ll capacity; // 当前残量，不是原始容量；必须始终非负
    };

    int n;
    vector<vector<Edge>> graph; // graph[u][i] 与其 reverseIndex 指向的残量边互为反边
    vector<int> level;          // 残量网络中从 source 出发的 BFS 层号，-1 为不可达
    vector<int> current;        // 当前弧：本轮分层图中每点尚未尝试的首条边

    // 参数 n 为点数；初始化空残量网络、层次和当前弧数组。
    explicit Dinic(int n)
        : n(n), graph(n + 1), level(n + 1), current(n + 1) {}

    // 参数为端点和非负容量；返回 void，加入正向边及零容量反向边，修改残量图。
    void addEdge(int from, int to, ll capacity) {
        // 加有向容量边 from->to；反向边初始容量为 0，只用于撤回已经发送的流。
        // 自环的两条边都进入同一 vector，正向边的反向下标要跳过自身。
        Edge forward{to, (int)graph[to].size() + (from == to), capacity};
        Edge backward{from, (int)graph[from].size(), 0};
        graph[from].push_back(forward);
        graph[to].push_back(backward);
    }

    // 参数 source/sink 为合法点号；返回汇点是否可达，并重建 level 数组。
    bool buildLevelGraph(int source, int sink) {
        // 只沿正残量边 BFS；后续 DFS 只能走 level 恰好增加 1 的边。
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[source] = 0;
        q.push(source);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
```

```
            for (const Edge& edge : graph[u]) {
                if (edge.capacity <= 0 || level[edge.to] != -1) continue;
                level[edge.to] = level[u] + 1;
                q.push(edge.to);
            }
        }
        return level[sink] != -1;
    }

    // 参数 u/sink 为点号、pushed 为非负上限；返回实际发送量，并原地修改正反残量边。
    ll sendFlow(int u, int sink, ll pushed) {
        // 返回本次从 u 实际送到 sink 的流量；pushed 是上游允许的上限。
        if (u == sink || pushed == 0) return pushed;
        // i 使用引用：失败边以后无需重扫，这就是当前弧优化的不变量。
        for (int& i = current[u]; i < (int)graph[u].size(); ++i) {
            Edge& edge = graph[u][i];
            if (edge.capacity <= 0 || level[edge.to] != level[u] + 1) continue;
            ll sent = sendFlow(edge.to, sink, min(pushed, edge.capacity));
            if (sent == 0) continue;
            // 正向残量减少、反向残量增加，保证之后可以撤销/改道这部分流。
            edge.capacity -= sent;
            graph[edge.to][edge.reverseIndex].capacity += sent;
            return sent;
        }
        return 0;
    }

    // 参数为不同源汇和非负流量上限；返回本次从当前残量网可增广流量，残量图被修改。
    ll maxFlow(int source, int sink, ll limit = INF) {
        // 要求 source!=sink、点编号 1..n；返回不超过 limit 的最大增广量。
        assert(source != sink);
        ll flow = 0;
        while (flow < limit && buildLevelGraph(source, sink)) {
            fill(current.begin(), current.end(), 0); // 新分层图必须重置当前弧
            while (flow < limit) {
                ll sent = sendFlow(source, sink, limit - flow);
                if (sent == 0) break;
                flow += sent;
            }
        }
        return flow;
    }
};
```

最大流可建模二分图匹配：源点连左部、右部连汇点的容量设为 1，左右兼容边也设为 1。若要取最小割，必须让 `maxFlow` 跑到无增广路为止；随后调用 `buildLevelGraph(source,sink)`，它返回 `false`，此时 `level[v]>=0` 的点构成残量可达侧。用原始输入边表累加从可达侧指向不可达侧的容量；不要把反向残量边当作原边计入。若有限 `limit` 提前截断，不能据此读取最小割。

易错点：

1. 反向边是残量边，不是把原边简单复制一份；其初始容量为 0。
2. 修改正向残量后必须同步增加对应反向残量。
3. `source==sink` 时最大流没有通常意义，调用前应保证两者不同。
4. 多次调用 `maxFlow` 会在当前残量网络上继续增广；重算需重新建图。

参考：

- OI Wiki：最大流

6.13 欧拉路径与回路

适用范围： 适用于判定条件满足的无向图边遍历任务。 **能做什么：** Hierholzer 算法构造每条边恰用一次的点序。 **输入 / 输出：** 输入：点数及每条无向边一次的边表；输出：成功时含 m+1 个点的欧拉点序，未覆盖全部边、度数条件不满足或无边时返回空数组；函数内部会检查是否消费全部边。

用途 / 用法： 每条边恰走一次。下例 Hierholzer 求无向图边序对应的点序，$O(n+m)$；平行边靠边编号区分。

```
// 参数 n 为点数、edges 为每条无向边一次的端点表；返回全边欧拉点序，无欧拉遍历或非连通时为空。
vector<int> eulerTrailUndirected(
    int n, const vector<pair<int, int>>& edges) {
    // Hierholzer 回溯入答案，末尾需 reverse；每条边只消费一次。
    vector<vector<pair<int, int>>> g(n + 1);
    vector<int> deg(n + 1), it(n + 1);
    vector<char>used(edges.size());
    for (int id = 0; id < (int)edges.size(); ++id) {
        auto [u, v] = edges[id];
        g[u].push_back({v,id});
        g[v].push_back({u,id});
        ++deg[u];
        ++deg[v];
    }
    int start = 0, odd = 0;
    for (int u = 1; u <= n; ++u) {
        if (deg[u] == 0) {
            continue;
        }
        if (!start) {
            start = u;
        }
        if (deg[u] & 1) {
            ++odd;
            start = u;
        }
    }
    if (!start || (odd != 0 && odd != 2)) {
        return {};
    }
    vector<int> st{start}, path;
    while (!st.empty()) {
        int u = st.back();
        while (it[u] < (int)g[u].size()
               && used[g[u][it[u]].second]) {
            ++it[u];
        }
        if (it[u] == (int)g[u].size()) {
            path.push_back(u);
            st.pop_back();
        } else {
            auto [v, id] = g[u][it[u]++];
            if (!used[id]) {
                used[id] = true;
                st.push_back(v);
            }
        }
```

```
    }
    if (path.size() != edges.size() + 1) {
        return {};
    }
    reverse(path.begin(), path.end());
    return path;
}
```

条件：忽略孤立点后连通；无向回路奇度点为 0，路径为 2。有向图需入出度平衡，并按有向边只加入一次。

坑：结果是逆序回退所得，最后反转；必须检查使用了全部边；无边图的返回约定按题意调整。

6.14 2-SAT

适用范围： 适用于每个布尔变量含两个文字、每条约束为二文字析取的可满足性问题。 **能做什么：** 判断可满足并构造一组布尔赋值。 **输入 / 输出：** 输入：变量数及蕴含/二文字约束；输出：optional<vector>，有解时含下标 1..n 的赋值，无解时为 nullopt。

用途 / 用法： 每个布尔变量二选一、约束是两个文字的析取。建蕴含图后 SCC，$O(n+m)$。

```
struct TwoSAT {
    // 每个变量 x 有 false/true 两个文字节点；蕴含边记在 g 中。
    int vars;
    vector<vector<int>> g;
    TarjanSCC scc; // 复用 6.8 节实现；它保存 g 的引用，故 g 必须先声明

    // 参数 n 为变量数；分配文字蕴含图及引用该图的 SCC 求解器。
    explicit TwoSAT(int n) : vars(n), g(2 * n + 1), scc(g) {}

    // 参数 x 属于 1..vars、value 指定真假文字；返回对应 1-based 蕴含图节点号。
    int node(int x, bool value) const {
        return 2 * (x - 1) + (value ? 1 : 0) + 1;
    }
    // 参数 a/b 为文字节点号；返回 void，向蕴含图添加有向边 a->b。
    void imply(int a, int b) {
        g[a].push_back(b);
    }
    // 参数给出两个变量及各自文字真假值；返回 void，加入析取子句对应的两条蕴含边。
    void addOr(int x, bool xv, int y, bool yv) {
        // (x==xv) OR (y==yv) 等价于 ¬x->y 与 ¬y->x。
        int a = node(x, xv), b = node(y, yv);
        imply(node(x, !xv), b);
        imply(node(y, !yv), a);
    }
    // 无参数；返回可满足赋值（下标 1..vars）或 nullopt；运行 SCC 并写内部分量状态，仅可运行一次。
    optional<vector<char>> solve() {
        scc.run();
        vector<char> answer(vars + 1);
        for (int x = 1; x <= vars; ++x) {
            int f = node(x, false), t = node(x, true);
            if (scc.component[f] == scc.component[t]) return nullopt;
            // TarjanSCC 按弹栈顺序编号，较小分量先赋值为 true。
            answer[x] = scc.component[t] < scc.component[f];
```

```
        }
        return answer;
    }
};
```

坑：复用 6.8 节的 `TarjanSCC`；节点编号及分量编号方向必须一致；若更换 SCC 实现，需重新核对赋值比较方向。

6.15 最小费用最大流

适用范围： 适用于带容量与单位费用的流网络，按最小费用发送尽可能多的流。 **能做什么：** 求指定上限内最大流量及该流量的最小费用。 **输入 / 输出：** 输入：点数、容量/费用边、源汇和流量上限；输出：流量与费用，残量结构会被修改。

用途 / 用法： 在最大流量前提下最小化费用；逐次找残量网络最短增广路。此 SPFA 版最坏复杂度高，适合中小网络；要求 `s!=t`、费用与总代价可由 `long long` 表示，且增广过程中不存在可达负费用环。

```
struct MinCostMaxFlow {
    // 残量网络逐次找最短路，parent 回溯增广；反向边费用为负。
    struct E {
        int to, rev;
        ll cap, cost;
    };
    int n;
    vector<vector<E>> g;
    // 参数 nodeCount 为点数；初始化空残量网络。
    explicit MinCostMaxFlow(int nodeCount)
        : n(nodeCount), g(nodeCount + 1) {
    }
    // 参数为端点、非负容量和单位费用；返回 void，加入正向边与负费用反向残量边。
    void addEdge(int u, int v, ll cap, ll cost) {
        E a{v, (int)g[v].size() + (u == v), cap, cost};
        E b{u, (int)g[u].size(), 0, -cost};
        g[u].push_back(a);
        g[v].push_back(b);
    }
    // 参数 s/t 为不同源汇、limit 为非负流量上限；返回本次 {流量,费用}，并修改残量图；须无可达负费用环。
    pair<ll, ll> run(int s, int t, ll limit = LLONG_MAX / 4) {
        assert(s != t);
        ll flow = 0;
        ll cost = 0;
        const ll INF = numeric_limits<ll>::max() / 4;
        while (flow < limit) {
            vector<ll> d(n + 1, INF);
            vector<int> pv(n + 1), pe(n + 1);
            vector<char> in(n + 1);
            queue<int> q;
            d[s]=0;
            q.push(s);
            in[s]=1;
            while (!q.empty()) {
                int u = q.front();
                q.pop();
                in[u]=0;
                for (int i = 0; i < (int)g[u].size(); ++i) {
                    auto& e = g[u][i];
                    if (e.cap && d[e.to] > d[u] + e.cost) {
```

```
                        d[e.to] = d[u] + e.cost;
                        pv[e.to] = u;
                        pe[e.to] = i;
                        if (!in[e.to]) {
                            in[e.to] = 1;
                            q.push(e.to);
                        }
                    }
                }
            }
            if (d[t] == INF) break;
            ll add = limit - flow;
            for (int v = t; v != s; v = pv[v]) add = min(add, g[pv[v]][pe[v]].cap);
            flow += add;
            cost += add * d[t];
            for (int v = t; v != s; v = pv[v]) {
                auto& e = g[pv[v]][pe[v]];
                e.cap -= add;
                g[v][e.rev].cap += add;
            }
        }
        return {flow,cost};
    }
};
```

要求点编号 `1..n`、容量非负、`s!=t`；可达距离和 `add * d[t]` 的费用累计必须处于 `long long` 安全范围，有限距离需小于 `INF`。SPFA 逐次最短增广适用于残量网络无可达负费用环的情况；存在此类环时，最短增广路过程可能无法收敛。`run(s,t,limit)` 最多发送 `limit` 单位并返回实际 `{flow,cost}`；重复调用会在已有残量上继续运行。

坑：反向边费用必须取负；若只需指定流量，应达到后提前停止并检查是否足量；大网络用势能+Dijkstra 版本。

7. 数学

本章默认整数为 `long long`，模数记为 `mod`。若无特别说明，模运算结果规范到 $[0, mod)$；乘法可能超过 `long long` 时用 `__int128`。

7.1 GCD 与扩展 GCD

适用范围： 整数最大公约数、Bézout 系数与同余预处理；输入及递归系数乘积须可由 `long long` 表示，不能含 `LLONG_MIN`。 **能做什么：** 欧几里得算法与扩展欧几里得算法在 $O(\log \min(|a|, |b|))$ 时间内求 gcd 和一组系数；`norm` 规范余数。 **输入 / 输出：** `exgcd(a,b,x,y)` 返回非负 gcd 并写 `x,y` 使 `ax+by=gcd`；(0,0) 返回 0、系数为 (1,0)。`norm(x,mod)` 要求 `mod>0`，返回 `[0,mod)`。

用法：

- `std::gcd(a,b)` 可直接求非负最大公约数。
- `exgcd(a,b,x,y)` 返回 `gcd(a,b)`，并写入一组 Bézout 系数。
- 方程 $ax + by = c$ 有整数解当且仅当 `c % gcd(a,b) == 0`。
- 模数、输入可能为负时，先用 `norm` 规范余数。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 输入可为负数；返回非负 gcd，并令 a*x+b*y=gcd(a,b)。系数本身不取模。
// 参数 a,b 及递归系数中间值须可由 long long 表示，不能含 LLONG_MIN；返回 gcd>=0，并覆盖 x,y 为 Bézout 系数。
ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        // 选 x 的符号是为了即使 a<0，右侧仍为非负的 |a|。
        x = (a >= 0 ? 1 : -1);
        y = 0;
        return llabs(a);
    }
    ll x1, y1;
    ll g = exgcd(b, a % b, x1, y1);
    // 由 b*x1+(a%b)*y1=g 反代；若输入接近 long long 上界，乘法可能溢出。
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

// 要求 mod>0；C++ 的负数余数可能为负，所以统一规范到 [0,mod)。
// 参数 mod>0；返回 x 的规范模余数 [0,mod)，不修改实参。
ll norm(ll x, ll mod) {
    x %= mod;
    if (x < 0) x += mod;
    return x;
}
```

易错点：

1. `gcd(0,0)` 通常按 0 处理，但对应的 Bézout 系数不唯一。
2. C++ 负数取模仍可能为负；作为答案或下标前必须规范化。
3. 系数可能很大，中间乘法可能溢出 `long long`。

参考：

- OI Wiki：最大公约数

7.2 快速幂

适用范围： 非负整数指数的模幂；负指数须先求逆元，模数必须为正。 **能做什么：** `qpow` 用二进制平方计算 $a^e \bmod mod$，$O(\log e)$ 时间、$O(1)$ 额外空间；`mul_mod` 以 `__int128` 承接乘积。 **输入 / 输出：** `mul_mod(a,b,mod)` 要求 `mod>0`，返回 C++ 有符号余数；`qpow(a,e,mod)` 要求 `e>=0,mod>0`，返回 `[0,mod)`，包括 `mod==1` 时为 0，`0^0` 按空乘积约定为 `1%mod`。

用法：

- 调用 `qpow(a, e, mod)`，要求 `e >= 0`、`mod > 0`。
- `mod == 1` 时答案为 0。
- 负底数会先规范到 $[0, mod)$。
- 负指数不能直接使用；模意义下应先求逆元。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 要求 mod>0，且 a、b 已在 long long 内；用 __int128 避免乘法先溢出再取模。
// 参数 mod>0；返回 a*b 的 C++ 余数，乘积在 __int128 中求值，不改实参。
ll mul_mod(ll a, ll b, ll mod) {
    // 提升后再乘，避免两个 long long 先溢出；要求 mod>0。
    return (ll)((__int128)a * b % mod);
}

// 参数 e>=0、mod>0；返回规范化的 a^e mod mod，内部不持有可变共享状态。
ll qpow(ll a, long long e, ll mod) {
    // e 必须非负；若题目出现负指数，应先求 a 的模逆元再调用。
    a %= mod;
    if (a < 0) a += mod;
    ll ans = 1 % mod;
    while (e > 0) {
        // 不变量：ans * a^e 与原始 a^原始e 在模 mod 下同余。
        if (e & 1) ans = mul_mod(ans, a, mod);
        a = mul_mod(a, a, mod);
        e >>= 1;
    }
    return ans;
}
```

易错点：

1. 指数必须是非负整数；不要把负指数传入循环。
2. `a * a` 即使最终取模也可能先溢出。
3. 不要把按位与写成 `e & 1 == 1`；加括号更清楚。

参考：

- OI Wiki：快速幂

7.3 素数筛

适用范围： 已知上界 n>=0，需要批量判素、查询 [1,n] 内各整数的正因数个数或分解范围内整数时使用；单个数值很大的分解更适合 7.14 的试除模板。 **能做什么：** 线性筛在 $O(n)$ 时间和空间内生成质数、最小质因子、该最小质因子的指数及正因数个数；判素和范围内因数个数查询均为 $O(1)$。因数数量维护新增两张长度 n+1 的 int 表。 **输入 / 输出：** PrimeSieve(n) 建立 [0,n] 表；isPrime(x) 对任意整数返回布尔值，超出范围或 x<2 返回 false；countDivisors(x) 要求 1<=x<=n，返回 x 的正因数个数。divisorCount[1]=1；divisorCount[0]=0 仅为占位，不表示数学上的因数个数。

用法：

- 构造 PrimeSieve sieve(n) 后通过 sieve.primes 读取素数表。
- primes 递增，minPrime[x] 是 x 的最小质因子，minPrimeExponent[x] 是该因子在 x 中的指数。
- divisorCount[x] 保存 x 的正因数个数；对素数为 2，对 1 为 1。筛表建好后查询 O(1)。
- 例如 sieve.countDivisors(12)==6，sieve.countDivisors(1)==1。
- isPrime(x) 可判断筛选范围内的整数。
- 多次分解不超过 n 的数，可反复除以 minPrime[x]。
- 对同一上界内的大量整数做计数适合本表；只分解一个大数时，筛到该数会浪费时间和空间。

```
#include <bits/stdc++.h>
using namespace std;

struct PrimeSieve {
    // 数组下标就是被筛整数；0、1 的 minPrime 和 minPrimeExponent 为 0，divisorCount[0] 仅作占位。
    vector<int> primes, minPrime, minPrimeExponent, divisorCount;

    // 若 i=p^e*m 且 gcd(p,m)=1，则 d(i)=(e+1)d(m)。已有最小质因子时 d(ip)=d(i)/(e+1)*(e+2)，新增质因子时 d(ip)=2d(i)；前者先做精确整除。
    // 参数 n>=0；构造 [0,n] 筛表，填充素数、最小质因子、其指数和正因数个数；n=0/1 也有效。
    explicit PrimeSieve(int n)
        : minPrime(n + 1, 0), minPrimeExponent(n + 1, 0), divisorCount(n + 1, 0) {
        if (n >= 1) divisorCount[1] = 1;
        for (int i = 2; i <= n; ++i) {
            if (minPrime[i] == 0) {
                minPrime[i] = i;
                minPrimeExponent[i] = 1;
                divisorCount[i] = 2;
                primes.push_back(i);
            }
            for (int p : primes) {
                // 只用 p<=minPrime[i]：保证 i*p 恰由其最小质因子筛到一次。
                if (p > minPrime[i] || 1LL * i * p > n) break;
                int product = i * p;
                minPrime[product] = p;
                if (p == minPrime[i]) {
                    // 同一最小质因子指数由 e 增到 e+1，其余质因子的计数因子不变。
                    minPrimeExponent[product] = minPrimeExponent[i] + 1;
                    divisorCount[product] = divisorCount[i] /
                        (minPrimeExponent[i] + 1) * (minPrimeExponent[product] + 1);
                    break;
                }
                // p<minPrime[i] 时新增一个一次因子 p，tau 值乘 2。
                minPrimeExponent[product] = 1;
                divisorCount[product] = divisorCount[i] * 2;
            }
        }
```

```
    }

    // 参数 x 为任意整数；返回 x 是否在筛范围内且为素数，越界返回 false，不改对象。
    bool isPrime(int x) const {
        return x >= 2 && x < (int)minPrime.size() && minPrime[x] == x;
    }

    // 参数 x 满足 1<=x<minPrime.size()；返回 x 的正因数个数，读取预处理表且不修改对象。
    int countDivisors(int x) const {
        return divisorCount[x];
    }
};
```

易错点：

1. 0 和 1 都不是素数。
2. 判断 `i * p <= n` 时先转成 `long long`。
3. 线性筛内层停止条件不能随意改，否则会失去正确性或线性复杂度。

参考：

- OI Wiki：筛法

7.4 质因数分解

适用范围： 分解单个正整数 `n>=1`，数值不大到需要 Pollard–Rho 时使用。 **能做什么：** 试除到平方根，时间 $O(\sqrt{n})$、额外空间随不同质因子数增长。 **输入 / 输出：** `factorize(n)` 返回按质因子升序排列的 `{p,e}` 向量；`n==1` 返回空向量，不改变输入。

用法：

- 调用 `factorize(n)`，要求 `n >= 1`。
- 返回 `{质因子，指数}`，质因子严格递增。
- 若需约数个数，计算各 `(指数 + 1)` 的乘积。
- 大量且数值有共同上界时，优先预处理最小质因子筛。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 参数 n>=1；返回升序 {质因子,指数}，n==1 返回空表，不改调用者数据。
vector<pair<ll, int>> factorize(ll n) {
    // 要求 n>=1；返回质因子递增的 {质因子，指数}，不记录因子 1。
    vector<pair<ll, int>> fac;
    // 用 p<=n/p 代替 p*p<=n，避免 long long 乘法溢出；n 会随试除缩小。
    for (ll p = 2; p <= n / p; ++p) {
        if (n % p != 0) continue;
        int e = 0;
        while (n % p == 0) {
            n /= p;
            ++e;
        }
        fac.push_back({p, e});
    }
```

```
    // 此时若仍有余数，它不可能再含两个质因子，因此本身就是最后一个质因子。
    if (n > 1) fac.push_back({n, 1});
    return fac;
}
```

易错点：

1. 循环条件写 `p <= n / p`，避免 `p * p` 溢出。
2. 试除过程中 `n` 会缩小，循环上界也应随之缩小。
3. 循环结束后若 `n > 1`，不能漏掉这个大质因子。

参考：

- OI Wiki：质因数分解

7.5 欧拉函数

适用范围： 求正整数 `n>=1` 的欧拉函数，或求固定上界内的整张函数表。 **能做什么：** `eulerPhi` 单次试除 $O(\sqrt{n})$；`phiSieve` 线性筛 $O(n)$ 时间、$O(n)$ 空间。 **输入 / 输出：** `eulerPhi(n)` 返回 `phi(n)` 且 `phi(1)=1`；`phiSieve(n)` 要求 `n>=0`，返回下标 `[0,n]` 的表，`n==0` 时只含 `phi[0]=0`。

用法：

- `eulerPhi(n)` 要求 `n >= 1`，并约定 `phi(1) == 1`。
- 每个不同质因子只更新一次 `ans = ans / p * (p - 1)`。
- 若 `gcd(a,n)==1`，则 $a^{\varphi(n)} \equiv 1 \pmod{n}$。
- 需要 $1 \ldots n$ 全部函数值时使用 `phiSieve(n)`。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 参数 n>=1；返回 phi(n)，不修改 n 或外部状态。
ll eulerPhi(ll n) {
    // 要求 n>=1；ans 始终等于原 n 乘上已处理质因子的 (p-1)/p。
    ll ans = n;
    for (ll p = 2; p <= n / p; ++p) {
        if (n % p != 0) continue;
        while (n % p == 0) n /= p;
        // p 一定整除当前 ans，先除后乘既保持精确，也降低中间溢出风险。
        ans = ans / p * (p - 1);
    }
    if (n > 1) ans = ans / n * (n - 1);
    return ans;
}

// 参数 n>=0；返回下标 [0,n] 的 phi 表，分配局部筛状态，不修改外部对象。
vector<int> phiSieve(int n) {
    // phi[x] 对应整数 x；线性筛中每个合数只由最小质因子产生一次。
    vector<int> phi(n + 1), primes;
    vector<bool> composite(n + 1, false);
    if (n >= 1) phi[1] = 1;
    for (int i = 2; i <= n; ++i) {
        if (!composite[i]) {
            primes.push_back(i);
            phi[i] = i - 1;
        }
```

```
        for (int p : primes) {
            if (1LL * i * p > n) break;
            composite[i * p] = true;
            if (i % p == 0) {
                // p 已整除 i：新增一个 p 因子，phi(i*p)=phi(i)*p，随后必须停止。
                phi[i * p] = phi[i] * p;
                break;
            }
            // gcd(i,p)=1，乘入新质因子 p 时欧拉函数乘以 p-1。
            phi[i * p] = phi[i] * (p - 1);
        }
    }
    return phi;
}
```

易错点：

1. 欧拉定理要求底数与模数互质。
2. 公式乘除应先做整除再乘，降低溢出风险。
3. `phi(1)` 为 1，不是 0。

参考：

- OI Wiki：欧拉函数

7.6 模逆元

适用范围： 模意义除法且 `mod>1`；逆元存在的充要条件是 `gcd(a,mod)==1`。 **能做什么：** 扩展 GCD 在 $O(\log mod)$ 时间求逆元，支持合数模。 **输入 / 输出：** `modInverse(a,mod)` 返回 `{true,x}` 且 `x∈[0,mod)`；若不互质返回 `{false,0}`。函数先把负 `a` 规范化。

用法：

- `modInverse(a, mod)` 返回 `{是否存在, 逆元}`。
- 模数要求 `mod > 1`，结果位于 $[0, mod)$。
- 计算 `b / a (mod mod)` 前先确认逆元存在。
- 组合数的阶乘逆元法通常要求模数为质数且阶乘不为 0。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 参数 a,b 为非负且递归系数运算须可表示；返回 gcd 并写 Bézout 系数 x,y。
ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
    ll g = exgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - a / b * y1;
    return g;
}

// 参数 mod>1、a 可带符号；返回 {是否存在,规范逆元}，不互质时为 {false,0}。
```

```
pair<bool, ll> modInverse(ll a, ll mod) {
    // 要求 mod>1；返回 false 表示 gcd(a,mod)!=1，此时任何模除法都无定义。
    a %= mod;
    if (a < 0) a += mod;
    ll x, y;
    ll g = exgcd(a, mod, x, y);
    if (g != 1) return {false, 0};
    // Bézout 等式给出 a*x+mod*y=1，所以 x 就是逆元；规范到 [0,mod)。
    x %= mod;
    if (x < 0) x += mod;
    return {true, x};
}
```

易错点：

1. 不互质时逆元不存在，不能继续做模除法。
2. 费马小定理求逆只适用于质数模数。
3. `a == 0 (mod mod)` 没有逆元。

参考：

- OI Wiki：模逆元

7.7 线性同余方程

适用范围： 求 `a*x ≡ b (mod m)`，要求正模数 `m>0`；系数和常数允许带符号。 **能做什么：** 扩展 GCD 判定可解性并给出一个剩余类，时间 $O(\log m)$、递归空间 $O(\log m)$。 **输入 / 输出：** `solveCongruence(a,b,m)` 返回 `{exists,x0,step}`；若有解，所有整数解为 `x=x0+k*step`，其中 `x0∈[0,step)`、`step=m/gcd(a,m)`。无解时其余字段为 0。

用法：

- `solveCongruence(a,b,m)` 返回 `{是否有解, x0, step}`。
- 所有整数解满足 `x = x0 + k * step`，其中 `step = m / gcd(a,m)`。
- 返回的 `x0` 位于 `[0, step)`。
- 要列出原模数内全部解，取 `x0 + k*step`，`0 <= k < gcd(a,m)`。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 参数 a,b 为非负；返回 gcd 并覆盖 x,y 为 Bézout 系数。
ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1;
    ll g = exgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - a / b * y1;
    return g;
}

struct CongruenceSolution {
    bool exists;
    // exists=true 时全部解为 x=x0+k*step，且 x0 位于 [0,step)。
    ll x0, step;
};
```

```
// 参数 m>0、a,b 可带符号；返回解存在标志及 x0、step，不解时三字段为 false,0,0。
CongruenceSolution solveCongruence(ll a, ll b, ll m) {
    // 要求 m>0；负系数和负常数先规范化，方程语义不变。
    a %= m;
    b %= m;
    if (a < 0) a += m;
    if (b < 0) b += m;
    ll x, y;
    ll g = exgcd(a, m, x, y);
    // ax≡b (mod m) 可解当且仅当 gcd(a,m) 整除 b。
    if (b % g != 0) return {false, 0, 0};
    ll step = m / g;
    // 约去 g 后 x 是 a/g 在模 m/g 下的逆元；乘法用 __int128 防溢出。
    ll x0 = (ll)((__int128)x * (b / g) % step);
    if (x0 < 0) x0 += step;
    return {true, x0, step};
}
```

易错点：

1. 先检查 `b % gcd(a,m)`，否则逆元并不存在。
2. 约去最大公约数后，答案模数是 `m/g` 而不是 `m`。
3. 中间的 `x * (b/g)` 可能溢出，应使用 `__int128`。

参考：

- OI Wiki：线性同余方程

7.8 中国剩余定理（非互质模数）

适用范围： 合并多个模同余式，模数允许不互质但每个 `mod[i]>0`；要求逐步 lcm 可放进 `long long`。 **能做什么：** 逐式合并并用 gcd 判相容，时间约 $O(k\log M)$、额外空间 $O(1)$。 **输入 / 输出：** `crt(rem,mod)` 要求两向量等长；返回 `{true,r,m}` 表示全部解 `x=r+k*m` 且 `r∈[0,m)`，矛盾返回 `{false,0,0}`。空输入返回 `0 (mod 1)`。

用法：

- `crt(rem, mod)` 中每个 `mod[i]` 必须为正。
- 返回 `{true, r, m}` 表示全部解为 `x = r + k*m`。
- `r` 会规范到 `[0,m)`；`false` 表示方程组矛盾。
- 若所有模数两两互质，最终模数为其乘积。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 参数 a,b 为正；返回 gcd 并覆盖 x,y 为 Bézout 系数，要求递归乘积不溢出。
ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1;
    ll g = exgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - a / b * y1;
    return g;
}

struct CRTResult {
```

```
    bool exists;
    // exists=true 时全部解为 x=remainder+k*modulus，余数已规范化。
    ll remainder, modulus;
};

// 参数 rem/mod 等长、每个模数正且逐步 lcm<=LLONG_MAX；返回规范解类或矛盾标志，不改输入。
CRTResult crt(const vector<ll>& rem, const vector<ll>& mod) {
    // 要求 rem.size()==mod.size() 且每个 mod[i]>0；空方程组返回 x≡0 (mod 1)。
    ll r = 0, m = 1;
    for (int i = 0; i < (int)rem.size(); ++i) {
        ll b = rem[i] % mod[i];
        if (b < 0) b += mod[i];
        ll x, y;
        ll g = exgcd(m, mod[i], x, y);
        ll diff = b - r;
        // 令新解为 r+m*t；只有 gcd(m,mod[i]) 能整除 b-r 时才可合并。
        if (diff % g != 0) return {false, 0, 0};

        ll q = mod[i] / g;
        // x 是 m/g 对模 q 的逆元；t 取最小非负剩余类即可。
        ll t = (ll)((__int128)(diff / g) * x % q);
        if (t < 0) t += q;
        __int128 newMod = (__int128)m * q;
        __int128 newR = (__int128)r + (__int128)m * t;
        // 若题目不保证最终 lcm 可放入 long long，需改用大整数。
        m = (ll)newMod;
        r = (ll)(newR % m);
        if (r < 0) r += m;
    }
    return {true, r, m};
}
```

易错点：

1. 经典 CRT 的乘积公式不能直接用于不互质模数。
2. `lcm`、余数更新都可能超过 `long long`；模板假设最终答案可表示。
3. 空方程组按 `x ≡ 0 (mod 1)` 处理。

参考：

- OI Wiki：中国剩余定理

7.9 组合数

适用范围： 质数模数下重复查询组合数，预处理界 `N` 满足 `0<=N<mod`、`mod` 为质数且大于 1。 **能做什么：** `Combinations` 预处理阶乘和逆阶乘，构造 $O(N+\log mod)$、单次查询 $O(1)$，空间 $O(N)$。 **输入 / 输出：** `Combinations(N,mod)` 拥有 `[0,N]` 表；`C(n,k)` 返回模 `mod` 的组合数，`n>N`、`k<0` 或 `k>n` 按约定返回 0。

用法：

- 构造 `Combinations(N, mod)`，模板要求 `mod` 为质数且 `N < mod`。
- `C(n,k)` 在越界或 `k<0`、`k>n` 时返回 0。
- 多次查询同一模数时只预处理一次。
- 小范围任意模数可用递推 $C(n,k)=C(n-1,k-1)+C(n-1,k)$。

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 参数 0<=a<mod、e>=0、mod>1；返回规范的 a^e 模余数，供阶乘逆元计算。
ll qpow(ll a, ll e, ll mod) {
    ll ans = 1;
    while (e > 0) {
        if (e & 1) ans = (ll)((__int128)ans * a % mod);
        a = (ll)((__int128)a * a % mod);
        e >>= 1;
    }
    return ans;
}

struct Combinations {
    ll mod;
    // fact[i]=i!，invFact[i]=(i!)^{-1}；数组下标覆盖 [0,n]。
    vector<ll> fact, invFact;

    // 参数 0<=n<mod、mod 为质数；构造并拥有 0..n 阶乘和逆阶乘表。
    Combinations(int n, ll mod) : mod(mod), fact(n + 1), invFact(n + 1) {
        // 前提：mod 为质数且 0<=n<mod，否则 fact[n] 可能为 0 且没有逆元。
        fact[0] = 1;
        for (int i = 1; i <= n; ++i)
            fact[i] = (ll)((__int128)fact[i - 1] * i % mod);
        // 费马小定理只在质数模下成立；改成任意模数时需替换整套求逆方案。
        invFact[n] = qpow(fact[n], mod - 2, mod);
        for (int i = n; i >= 1; --i)
            invFact[i - 1] = (ll)((__int128)invFact[i] * i % mod);
    }

    // 参数 n,k 为整数；返回 C(n,k) mod mod，越出预处理范围或 k 不合法时返回 0，不改表。
    ll C(int n, int k) const {
        // 返回 C(n,k) mod mod；非法 k 或超出预处理上界时约定返回 0。
        if (k < 0 || k > n || n >= (int)fact.size()) return 0;
        ll ans = (ll)((__int128)fact[n] * invFact[k] % mod);
        return (ll)((__int128)ans * invFact[n - k] % mod);
    }
};
```

易错点：

1. 逆阶乘写法依赖质数模数和 `N < mod`。
2. `k > n` 时组合数是 0，不要访问负下标。
3. 不要为每次查询重复求幂；先统一预处理。

参考：

- OI Wiki：组合数学

7.10 容斥原理

适用范围： 统计 `[1,n]` 中被给定正除数至少一个整除的数；`n>=0`，条件数必须小于 64 且足够小以承受指数枚举。 **能做什么：** 容斥枚举除数子集并用 lcm 算交集，时间 $O(2^k k)$、额外空间 $O(k)$；超过 n 的 lcm 贡献为 0。 **输入 / 输出：** `countDivisible(n,a)` 忽略 0、去重并删除冗余倍数，返回并集计数；负除数不在输入约定内。

用法：

- 示例 `countDivisible(n,a)` 统计 `1..n` 中能被至少一个 `a[i]` 整除的数。
- 先删除 0、重复项，以及被更小元素整除的冗余项。
- 子集交集对应被这些数的 lcm 整除。
- 条件数较大时，2^k 不可接受，应寻找其他计数方法。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 参数 n>=0、a 含非负除数且有效项数<64；返回 [1,n] 并集计数，传值副本可排序删项。
ll countDivisible(ll n, vector<ll> a) {
    // 统计闭区间 [1,n]；模板假设 n>=0、除数为非负数，且有效条件数 k<64。
    sort(a.begin(), a.end());
    a.erase(remove(a.begin(), a.end(), 0), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    vector<ll> b;
    for (ll x : a) {
        bool redundant = false;
        // 若 x 是已有 y 的倍数，“能被 x 整除”已包含于“能被 y 整除”，可删除。
        for (ll y : b) if (x % y == 0) {
            redundant = true;
            break;
        }
        if (!redundant && x <= n) b.push_back(x);
    }

    ll ans = 0;
    int k = (int)b.size();
    for (unsigned long long mask = 1; mask < (1ULL << k); ++mask) {
        // 一个子集的交集就是被这些除数的 lcm 整除；奇数个集合加，偶数个减。
        ll lcm = 1;
        bool tooLarge = false;
        for (int i = 0; i < k; ++i) if (mask >> i & 1ULL) {
            ll g = std::gcd(lcm, b[i]);
            // 一旦 lcm>n，本子集贡献为 0；先除后比较同时避免 lcm 乘法溢出。
            if (lcm > n / (b[i] / g)) { tooLarge = true; break; }
            lcm *= b[i] / g;
        }
        if (tooLarge) continue;
        if (__builtin_popcountll(mask) & 1) ans += n / lcm;
        else ans -= n / lcm;
    }
    return ans;
}
```

易错点：

1. 正负号由所选集合数决定：奇加偶减。
2. lcm 计算前先判断是否超过 `n`，同时避免溢出。
3. `1ULL << k` 要求 `k < 64`，实际指数复杂度通常更早成为瓶颈。

参考：

- OI Wiki：容斥原理

7.11 矩阵快速幂

适用范围： 维数较小、指数较大的线性递推或状态转移；指数非负，模数正。 **能做什么：** `matPow` 用二进制快速幂，$O(d^3 \log e)$ 时间、$O(d^2)$ 空间。 **输入 / 输出：** `Matrix(r,c,mod)` 建立零矩阵；乘法要求维度匹配、模数一致，`matPow(base,e)` 要求方阵并返回 `base^e`，`e==0` 返回单位矩阵。

用法：

- `Matrix::identity(n,mod)` 是乘法单位矩阵。
- `matPow(A,e)` 要求 `A` 为方阵、`e >= 0`。
- 列向量约定下，若 `state(t+1)=A*state(t)`，则答案为 `A^t*state(0)`。
- 模板执行模运算；不取模时需改类型并评估溢出。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct Matrix {
    int rows, cols;
    // 本模板所有元素都在同一正模数 mod 下运算；不同矩阵相乘时模数须一致。
    ll mod;
    vector<vector<ll>> a;

    // 参数 r,c>=0、mod>0；构造并拥有 r*c 个零元素。
    Matrix(int r, int c, ll mod) : rows(r), cols(c), mod(mod), a(r, vector<ll>(c)) {}

    // 参数 n>=0、mod>0；返回 n 阶模乘单位阵，不修改外部状态。
    static Matrix identity(int n, ll mod) {
        Matrix id(n, n, mod);
        for (int i = 0; i < n; ++i) id.a[i][i] = 1 % mod;
        return id;
    }
};

// 参数矩阵满足 x.cols==y.rows 且模数相同为正；返回新乘积矩阵，不改 x,y。
Matrix operator*(const Matrix& x, const Matrix& y) {
    // 前提 x.cols==y.rows；i-k-j 顺序可跳过零项，乘积用 __int128 防溢出。
    Matrix z(x.rows, y.cols, x.mod);
    for (int i = 0; i < x.rows; ++i)
        for (int k = 0; k < x.cols; ++k) if (x.a[i][k])
            for (int j = 0; j < y.cols; ++j)
                z.a[i][j] = (z.a[i][j] +
                    (__int128)x.a[i][k] * y.a[k][j]) % x.mod;
    return z;
}

// 参数 base 为正模方阵、e>=0；按值接收底矩阵并返回 base^e，实参不变。
Matrix matPow(Matrix base, long long e) {
    // 要求 base 为方阵、e>=0；列向量转移通常写 state(t)=base^t*state(0)。
    Matrix ans = Matrix::identity(base.rows, base.mod);
    while (e > 0) {
        // 不变量：ans * base^e 等于原矩阵的原指数次幂。
        if (e & 1) ans = ans * base;
        base = base * base;
        e >>= 1;
    }
    return ans;
}
```

易错点：

1. 明确状态是列向量还是行向量，矩阵方向不能写反。
2. `e == 0` 应返回单位矩阵。
3. 矩阵维度、模数必须匹配；模板未做运行时检查。

参考：

- OI Wiki：矩阵

7.12 高斯消元

适用范围： 求有限实数系数的线性方程组，至少一行且每行列数相同；尺度适合绝对 EPS 判定。 **能做什么：** 部分主元高斯消元判别无解/唯一/多解，$m \times n$ 时间 $O(mn \min(m,n))$、空间 $O(mn)$。 **输入 / 输出：** `gauss(a,answer)` 输入 `m×(n+1)` 增广矩阵并覆盖 `answer`；返回 0 无解（此时答案不可用）、1 唯一解、2 无穷多解（给自由变量取 0 的一组解）。

用法：

- 增广矩阵每行含 `n+1` 项，最后一列为常数。
- 返回 `0` 表示无解，`1` 表示唯一解，`2` 表示无穷多解。
- `answer` 在有解时给出一组解；自由变量默认取 0。
- 模意义方程组应改用模逆元，不可照搬浮点模板。

```
#include <bits/stdc++.h>
using namespace std;

// 0: 无解; 1: 唯一解; 2: 无穷多解。
// 参数 a 为有限数值且形状 m>0、每行 n+1; 覆盖 answer，返回 0无解/1唯一/2多解。无解时 answer 不可用。
int gauss(vector<vector<long double>> a, vector<long double>& answer) {
    // a 是 m 行、n+1 列的增广矩阵; 模板要求 m>0 且各行长度一致。
    // EPS 应按输入尺度调整: 数值很大或病态矩阵可能需要更稳健的方法。
    const long double EPS = 1e-12L;
    int m = (int)a.size();
    int n = (int)a[0].size() - 1;
    vector<int> where(n, -1);
    // where[col] 是该未知数主元所在行; -1 表示自由变量，最终默认取 0。
    int row = 0;

    for (int col = 0; col < n && row < m; ++col) {
        int pivot = row;
        // 部分主元: 选绝对值最大项，减少除以小数造成的误差放大。
        for (int i = row; i < m; ++i)
            if (fabsl(a[i][col]) > fabsl(a[pivot][col])) pivot = i;
        // 整列近似为 0 时不能制造主元，这一列成为自由变量候选。
        if (fabsl(a[pivot][col]) < EPS) continue;
        swap(a[pivot], a[row]);
        where[col] = row;

        long double div = a[row][col];
        for (int j = col; j <= n; ++j) a[row][j] /= div;
        // 消去所有其他行，直接得到简化行阶梯形; 只消下方则需额外回代。
        for (int i = 0; i < m; ++i) if (i != row) {
            long double factor = a[i][col];
            if (fabsl(factor) < EPS) continue;
            for (int j = col; j <= n; ++j)
```

```
                a[i][j] -= factor * a[row][j];
        }
        ++row;
    }

    answer.assign(n, 0);
    for (int col = 0; col < n; ++col)
        if (where[col] != -1) answer[col] = a[where[col]][n];

    for (int i = 0; i < m; ++i) {
        // 用得到的一组解回代等价方程；不满足说明出现 0=非0，即无解。
        long double lhs = 0;
        for (int j = 0; j < n; ++j) lhs += a[i][j] * answer[j];
        if (fabsl(lhs - a[i][n]) > EPS) return 0;
    }
    for (int col = 0; col < n; ++col)
        if (where[col] == -1) return 2;
    return 1;
}
```

易错点：

1. 必须选绝对值较大的主元，直接用当前行数值误差更大。
2. 无主元的列对应自由变量，不代表无解。
3. 空矩阵需要在调用前单独处理，模板假设至少有一行。

参考：

- OI Wiki：高斯消元

7.13 公平组合游戏与 SG 函数

适用范围： 无随机、完全信息、正常玩法且状态转移图为 DAG 的公平组合游戏；多个子游戏须彼此独立。**能做什么：** `sg(u)` 记忆化求 mex，遍历复杂度 $O(V+E)$ 外加 mex 标记；空间 $O(V+E)$。 **输入 / 输出：** `SGGame(graph)` 接管 0-based 状态图；`sg(u)` 返回状态值并写入共享 `memo`；`firstPlayerWins(game,starts)` 会填充 memo，返回起始局面 SG 异或是否非零。节点下标须合法。

用法：

- 将每个局面视为节点，可执行操作视为有向边，且状态必须无环。
- `sg(u)` 返回该局面的 Sprague–Grundy 值。
- 多堆或多个独立局面把 SG 值异或；结果 0 为后手必胜。
- 仅判断普通取石游戏时，先确认确实满足正常玩法与独立性。

```
#include <bits/stdc++.h>
using namespace std;

struct SGGame {
    // nextState[u] 列出局面 u 一步可达的所有局面；状态图必须是 DAG。
    vector<vector<int>> nextState;
    // memo[u]=-1 表示尚未计算，否则保存 SG(u)，使每个状态只展开一次。
    vector<int> memo;

    // 参数 graph 是 0-based DAG 邻接表；移动其内容初始化游戏和 -1 memo。
    explicit SGGame(vector<vector<int>> graph)
        : nextState(std::move(graph)), memo(nextState.size(), -1) {}
```

```
    // 参数 u 是合法状态编号；返回 SG(u)，并递归填充本对象 memo，要求图无环。
    int sg(int u) {
        if (memo[u] != -1) return memo[u];
        vector<int> values;
        values.reserve(nextState[u].size());
        for (int v : nextState[u]) values.push_back(sg(v));

        // k 个后继的 mex 至多为 k，因此只需标记 [0,k]，更大的 SG 值可忽略。
        vector<bool> seen(values.size() + 1, false);
        for (int x : values)
            if (x < (int)seen.size()) seen[x] = true;
        int mex = 0;
        while (mex < (int)seen.size() && seen[mex]) ++mex;
        return memo[u] = mex;
    }
};

// 参数 starts 是合法且互相独立的起始状态；返回 SG 异或非零，并通过 game 填充共享 memo。
bool firstPlayerWins(SGGame& game, const vector<int>& starts) {
    // starts 表示彼此独立的子游戏；普通玩法下异或和非 0 当且仅当先手必胜。
    int total = 0;
    for (int s : starts) total ^= game.sg(s);
    return total != 0;
}
```

易错点：

1. SG 定理不直接适用于有环游戏、随机游戏或“不能行动者胜”的反常玩法。
2. 合并独立游戏使用异或，不是相加。
3. mex 是未出现的最小非负整数，后继为空时 SG 为 0。

参考：

- OI Wiki：公平组合游戏

7.14 质因数分解、约数个数与约数和

适用范围： 分解 n>=1 后计算约数数量、约数和或完整约数列表；各结果须能放进 long long。 **能做什么：** 试除分解 $O(\sqrt{n})$；枚举约数 $O(\sqrt{n}+d\log d)$（d 为约数个数），额外空间 $O(d)$。 **输入 / 输出：** factorize(n) 返回有序 {p,e}；divisorCountSum(n) 返回 {约数个数,约数和}；enumerateDivisors(n) 返回升序正约数，n==1 三者均给出自然单位结果。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 参数 n>=1；返回按质因子升序的 {p,e}，1 返回空向量，不改外部状态。
vector<pair<ll,int>> factorize(ll n) {
    // 试除到 p*p>n；剩余 n>1 必为大质因子。
    vector<pair<ll, int>> f;
    for (ll p = 2; p <= n / p; ++p) {
        if (n % p != 0) continue;
        int e=0;
        do {
            n/=p;
            ++e;
        }
```

```
        while (n % p == 0);
        f.push_back({p,e});
    }
    if(n>1)f.push_back({n,1});
    return f;
}
// 参数 n>=1 且约数个数、约数和及中间幂可放进 ll；返回 {个数,总和}。
pair<ll,ll> divisorCountSum(ll n) {
    // 假设结果可放入 ll
    ll count = 1, sum = 1;
    for (auto [p, e] : factorize(n)) {
        ll term = 1, power = 1;
        for (int i = 0; i < e; ++i) {
            power *= p;
            term += power;
        }
        count*=e+1;
        sum*=term;
    }
    return {count,sum};
}
// 参数 n>=1；返回全部正约数升序列表，1 返回 {1}，不修改输入。
vector<ll> enumerateDivisors(ll n) {
    vector<ll>d;
    for (ll i = 1; i <= n / i; ++i) {
        if (n % i != 0) continue;
        d.push_back(i);
        if(i!=n/i)d.push_back(n/i);
    }
    sort(d.begin(),d.end());
    return d;
}
```

坑：要求 n>=1；循环条件写 p<=n/p 防平方溢出；约数和与个数也可能溢出，需要时取模或用大整数。

8. 字符串

本章字符串统一使用 0-based 下标，区间统一写成左闭右开 `[l,r)`。长度为 `n` 的字符串合法字符下标为 `0..n-1`；各模板用 `int` 保存长度和下标，因此输入长度及下标表达式必须可由 `int` 表示。未特别说明时字符按字节处理，不直接支持 UTF-8 的“单个汉字”语义。

8.1 字符串哈希

适用范围： 字节串的子串快速判等，使用 0-based 半开区间；固定双哈希仍可能碰撞，不适用于要求零误判或可被刻意构造的输入。 **能做什么：** `StringHash` 预处理 $O(n)$、占 $O(n)$ 空间；每次子串哈希 $O(1)$。 **输入 / 输出：** `StringHash(s)` 保存输入串的前缀表；`get(l,r)` 要求 `0<=l<=r<=|s|`，返回 `s[l..r)` 双哈希，空串区间返回 (0,0)。

用法：

- 构造 `StringHash(s)` 后，`get(l,r)` 返回 `s[l..r)` 的双哈希。
- 两个子串只有长度相等且哈希相等时才判为相同。
- 同一对象内或使用相同底数、模数的对象间可以比较。
- 对抗性强或必须零误判的题，应改用 KMP、后缀结构或直接校验。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct StringHash {
    // 两个模数与 BASE 必须在所有待比较对象中完全一致；哈希相等仍有极小碰撞概率。
    static constexpr ll MOD1 = 1000000007;
    static constexpr ll MOD2 = 1000000009;
    static constexpr ll BASE = 911382323;
    vector<ll> h1, h2, p1, p2;

    // 参数 s 按字节读取；构造并拥有前缀哈希与幂数组，不修改 s。
    explicit StringHash(const string& s) {
        int n = (int)s.size();
        // h[i] 表示前缀 s[0..i) 的哈希，p[i]=BASE^i；两数组都按长度而非字符下标。
        h1.assign(n + 1, 0); h2.assign(n + 1, 0);
        p1.assign(n + 1, 1); p2.assign(n + 1, 1);
        // 每轮把一个字节扩展进两个前缀哈希，并同步预存下一次查询所需的幂。
        for (int i = 0; i < n; ++i) {
            int value = (unsigned char)s[i] + 1;
            h1[i + 1] = (h1[i] * BASE + value) % MOD1;
            h2[i + 1] = (h2[i] * BASE + value) % MOD2;
            p1[i + 1] = p1[i] * BASE % MOD1;
            p2[i + 1] = p2[i] * BASE % MOD2;
        }
    }

    // 返回 0-based 左闭右开子串 s[l..r) 的双哈希；要求 0<=l<=r<=s.size()。
    // 参数是 s 的 0-based 半开区间 [l,r)；返回双模哈希，调用者须保证边界合法；不改状态。
    pair<ll, ll> get(int l, int r) const {
        // h[r]-h[l]*BASE^(r-l) 消掉左侧前缀，使不同起点的等长子串可直接比较。
        ll x1 = (h1[r] - h1[l] * p1[r - l]) % MOD1;
        ll x2 = (h2[r] - h2[l] * p2[r - l]) % MOD2;
        if (x1 < 0) x1 += MOD1;
```

```
        if (x2 < 0) x2 += MOD2;
        return {x1, x2};
    }
};
```

易错点：

1. 比较哈希前必须比较子串长度。
2. 区间是 `[l,r)`，幂次应为 `r-l`。
3. 哈希相等不构成数学上的绝对相等，不能忽视碰撞要求。

参考：

- OI Wiki：字符串哈希

8.2 前缀函数与 KMP

适用范围： 在字节字符串中找模式串的所有出现，支持重叠；空模式按本模板约定返回空列表。 **能做什么：** `prefixFunction` 和 KMP 匹配总时间 $O(n+m)$，前缀表空间 $O(m)$。 **输入 / 输出：** `prefixFunction(s)` 返回每个字符位置的最长相等真前后缀长度；`kmpSearch(text,pattern)` 返回 0-based 起点升序列表，不改动输入。

用法：

- `prefixFunction(pattern)` 返回长度为 `m` 的 `pi`，且 `pi[0]=0`。
- `kmpSearch(text,pattern)` 返回模式串在文本中的所有 0-based 起点。
- 模板保留重叠匹配，例如 `"aaa"` 中 `"aa"` 的位置为 0、1。
- 空模式串的定义因题而异，模板直接返回空结果。

```
#include <bits/stdc++.h>
using namespace std;

// 参数 s 是字节串；返回与 s 等长的 pi 长度数组，空串返回空数组，不改输入。
vector<int> prefixFunction(const string& s) {
    int n = (int)s.size();
    // pi[i] 是 s[0..i] 的最长相等真前后缀“长度”，不是末尾下标。
    vector<int> pi(n, 0);
    for (int i = 1; i < n; ++i) {
        int j = pi[i - 1];
        // 失配后只可能尝试当前边界的次长边界；因此跳 pi[j-1]，文本位置 i 不回退。
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}

// 参数 text/pattern 是字节串；返回 pattern 的全部 0-based 起点（含重叠），空模式返回空表。
vector<int> kmpSearch(const string& text, const string& pattern) {
    // 返回 pattern 在 text 中所有 0-based 起点；空模式的语义由题目决定，这里返回空。
    vector<int> positions;
    if (pattern.empty()) return positions;
    vector<int> pi = prefixFunction(pattern);
    // 扫描 text[0..i] 前，j 表示已匹配的 pattern 前缀长度，始终有 0<=j<m。
    int j = 0;
    for (int i = 0; i < (int)text.size(); ++i) {
        while (j > 0 && text[i] != pattern[j]) j = pi[j - 1];
```

```
        if (text[i] == pattern[j]) ++j;
        if (j == (int)pattern.size()) {
            positions.push_back(i - j + 1);
            // 匹配后回到最长真边界，才能继续寻找允许重叠的下一次出现。
            j = pi[j - 1];
        }
    }
    return positions;
}
```

易错点：

1. `pi[i]` 存的是长度，不是末尾下标。
2. 失配回退写 `pi[j-1]`，不能写 `pi[j]`。
3. 找到一次匹配后仍要回退，才能找到重叠出现。

参考：

- OI Wiki：前缀函数与 KMP 算法

8.3 Z 函数（扩展 KMP）

适用范围： 求字符串各后缀与整串的 LCP，或匹配字节串；匹配时分隔符必须不存在于两输入串。 **能做什么：** `zFunction` 与 Z 匹配均为线性时间，空间 $O(n+m)$。 **输入 / 输出：** `zFunction(s)` 返回长度 `|s|` 的数组，`z[0]=|s|`（空串为空数组）；`zSearch(text,pattern)` 返回所有 0-based 起点，空模式返回空列表。

用法：

- 调用 `zFunction(s)` 获得每个后缀与整串的 LCP 长度。
- 匹配 `pattern` 与 `text` 时，可计算 `pattern + '#' + text` 的 Z 数组。
- 分隔符必须是不出现在两个原串中的单个字符。
- 拼接后位置 `m+1+i` 的 Z 值至少为 `m`，表示文本位置 `i` 匹配。

```
#include <bits/stdc++.h>
using namespace std;

// 参数 s 是字节串；返回各位置的 Z 长度，非空时 z[0]=|s|，空串返回空数组。
vector<int> zFunction(const string& s) {
    int n = (int)s.size();
    // z[i]=LCP(s, s[i..n))；所有字符与区间均为 0-based、左闭右开。
    vector<int> z(n, 0);
    if (n == 0) return z;
    z[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        // [l,r) 是目前右端点最远的前缀匹配段，且 s[l..r)==s[0..r-l)。
        // i 在盒内时先复用镜像 z[i-l]，但不能越过尚未验证的右端点 r。
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}

// 参数 text/pattern 为字节串且均不含 '\0' 分隔符；返回 0-based 匹配起点，空模式返回空表。
```

```
vector<int> zSearch(const string& text, const string& pattern) {
    // 返回所有 0-based 起点；若字符集可能含 '\0'，需把分隔符换成确定不存在的值。
    vector<int> positions;
    if (pattern.empty()) return positions;
    string joined = pattern + '\0' + text;
    vector<int> z = zFunction(joined);
    int m = (int)pattern.size();
    // joined 中 text[i] 的下标是 m+1+i；z 值至少为 m 才覆盖整个模式串。
    for (int i = 0; i < (int)text.size(); ++i)
        if (z[m + 1 + i] >= m) positions.push_back(i);
    return positions;
}
```

易错点：

1. 本模板维护的是左闭右开区间 `[l,r)`，判断条件是 `i < r`。
2. 不同资料对 `z[0]` 定义不同，使用前先统一。
3. 若输入可能含 `\0`，拼接时应换用确定不存在的分隔符或整数数组。

参考：

- OI Wiki：Z 函数（扩展 KMP）

8.4 Manacher

适用范围： 单个字节串的所有奇/偶长度回文中心半径；空串可输入。 **能做什么：** Manacher 在线性 $O(n)$ 时间、$O(n)$ 空间得到奇偶半径。 **输入 / 输出：** `manacher(s)` 返回两个与 `s` 等长数组；`odd[i]` 含中心，`even[i]` 对应缝隙 `i-1|i`，空串返回两空数组。

用法：

- 最长奇回文长度为 `2*odd[i]-1`，区间为 `[i-odd[i]+1, i+odd[i])`。
- 最长偶回文长度为 `2*even[i]`，区间为 `[i-even[i], i+even[i])`。
- 全局最长值取所有上述长度的最大值。
- 判断 `[l,r)` 是否回文，可按奇偶长度找到中心并检查对应半径。

```
#include <bits/stdc++.h>
using namespace std;

struct ManacherResult {
    // odd/even 均以原串 0-based 字符位置为中心索引，数值是单侧半径而非总长度。
    vector<int> odd, even;
};

// 参数 s 是字节串；返回等长 odd/even 半径数组，空串对应两个空数组，不改输入。
ManacherResult manacher(const string& s) {
    int n = (int)s.size();
    vector<int> odd(n), even(n);

    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        // [l,r] 是当前最右奇回文；盒内位置先复用关于中心的镜像半径。
        int k = (i > r ? 1 : min(odd[l + r - i], r - i + 1));
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) ++k;
        odd[i] = k;
        if (i + k - 1 > r) {
            l = i - k + 1;
            r = i + k - 1;
```

```
        }
    }

    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        // even[i] 的中心是缝隙 i-1|i；镜像下标因此比奇回文多一个 +1。
        int k = (i > r ? 0 : min(even[l + r - i + 1], r - i + 1));
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) ++k;
        even[i] = k;
        if (i + k - 1 > r) {
            l = i - k;
            r = i + k - 1;
        }
    }
    return {odd, even};
}
```

易错点：

1. 奇半径含中心，最小为 1；偶半径可为 0。
2. `even[i]` 的中心是 `i-1` 与 `i` 之间，不是字符 `i`。
3. 半径不是回文总长度，输出前需按奇偶换算。

参考：

- OI Wiki：Manacher

8.5 Trie 字典树

适用范围： 仅由小写 ASCII 字母构成的字符串多重集合，需前缀或精确计数。 **能做什么：** Trie 插入、完整词计数、前缀计数均为 $O(L)$；空间为已分配节点数乘 26。 **输入 / 输出：** `insert(s)` 修改 Trie 并累计重复次数；`countWord/countPrefix` 返回次数，`findNode` 返回节点号或 `-1`。空串在根节点计数；输入字符必须为 a-z。

用法：

- `insert(s)` 插入一个小写串，并保留重复次数。
- `countWord(s)` 返回该完整字符串插入次数。
- `countPrefix(p)` 返回以 `p` 为前缀的已插入字符串数。
- 其他字符集可改 `array<int,26>` 的大小与 `id` 映射。

```
#include <bits/stdc++.h>
using namespace std;

struct Trie {
    struct Node {
        // next[c]=0 表示无边；根恰为 0，所以实际新节点编号必须从 1 开始。
        array<int, 26> next{};
        // pass：经过该节点的已插入字符串数；end：恰在该节点结束的字符串数。
        int pass = 0;
        int end = 0;
    };
    vector<Node> node{1};

    // 参数 s 只含 a..z；把一个词加入本 Trie，重复插入会增加 pass/end 计数。
    void insert(const string& s) {
        // 字符映射固定为 'a'..'z'；改题换字符集时同时修改 26 与 ch-'a'。
        int u = 0;
```

```
        // 根的 pass 也计数，因此 countPrefix("") 会返回插入串总数。
        ++node[u].pass;
        for (char ch : s) {
            int c = ch - 'a';
            if (node[u].next[c] == 0) {
                node[u].next[c] = (int)node.size();
                node.push_back(Node{});
            }
            u = node[u].next[c];
            ++node[u].pass;
        }
        ++node[u].end;
    }

    // 参数 s 只含 a..z；返回前缀终点节点号，缺边返回 -1；不修改 Trie。
    int findNode(const string& s) const {
        // 返回前缀终点编号；-1 是查询失败状态，与合法根节点 0 区分。
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (node[u].next[c] == 0) return -1;
            u = node[u].next[c];
        }
        return u;
    }

    // 参数 s 只含 a..z；返回完整词插入次数，未出现为 0；不修改 Trie。
    int countWord(const string& s) const {
        int u = findNode(s);
        return u == -1 ? 0 : node[u].end;
    }

    // 参数 s 只含 a..z；返回以 s 为前缀的插入次数；空串返回总插入次数。
    int countPrefix(const string& s) const {
        int u = findNode(s);
        return u == -1 ? 0 : node[u].pass;
    }
};
```

易错点：

1. 根节点编号为 0，因此 0 同时作为“无转移”，新节点编号必须从 1 开始。
2. 模板未检查字符范围，非小写字母会越界。
3. “经过次数”和“单词结束次数”含义不同，不能混用。

参考：

- OI Wiki：字典树

8.6 AC 自动机

适用范围： 小写 ASCII 字符集下的多模式匹配，要求先插入全部非空模式再 build；状态图为 Trie。 **能做什么：** 固定数组转移构建 $O(S\Sigma)$、扫描与 fail 传播 $O(T+S)$，空间 $O(S\Sigma)$。 **输入 / 输出：** `insert(s)` 返回终止节点；`build()` 原地补全转移且只能在插入结束后调用一次；`countOccurrences(text,terminal)` 返回与 terminal 等长的计数，包含重叠，terminal 只能指向非空模式终点。

用法：

- 先逐个 insert(pattern)，保存返回的终止节点，再调用 build()。
- countOccurrences(text,terminal) 返回各模式出现次数，包含重叠出现。
- 相同模式的终止节点相同，因此得到相同计数。
- 模板限定小写英文字母；build() 后不要再插入模式。

```
#include <bits/stdc++.h>
using namespace std;

struct AhoCorasick {
    struct Node {
        // build 前 next 是 Trie 边；build 后缺边被补为自动机转移，不能再继续 insert。
        array<int, 26> next{};
        // fail 指向当前字符串的最长真后缀状态，根及根的孩子失败时回到根 0。
        int fail = 0;
    };
    vector<Node> node{1};
    vector<int> bfsOrder;

    // 参数 s 是非空小写模式串；加入 Trie 并返回终止节点号，须在 build 前调用。
    int insert(const string& s) {
        // 返回模式串终止节点，调用者须保存它；字符集固定为小写 'a'..'z'。
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (node[u].next[c] == 0) {
                node[u].next[c] = (int)node.size();
                node.push_back(Node{});
            }
            u = node[u].next[c];
        }
        return u;
    }

    // 无参数；补全本对象转移并填 bfsOrder，改变 Trie 边语义；插入结束后仅调用一次。
    void build() {
        // 只调用一次；BFS 保证计算子节点 fail 前，其 fail 父节点的完整转移已就绪。
        queue<int> q;
        for (int c = 0; c < 26; ++c) {
            int v = node[0].next[c];
            if (v != 0) q.push(v);
        }
        while (!q.empty()) {
            int u = q.front(); q.pop();
            bfsOrder.push_back(u);
            for (int c = 0; c < 26; ++c) {
                int v = node[u].next[c];
                if (v != 0) {
                    // u 经 c 到 v，则 v 的最长可用后缀是 fail[u] 经同一字符的转移。
                    node[v].fail = node[node[u].fail].next[c];
                    q.push(v);
                } else {
                    // 补全缺边后，文本扫描每个字符只做一次数组跳转，不需 while 回退。
                    node[u].next[c] = node[node[u].fail].next[c];
                }
            }
        }
    }

    // 参数 text 是小写文本，terminal 是有效的非根模式终点列表；返回逐项出现次数（允许重
```

```
叠），不改自动机。
    vector<long long> countOccurrences(
        const string& text, const vector<int>& terminal
    ) const {
        // visits[u] 先统计扫描时恰好落到 u 的次数；terminal 中存各模式的终止节点。
        vector<long long> visits(node.size(), 0), answer;
        int u = 0;
        for (char ch : text) {
            u = node[u].next[ch - 'a'];
            ++visits[u];
        }
        // fail 树上由深到浅累加：到达 u 也代表其每个 fail 后缀模式出现一次。
        for (int i = (int)bfsOrder.size() - 1; i >= 0; --i) {
            int v = bfsOrder[i];
            visits[node[v].fail] += visits[v];
        }
        for (int v : terminal) answer.push_back(visits[v]);
        return answer;
    }
};
```

易错点：

1. 必须先插入全部模式串，再构建失配指针。
2. 统计每个模式次数时，要按 BFS 逆序把访问量累加到 fail 父节点。
3. 空模式串会终止在根节点，通常应在输入层单独处理。
4. 模板将缺失转移补全，构建后原 Trie 结构已改变。

参考：

- OI Wiki：AC 自动机

8.7 后缀数组

适用范围： 需要按字典序排序所有字节后缀并查询相邻后缀 LCP；适合中等长度字符串。 **能做什么：** `suffixArray` 倍增比较排序 $O(n\log^2 n)$、空间 $O(n)$；Kasai 的 LCP 构造 $O(n)$。 **输入 / 输出：** `suffixArray(s)` 返回按后缀字典序排列的 0-based 起点数组，空串返回空数组；`lcpArray(s,sa)` 要求 `sa` 是按后缀字典序排列的 `[0,n)` 起点排列，返回长度 `max(0,n-1)`，第 i 项为 `sa[i]` 与 `sa[i+1]` 的 LCP。

```
#include <bits/stdc++.h>
using namespace std;

// 参数 s 是字节串；返回按字典序排列的后缀起点，空串返回空表，不改输入。
vector<int> suffixArray(const string& s) {
    // 倍增按两个 rank 排序，越界第二关键字视为 -1。
    int n = s.size();
    vector<int> sa(n), rk(n), tmp(n);
    iota(sa.begin(), sa.end(), 0);
    for (int i = 0; i < n; ++i) rk[i] = (unsigned char) s[i];
    // k 逐次翻倍，rank 对应长度 2k 的前缀；所有后缀按新 rank 重排。
    for (long long k = 1; k < n; k <<= 1) {
        // 参数 i 是 0..n-1 的后缀起点；返回当前 2k 长度排序键，只读捕获的 rk/k。
        auto rankPair = [&](int i) {
            return pair{rk[i], i + k < n ? rk[i + k] : -1};
        };
        // 比较器参数是两个后缀起点；按 rankPair 严格升序比较，不改捕获状态。
        sort(sa.begin(), sa.end(), [&](int i, int j) {
```

```
            return rankPair(i) < rankPair(j);
        });
        if (n) tmp[sa[0]] = 0;
        for (int i = 1; i < n; ++i) {
            int a = sa[i - 1], b = sa[i];
            tmp[b] = tmp[a] + (rankPair(a) < rankPair(b));
        }
        rk=tmp;
        if (n && rk[sa.back()] == n - 1) break;
    }
    return sa;
}
// 参数 sa 必须是按后缀字典序排列的 [0,|s|) 起点排列；返回相邻后缀 LCP，长度 max(0,n-1)，不改输入。
vector<int> lcpArray(const string& s, const vector<int>& sa) {
    int n = s.size();
    vector<int> rk(n), lcp(max(0, n - 1));
    for (int i = 0; i < n; ++i) rk[sa[i]] = i;
    // Kasai 复用相邻后缀的公共前缀，向右移动起点后最多只需少比较一个字符。
    for (int i = 0, h = 0; i < n; ++i) {
        int r = rk[i];
        if (r == n - 1) {
            h = 0;
            continue;
        }
        int j = sa[r + 1];
        while (i + h < n && j + h < n && s[i + h] == s[j + h]) ++h;
        lcp[r] = h;
        if (h) --h;
    }
    return lcp;
}
```

坑：空串要单独确认调用逻辑；`lcp[i]` 对应 `sa[i]` 与 `sa[i+1]`；百万级数据应改计数排序版本。

9. 计算几何

本章要求浮点输入为有限值，并采用两套明确的数值约定：含交点、距离、投影时使用 `long double` 和绝对误差 `EPS = 1e-12L`；坐标尺度变化明显时应按题目精度调整 EPS，或采用相对误差。整数输入只判断方向、求凸包或平方距离时，输入用 `long long`、运算前提升到 `__int128`；坐标差、乘积、叉积、面积和平方距离的中间量都必须落在 signed `__int128` 范围内，否则需增加坐标限制或使用大整数。浮点数不能直接用于有序容器的“近似相等”比较。

9.1 点、向量与基本运算

适用范围： 二维向量加减、缩放、点积/叉积、长度和旋转；含除法与三角函数的量使用 `long double`。 **能做什么：** 各操作 $O(1)$；`sgn` 使用绝对 EPS=1e-12，坐标尺度过大需重新选择容差。 **输入 / 输出：** `Point` 字段为实数坐标；运算符返回新值不修改操作数。`dot/cross/norm2/length` 分别返回标量量值；`rotate(a,angle)` 以弧度逆时针旋转。除法标量必须非零。

用法：

- `Point(x,y)` 表示浮点坐标；向量 `b-a` 从 `a` 指向 `b`。
- `dot(a,b)` 为点积，`cross(a,b)` 为二维叉积标量。
- `length(v)` 求长度，`rotate(v,angle)` 按弧度逆时针旋转。
- `sgn(x)` 将浮点量按 EPS 归为负、零、正。

```
#include <bits/stdc++.h>
using namespace std;
using ld = long double;

const ld EPS = 1e-12L;
const ld PI = acosl(-1.0L);

// 这是绝对误差判号；若坐标尺度极大，应按题目误差要求调整 EPS 或改用相对误差。
// 参数 x 为 long double；按绝对 EPS 返回 -1/0/1，不修改全局容差。
int sgn(ld x) {
    if (x > EPS) return 1;
    if (x < -EPS) return -1;
    return 0;
}

struct Point {
    // 浮点点也作为向量使用；b-a 表示从 a 指向 b，坐标下标不涉及点编号。
    ld x = 0, y = 0;
    // 参数 p 是另一向量；返回逐坐标相加的新点，不改两操作数。
    Point operator+(const Point& p) const { return {x + p.x, y + p.y}; }
    // 参数 p 是被减向量；返回 this-p 的新向量，不改两操作数。
    Point operator-(const Point& p) const { return {x - p.x, y - p.y}; }
    // 参数 k 为实数缩放因子；返回缩放后的新向量，不改 this。
    Point operator*(ld k) const { return {x * k, y * k}; }
    // 参数 k 必须非零；返回逐坐标除以 k 的新向量，不改 this。
    Point operator/(ld k) const { return {x / k, y / k}; }
};

// 参数 a,b 为向量；返回点积标量，不修改输入。
ld dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
// cross(a,b)>0 表示从向量 a 逆时针转到 b；绝对值是平行四边形面积。
// 参数 a,b 为向量；返回有向叉积标量，不修改输入。
ld cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
```

```
// 参数 a 为向量；返回平方长度，不开方且不修改输入。
ld norm2(Point a) { return dot(a, a); }
// 参数 a 为向量；返回非负欧氏长度，不修改输入。
ld length(Point a) { return sqrtl(norm2(a)); }

// 参数 a 为向量、angle 为弧度；返回逆时针旋转后的新向量，不改输入。
Point rotate(Point a, ld angle) {
    // angle 使用弧度，正值按数学坐标系逆时针旋转；屏幕 y 轴向下时视觉方向相反。
    ld c = cosl(angle), s = sinl(angle);
    return {a.x * c - a.y * s, a.x * s + a.y * c};
}
```

易错点：

1. 角度函数使用弧度；角度制需乘 `PI/180`。
2. 浮点判零用 `sgn`，不要直接写 `x == 0`。
3. 除以向量长度前要先判断是否为零向量。

参考：

- OI Wiki：二维计算几何基础

9.2 叉积与方向判断

适用范围： 整数点的精确方向、共线和闭线段包含判断；坐标差及叉积须能由 signed `__int128` 表示。 **能做什么：** `cross` 与 `orientation` 为 $O(1)$；`onSegment` 为 $O(1)$ 且端点包含。 **输入 / 输出：** `cross(a,b)` 返回向量叉积；三点重载返回 `(b-a)x(c-a)`；`orientation` 输出 1/0/-1；`onSegment(a,b,p)` 返回布尔值。输入点为 `long long` 整数坐标。

用法：

- `orientation(a,b,c)` 返回 1、0、-1，分别表示左转、共线、右转。
- `onSegment(a,b,p)` 判断 `p` 是否在闭线段 `ab` 上，包含端点。
- 模板适用于整数输入，坐标类型为 `long long`。
- 浮点坐标应把叉积传给 9.1 的 `sgn`，并用 EPS 判断范围。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;

struct IPoint {
    ll x = 0, y = 0;
};

// 两向量叉积；整数坐标下精确判号，乘法提升到 i128 以避免 long long 溢出。
// 参数 a,b 为整数向量；返回叉积，结果须可由 signed i128 表示。
i128 cross(IPoint a, IPoint b) {
    return (i128)a.x * b.y - (i128)a.y * b.x;
}

// 参数 a,b,c 为整数点；返回 (b-a)x(c-a)，精确结果须在 signed i128 范围内。
i128 cross(IPoint a, IPoint b, IPoint c) {
    // 返回 (b-a)x(c-a)：正为 a->b 到 a->c 左转，负为右转，零为共线。
    i128 abx = (i128)b.x - a.x, aby = (i128)b.y - a.y;
    i128 acx = (i128)c.x - a.x, acy = (i128)c.y - a.y;
    return abx * acy - aby * acx;
```

```
}

// 参数为三点；返回左转 1、共线 0、右转 -1，不修改输入。
int orientation(IPoint a, IPoint b, IPoint c) {
    i128 value = cross(a, b, c);
    return (value > 0) - (value < 0);
}

// 参数 a,b 定义闭线段、p 为查询点；共线且在包围盒内（含端点）时返回 true。
bool onSegment(IPoint a, IPoint b, IPoint p) {
    // 闭线段判定：共线还不够，还要落在两端点的横纵坐标包围盒内；端点算在线。
    return orientation(a, b, p) == 0 &&
           min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
           min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}
```

易错点：

1. 三点顺序决定叉积符号，交换两个点会反号。
2. 共线不等于在线段上，还需检查横纵坐标范围。
3. 三点叉积先把坐标提升到 `__int128` 再相减；若坐标接近 `long long` 极限，叉积本身仍可能越界。

参考：

- OI Wiki：极坐标与叉积

9.3 直线交点与线段相交

适用范围： 浮点直线唯一交点与两条闭线段相交判断；直线方向向量平方长度须大于 EPS，几何判断采用绝对 EPS=1e-12。 **能做什么：** 所有单次操作 $O(1)$；端点接触、共线重叠均算线段相交。 **输入 / 输出：** `lineIntersection(p,v,q,w,out)` 对 `p+t*v` 与 `q+s*w` 成功时写交点并返回 true；平行/重合返回 false 且 out 不变。`segmentsIntersect(a,b,c,d)` 返回是否共享点。

用法：

- `lineIntersection(p,v,q,w,out)` 求两无限直线唯一交点。
- 返回 `false` 表示平行或重合，此时 `out` 无效。
- `segmentsIntersect(a,b,c,d)` 判断两条闭线段是否有公共点。
- 若整数输入只需判断相交，优先使用 9.2 的整数叉积改写，结果精确。

```
#include <bits/stdc++.h>
using namespace std;
using ld = long double;
const ld EPS = 1e-12L;

// EPS 是绝对误差阈值；整数输入且只判相交时应改用 i128 叉积以得到精确结果。
// 参数 x 为浮点量；按本节绝对 EPS 返回 -1/0/1。
int sgn(ld x) { return (x > EPS) - (x < -EPS); }

struct Point {
    ld x = 0, y = 0;
    // 参数 p 为向量；返回坐标和的新值，不改操作数。
    Point operator+(Point p) const { return {x + p.x, y + p.y}; }
    // 参数 p 为向量；返回 this-p 的新值，不改操作数。
    Point operator-(Point p) const { return {x - p.x, y - p.y}; }
    // 参数 k 为实数；返回按 k 缩放的新向量，不改 this。
    Point operator*(ld k) const { return {x * k, y * k}; }
```

```
};

// 参数 a,b 为向量；返回浮点有向叉积，不修改输入。
ld cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
// 参数为三点；返回 (b-a)x(c-a)，供方向判定使用。
ld cross(Point a, Point b, Point c) { return cross(b - a, c - a); }

// 参数给出 p+t*v、q+s*w 且方向非零；成功写 out 返回 true，平行/重合返回 false 且 out 不变。
bool lineIntersection(Point p, Point v, Point q, Point w, Point& out) {
    // 两条无限直线分别是 p+t*v、q+s*w；v、w 必须是非零方向向量。
    ld denominator = cross(v, w);
    // 叉积近零包含“平行不重合”和“整条重合”，二者都没有唯一交点。
    if (sgn(denominator) == 0) return false;
    // 由 (q-p)xw=t*(v x w) 解出参数 t；交换叉积顺序会把符号写反。
    ld t = cross(q - p, w) / denominator;
    out = p + v * t;
    return true;
}

// 参数 a,b 定义闭线段；p 在线段上或端点处返回 true，a==b 时只包含该点。
bool onSegment(Point a, Point b, Point p) {
    // 使用闭区间语义并包含端点；退化线段 a==b 时仅该点自身通过判定。
    return sgn(cross(a, b, p)) == 0 &&
           sgn((p.x - a.x) * (p.x - b.x)) <= 0 &&
           sgn((p.y - a.y) * (p.y - b.y)) <= 0;
}

// 参数为两条闭线段端点；返回是否共享一点，端点接触和共线重叠都算相交。
bool segmentsIntersect(Point a, Point b, Point c, Point d) {
    // c1、c2 是 c、d 相对有向直线 a->b 的方向；异号才是严格跨越。
    int c1 = sgn(cross(a, b, c)), c2 = sgn(cross(a, b, d));
    int c3 = sgn(cross(c, d, a)), c4 = sgn(cross(c, d, b));
    if (c1 * c2 < 0 && c3 * c4 < 0) return true;
    return (c1 == 0 && onSegment(a, b, c)) ||
           (c2 == 0 && onSegment(a, b, d)) ||
           (c3 == 0 && onSegment(c, d, a)) ||
           (c4 == 0 && onSegment(c, d, b));
}
```

易错点：

1. “严格相交”不含端点和重合；本模板判断的是闭线段有公共点。
2. 平行直线可能重合，`false` 不等于“没有公共点”。
3. 退化线段 `a==b` 仍可由 `segmentsIntersect` 判断，但不能作为直线方向。

参考：

- OI Wiki：直线与线段相关

9.4 距离与投影

适用范围： 实数坐标的垂足、点到无限直线或闭线段的距离；直线端点不同，短线段按 EPS^2 退化处理。 **能做什么：** 每次投影/距离 $O(1)$；线段投影参数截到 `[0,1]`。 **输入 / 输出：**
`projectionOnLine/distanceToLine(p,a,b)` 以 a、b 定义无限直线且要求 `a!=b`；
`closestPointOnSegment` 退化时返回 a；距离函数返回非负 `long double`。

用法：

- `projectionOnLine(p,a,b)` 返回 `p` 到无限直线 `ab` 的垂足。
- `distanceToLine` 返回点到无限直线距离。
- `closestPointOnSegment` 返回闭线段上离 `p` 最近的点。
- `distanceToSegment` 同时处理垂足在线段外和 `a==b`。

```
#include <bits/stdc++.h>
using namespace std;
using ld = long double;
const ld EPS = 1e-12L;

struct Point {
    ld x = 0, y = 0;
    // 参数 p 为向量；返回坐标和的新值，不改操作数。
    Point operator+(Point p) const { return {x + p.x, y + p.y}; }
    // 参数 p 为向量；返回 this-p 的新值，不改操作数。
    Point operator-(Point p) const { return {x - p.x, y - p.y}; }
    // 参数 k 为缩放因子；返回新向量，不改 this。
    Point operator*(ld k) const { return {x * k, y * k}; }
};

// 参数 a,b 为向量；返回点积标量。
ld dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
// 参数 a 为向量；返回平方长度。
ld norm2(Point a) { return dot(a, a); }
// 参数为两点；返回非负欧氏距离，平方量以 long double 计算。
ld distance(Point a, Point b) { return sqrtl(norm2(a - b)); }

// 参数 p 为查询点、a!=b 定义无限直线；返回垂足，不限制投影参数。
Point projectionOnLine(Point p, Point a, Point b) {
    // 求无限直线参数 t；前提 a!=b，否则 norm2(b-a)=0。线段版本在下方单独处理退化。
    Point v = b - a;
    return a + v * (dot(p - a, v) / norm2(v));
}

// 参数 p 为查询点、a!=b 定义直线；返回到无限直线的非负距离。
ld distanceToLine(Point p, Point a, Point b) {
    return distance(p, projectionOnLine(p, a, b));
}

// 参数 p 为查询点、a,b 为线段端点；返回最近点，EPS² 内退化段返回 a。
Point closestPointOnSegment(Point p, Point a, Point b) {
    Point v = b - a;
    ld length2 = norm2(v);
    // EPS^2 与平方长度同量纲；退化或极短线段按单点 a 处理。
    if (length2 < EPS * EPS) return a;
    ld t = dot(p - a, v) / length2;
    // 无限直线投影参数截到 [0,1]，分别对应端点 a、b 和线段内部。
    t = max((ld)0, min((ld)1, t));
    return a + v * t;
}

// 参数 p 为查询点、a,b 为闭线段端点；返回非负距离，退化段按点 a 计算。
ld distanceToSegment(Point p, Point a, Point b) {
    return distance(p, closestPointOnSegment(p, a, b));
}
```

易错点：

1. 点到直线与点到线段不同；线段要截断投影参数。

2. `a==b` 时分母为零，必须退化为点距离。
3. 比较距离大小时可优先比较平方距离，减少开方误差与开销。

参考：

- OI Wiki：距离

9.5 多边形面积

适用范围： 顶点按简单多边形边界顺序排列的整数多边形；坐标差、叉积和总和都须能放进 signed `__int128`。 **能做什么：** 鞋带公式一次遍历，$O(n)$ 时间、$O(1)$ 额外空间；精确得到有向二倍面积。 **输入 / 输出：** `signedDoubleArea(polygon)` 不修改输入，首点不需重复；返回逆时针为正的二倍面积，少于 3 点或空点集返回 0。

用法：

- 顶点必须按多边形边界的顺时针或逆时针顺序给出。
- 首点无需在数组末尾重复，模板用取模闭合末边。
- `signedDoubleArea` 返回整数有向二倍面积；实际面积取绝对值除以 2。
- 若答案可能为半整数，可输出 `abs(value)/2` 加 `.0` 或 `.5`。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;

struct IPoint { ll x = 0, y = 0; };

// 单项叉积提升到 i128；若坐标约在 1e18 量级，多个叉积之和仍可能超过 i128。
// 参数 a,b 为整数位置向量；返回叉积，signed i128 须容纳乘积差。
i128 cross(IPoint a, IPoint b) {
    return (i128)a.x * b.y - (i128)a.y * b.x;
}

// 参数为按边界顺序且首点不重复的顶点列；返回有向二倍面积，逆时针为正，不改输入。
i128 signedDoubleArea(const vector<IPoint>& polygon) {
    // 顶点须按边界顺序给出且不重复首点；返回有向二倍面积，逆时针为正。
    i128 sum = 0;
    int n = (int)polygon.size();
    // (i+1)%n 自动补上末点到首点的闭合边；n<3 时自然得到退化面积 0。
    for (int i = 0; i < n; ++i)
        sum += cross(polygon[i], polygon[(i + 1) % n]);
    return sum;
}
```

易错点：

1. 无序点集不能直接代入公式，必须先得到边界顺序。
2. 有向面积可能为负；只求面积时取绝对值。
3. 面积公式返回二倍面积，别忘记除以 2。

参考：

- OI Wiki：多边形面积

9.6 点在多边形内

适用范围： 顶点顺序给定的简单浮点多边形；采用绝对 EPS=1e-12。自交多边形不适用。 **能做什么：** 射线奇偶规则单次 $O(n)$、额外空间 $O(1)$，并区分边界。 **输入 / 输出：** `pointInPolygon(polygon,p)` 返回 OUTSIDE/BOUNDARY/INSIDE；空点列返回 OUTSIDE，边界按闭边判定。坐标尺度过大时需调整 EPS。

用法：

- `pointInPolygon` 返回 `OUTSIDE`、`BOUNDARY` 或 `INSIDE`。
- 多边形可以顺时针或逆时针，首点无需重复。
- 模板使用浮点 EPS，适合一般实数坐标。
- 整数坐标且要求精确时，可用 `__int128` 叉积实现相同的奇偶规则。

```
#include <bits/stdc++.h>
using namespace std;
using ld = long double;
const ld EPS = 1e-12L;

// 参数 x 为浮点量；按本节绝对 EPS 返回 -1/0/1。
int sgn(ld x) { return (x > EPS) - (x < -EPS); }
struct Point {
    ld x = 0, y = 0;
    // 参数 p 为向量；返回 this-p 的新值，不改操作数。
    Point operator-(Point p) const { return {x - p.x, y - p.y}; }
};
// 参数 a,b 为向量；返回有向叉积标量。
ld cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }

// 参数 a,b 定义闭边、p 为点；共线且落在包围盒内（含端点）时返回 true。
bool onSegment(Point a, Point b, Point p) {
    // 边界优先单独判定，避免射线与水平边或顶点重合时归属不明确。
    return sgn(cross(b - a, p - a)) == 0 &&
           sgn((p.x - a.x) * (p.x - b.x)) <= 0 &&
           sgn((p.y - a.y) * (p.y - b.y)) <= 0;
}

enum PointLocation { OUTSIDE, BOUNDARY, INSIDE };

// 参数是按边界顺序的简单多边形和查询点；返回 OUTSIDE/BOUNDARY/INSIDE，不修改输入。
PointLocation pointInPolygon(const vector<Point>& polygon, Point p) {
    // 适用于顶点按边界排列的简单多边形；返回值明确区分边界与内部。
    bool inside = false;
    int n = (int)polygon.size();
    for (int i = 0; i < n; ++i) {
        Point a = polygon[i], b = polygon[(i + 1) % n];
        if (onSegment(a, b, p)) return BOUNDARY;
        // 半开高度条件保证射线经过顶点时只让相邻两边中的一条计数。
        bool straddles = (a.y > p.y) != (b.y > p.y);
        if (straddles) {
            ld intersectionX = a.x + (b.x - a.x) *
                (p.y - a.y) / (b.y - a.y);
            // 只统计查询点右侧交点；每跨过一次边界就翻转内外奇偶性。
            if (intersectionX > p.x + EPS) inside = !inside;
        }
    }
    return inside ? INSIDE : OUTSIDE;
}
```

易错点：

1. 射线恰过顶点时不能把相邻两条边重复计数；模板用半开高度条件处理。
2. 必须先判断边界，否则题意中的边界归属会混乱。
3. 自交多边形的“内部”定义不同，本模板按简单多边形使用。

参考：

- OI Wiki：判断一个点是否在多边形内部

9.7 凸包

适用范围： 无序整数点集求凸包；输入重复点可有，坐标差与叉积必须适配 signed `__int128`。 **能做什么：** Andrew 单调链 $O(n\log n)$ 时间、$O(n)$ 空间；删去边上中间共线点。 **输入 / 输出：** `convexHull(points)` 按值接收并排序去重，返回逆时针点列且不重复首点；空集空返回、单点保留、全共线只返两端点。

用法：

- `convexHull(points)` 返回逆时针顶点，起点为字典序最小点。
- 返回结果不重复首点；重复输入点会先被删除。
- 全部共线时只返回两个端点；单点时返回该点。
- 若要保留凸包边上的所有共线点，把弹栈条件改为 `< 0`，并谨慎处理全共线情形。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;

struct IPoint {
    ll x = 0, y = 0;
    // 参数 p 为另一整数点；按 x 再 y 严格字典序比较，用于排序，不改点。
    bool operator<(const IPoint& p) const { return tie(x, y) < tie(p.x, p.y); }
    // 参数 p 为另一整数点；仅两坐标完全相等时返回 true。
    bool operator==(const IPoint& p) const { return x == p.x && y == p.y; }
};

// 参数为整数点；返回 (b-a)x(c-a)，坐标差和乘积差须在 signed i128 范围。
i128 cross(IPoint a, IPoint b, IPoint c) {
    // (b-a)x(c-a)>0 是严格左转；整数 i128 判号不需要 EPS。
    i128 abx = (i128)b.x - a.x, aby = (i128)b.y - a.y;
    i128 acx = (i128)c.x - a.x, acy = (i128)c.y - a.y;
    return abx * acy - aby * acx;
}

// 参数 points 按值接收，可乱序/重复；返回逆时针凸包且不重复首点，删去边上中间共线点。
vector<IPoint> convexHull(vector<IPoint> points) {
    // 输入可无序且可重复；输出 0-based vector 中按逆时针排列，不重复首点。
    sort(points.begin(), points.end());
    points.erase(unique(points.begin(), points.end()), points.end());
    if (points.size() <= 1) return points;

    vector<IPoint> hull;
    for (const IPoint& p : points) {
        // 维护下凸壳连续三点严格左转；<=0 会同时删除右转和边上共线中间点。
        // 若题目要求保留边界共线点，改为 <0，并额外处理全部共线的输入。
        while (hull.size() >= 2 &&
               cross(hull[hull.size() - 2], hull.back(), p) <= 0)
```

```
            hull.pop_back();
        hull.push_back(p);
    }
    size_t lowerSize = hull.size();
    // 逆序扫描构造上凸壳；lowerSize 是边界，防止弹掉已完成的整条下凸壳。
    for (int i = (int)points.size() - 2; i >= 0; --i) {
        const IPoint& p = points[i];
        while (hull.size() > lowerSize &&
               cross(hull[hull.size() - 2], hull.back(), p) <= 0)
            hull.pop_back();
        hull.push_back(p);
    }
    // 最后压入的是起点副本，删除它以保持“首点不重复”的返回约定。
    hull.pop_back();
    return hull;
}
```

易错点：

1. 必须先去重，否则相同点会破坏转向判断。
2. `<= 0` 会删除边上共线点，改成 `< 0` 的语义不同。
3. 返回值不含重复首点，遍历边时应使用 `(i+1)%n`。

参考：

- OI Wiki：凸包

9.8 旋转卡壳：凸包直径

适用范围： 点集已是逆时针凸包且不重复首点；叉积、绝对值及平方距离全部须适配 signed `__int128`。 **能做什么：** 旋转卡壳 $O(h)$ 时间、$O(1)$ 额外空间；含建凸包总计 $O(n\log n)$。 **输入 / 输出：** `convexDiameterSquared(hull)` 返回最远点对的平方距离；0/1 个点返回 0，2 点返回唯一距离。`toString` 仅接受非负值。

用法：

- 输入 `hull` 必须是逆时针严格凸包，且不重复首点，可直接接 9.7 输出。
- `convexDiameterSquared` 返回最大欧氏距离的平方。
- 一个点时答案为 0；两个点时直接返回二者平方距离。
- 需要实际距离时转成 `long double` 后开方；需要点对可在更新最大值时同步记录下标。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;

struct IPoint {
    ll x = 0, y = 0;
};

// 参数为整数点；返回有向叉积，调用者保证差值和乘积在 signed i128 范围。
i128 cross(IPoint a, IPoint b, IPoint c) {
    i128 abx = (i128)b.x - a.x, aby = (i128)b.y - a.y;
    i128 acx = (i128)c.x - a.x, acy = (i128)c.y - a.y;
    return abx * acy - aby * acx;
}
```

```
// 参数为两整数点；返回平方距离，坐标差、平方和须可由 signed i128 表示。
i128 distanceSquared(IPoint a, IPoint b) {
    // 差值和平方均在 i128 中完成；返回平方距离，避免开方误差。
    i128 dx = (i128)a.x - b.x;
    i128 dy = (i128)a.y - b.y;
    return dx * dx + dy * dy;
}

// 参数 x 不得是 signed i128 最小值；返回 |x|，不修改外部状态。
i128 abs128(i128 x) { return x < 0 ? -x : x; }

// 参数是逆时针且不重复首点凸包；返回最远平方距离，0/1 点为 0、2 点为点距。
i128 convexDiameterSquared(const vector<IPoint>& hull) {
    // 前提：hull 是逆时针严格凸包且不重复首点；可直接接 convexHull 的输出。
    int n = (int)hull.size();
    if (n <= 1) return 0;
    if (n == 2) return distanceSquared(hull[0], hull[1]);

    i128 answer = 0;
    int j = 1;
    for (int i = 0; i < n; ++i) {
        int nextI = (i + 1) % n;
        // 固定有向边 i->nextI，面积随对踵点前进先增后减，因此 j 全程只向前转。
        // 使用严格 > 可停在面积平台任一端；随后同时检查边的两个端点与 j。
        while (abs128(cross(hull[i], hull[nextI], hull[(j + 1) % n])) >
               abs128(cross(hull[i], hull[nextI], hull[j]))) {
            j = (j + 1) % n;
        }
        answer = max(answer, distanceSquared(hull[i], hull[j]));
        answer = max(answer, distanceSquared(hull[nextI], hull[j]));
    }
    return answer;
}

// 参数 x>=0；返回十进制字符串，仅转换非负距离值，不修改输入。
string toString(i128 x) {
    // 本章仅用它输出非负平方距离；若复用于有向面积，需先处理负号和最小值。
    if (x == 0) return "0";
    string s;
    while (x > 0) {
        s.push_back(char('0' + x % 10));
        x /= 10;
    }
    reverse(s.begin(), s.end());
    return s;
}
```

易错点：

1. 旋转卡壳不能直接用于未排序点集；必须先求凸包。
2. 凸包含重复首点或大量边上共线点时，指针逻辑容易出错。
3. `__int128` 不能直接用 `cout` 输出，模板提供了非负整数转换函数。

参考：

- OI Wiki：旋转卡壳

9.9 圆与直线、圆与圆

适用范围： 圆半径非负，直线方向向量平方长度须大于 EPS；圆心、半径均为 `long double`，判别式使用绝对 EPS=1e-12。 **能做什么：** 求直线/圆及圆/圆交点每次 $O(1)$；切点只返回一个点。 **输入 / 输出：** `lineCircleIntersections(p,v,c)` 返回 0/1/2 个点；`dot(v,v)<=EPS` 的近零方向与无交点都返回空。`circleIntersections(a,b)` `d<=EPS` 的近同心情形（包括重合圆的无限交点情形）返回空；相切返回一个点，分离/包含返回空。

```
struct Circle {
    Point o;
    long double r;
};
// 参数 p,v 定义直线且 dot(v,v)>EPS，圆半径非负；返回 0/1/2 交点，近零方向返回空。
vector<Point> lineCircleIntersections(Point p,Point v,Circle c) {
    long double vv=dot(v,v);
    if (vv <= EPS) return {};
    Point foot=p+v*(dot(c.o-p,v)/vv);
    long double h2=c.r*c.r-dot(foot-c.o,foot-c.o);
    if (h2 < -EPS) return {};
    if (h2 <= EPS) return {foot};
    Point off=v*(sqrtl(max((long double)0,h2)/vv));
    return {foot-off,foot+off};
}
// 参数是半径非负的两圆；返回 0/1/2 交点，`d<=EPS` 的近同心情形（含重合圆）返回空，切点只返一次。
vector<Point> circleIntersections(Circle a,Circle b) {
    // 先判相离、包含、同心等退化，再计算垂足与偏移。
    Point dvec=b.o-a.o;
    long double d=length(dvec);
    if(d<=EPS) {
        return {};
    }
    // 同心：无交点或无穷多交点，均返回空
    if (d > a.r + b.r + EPS || d < fabsl(a.r - b.r) - EPS) return {};
    long double x = (a.r * a.r - b.r * b.r + d * d) / (2 * d);
    long double h2 = a.r * a.r - x * x;
    Point base = a.o + dvec * (x / d);
    if (h2 <= EPS) return {base};
    Point perp{-dvec.y / d, dvec.x / d};
    Point off = perp * sqrtl(max((long double) 0, h2));
    return {base - off, base + off};
}
```

坑：零方向向量不是直线；同心圆要区分无交点与无限交点时应另设返回状态；先用 EPS 判定再对根号内取 `max(0,·)`。

9.10 整数几何方向模板

适用范围： 整数坐标下精确判方向和点在线段上；坐标差及叉积需不超出 signed `__int128`。 **能做什么：** 叉积、符号和闭线段判断各 $O(1)$，无浮点 EPS。 **输入 / 输出：** `crossInt(a,b,c)` 返回三点有向叉积；`signInt(x)` 返回 -1/0/1；`onSegmentInt(p,a,b)` 端点包含并返回布尔值。距离平方需另行检查 i128 范围。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct IPoint {
```

```
    ll x,y;
};
// 参数为整数点；返回 (b-a)x(c-a)，坐标差和叉积须在 signed i128 范围。
__int128 crossInt(IPoint a, IPoint b, IPoint c) {
    // 转换发生在减法之前，避免 long long 坐标差先溢出。
    __int128 abx = (__int128)b.x - a.x, aby = (__int128)b.y - a.y;
    __int128 acx = (__int128)c.x - a.x, acy = (__int128)c.y - a.y;
    return abx * acy - aby * acx;
}
// 参数 x 为 signed i128；返回 -1/0/1，不修改输入。
int signInt(__int128 x) {
    return (x>0)-(x<0);
}
// 参数 p 为查询点、a,b 为线段端点；闭线段含端点判定，crossInt 不得溢出。
bool onSegmentInt(IPoint p,IPoint a,IPoint b) {
    return crossInt(a, b, p) == 0
        && min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x)
        && min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}
```

坑：__int128 输出需自行转换；距离平方也可能超过 long long；真正交点通常是分数，不能强行存整数点。

10. 题意关键词索引

先用题意关键词定位候选模板，再核对数据范围和前提；关键词相似不代表模型一定相同。

10.1 上机、语言与 STL

题意/现场信号	优先查看	使用前核对
多组数据、第二组错、状态残留	IO、多测与状态	题面是否真的给 T；清空范围与总规模
算法可能超时、二维表可能爆内存	复杂度与内存速判	看最坏范围和总规模
溢出、精度、INF、负数取模	整数、浮点与边界	中间值而非只看最终答案
RE、UB、偶发崩溃	编译、日志与 Sanitizer	第一处错误；失效迭代器与递归栈
连续数组、邻接表、排序去重	`vector` 与 `array`	`reserve` 不改变 `size`；扩容失效
文本处理、字符序列	`string`	下标是字节位置
点权、边、异质记录	`pair`、`tuple`	字段较多时用有名 `struct`
两端插删、栈、队列、BFS	`deque` 与适配器	`pop` 不返回元素；空容器不可读
动态取最大/最小	`priority_queue`	默认最大堆；任意删除不便
前驱后继、有序动态集合	`set`、`multiset`	重复项删除语义；用成员二分
键值映射且要排序	`map`	`operator[]` 会插入默认值
频率表、状态判重、稀疏映射	哈希容器	无顺序保证；最坏退化与扩容
固定位集合、批量可达性	`bitset`	长度编译期确定；移位状态不越界
排序、等值范围、二分边界	排序与二分	区间有序且比较规则一致
去重、排列枚举	去重与排列	`unique` 不缩容；排列数可达 $n!$
区间求和、填充、局部谓词	数值算法与 Lambda	累加初值类型；捕获生命周期
迭代器边界、`end()`、算法迭代器要求	迭代器与半开区间	类别决定可用操作；容器修改可能使其失效

10.2 序列、搜索、DP 与数据结构

题意关键词	候选模板	区分信号
区间和、矩形和、批量区间加	前缀和与差分	静态查询用前缀；批量修改后统一恢复用差分
“最小可行/最大可行”、单调答案	二分答案	必须证明判定谓词单调
有序数组两端、连续区间、窗口	双指针与滑动窗口	指针只向前；和式窗口常要求非负数
值域很大但只关心相对次序	离散化	后续是否仍需原值距离
逆序对、跨左右两段统计	分治与归并计数	合并时计算跨区间贡献
区间最多选若干、按结束时间选	贪心与区间调度	交换论证；别凭局部直觉
连通块、枚举路径、递归状态	DFS	深度可达 n 时注意栈溢出

题意关键词	候选模板	区分信号
最少步数、分层扩展、等权图	BFS	入队即标记；状态空间是否过大
边权只有 0/1、最短路	0-1 BFS	0 边放队首，1 边放队尾
排列/组合枚举、满足约束的方案	回溯与剪枝	恢复现场；估算指数规模
要输出满足限制的对象而非计算最优值	常用构造题思路	先证明可行条件，再核对不变量和小规模样例
$n \approx 40$ 的子集枚举	折半搜索	两半状态能否排序/哈希合并
每件最多一次、容量限制	0-1 背包	容量倒序
每件无限次/有限次数	完全/多重背包	完全背包容量正序；多重拆分
每件最多一次、容量限制	0-1 背包	组限制需另写组间转移；本章未给分组背包模板
最长严格/非严格上升子序列	LIS	严格用 `lower_bound`，非严格常用 `upper_bound`
两串保序公共序列、编辑操作	LCS / 编辑距离	子序列不要求连续
合并相邻段、括号/回文区间	区间 DP	按区间长度递增
树上选点、子树贡献	树形 DP	父子方向；是否要换根得到全点答案
每个点作根的答案	换根 DP	先子树 DP，再父到子传贡献
n 很小、集合状态	状压 DP	估算状态数与转移枚举
统计 $[0, N]$ 中满足数位限制的数	数位 DP	前导零、上界限制、记忆化维度
依赖无环、路径计数/最优值	DAG DP	拓扑顺序与不可达初值
随机过程的期望步数/收益	概率与期望 DP	线性方程、自环概率与终止条件
左/右首个更大更小、贡献范围	单调栈	严格与非严格比较符号
固长窗口最值、DP 滑窗优化	单调队列	队首过期；队尾维护单调性
动态连通、合并集合	并查集	不能直接处理删除；路径压缩+按秩合并
单点改、前缀和、区间和	树状数组	下标通常从 1 开始
区间加与区间求和	线段树（区间加）	懒标记合并、闭区间端点与建树范围
区间赋值、区间加与区间求和	线段树（区间赋值与加法）	赋值覆盖旧加法；之后的新加法累加到赋值值上
单点修改、自定义区间聚合	单点修改线段树与自定义合并	合并需满足结合律并提供双侧单位元；非交换合并须保持左右顺序
静态区间最值、幂等运算	ST 表	无修改；查询 $O(1)$

10.3 图与树

题意关键词	候选模板	使用前核对
依赖、先修、判有向环	拓扑排序	只适用于 DAG；输出数不足即有环
无权/等权最短路	BFS	多源时所有源同时入队

题意关键词	候选模板	使用前核对
非负边最短路	Dijkstra	负边不可用；跳过堆中旧状态
有负边、判可达负环	Bellman–Ford	第 `n` 轮松弛；负环定义是否限源可达
全源最短路、点数较小	Floyd	$O(n^3)$；中转点必须最外层
无向图最小生成森林	Kruskal	本章模板用排序边集加并查集；不连通时得到森林
左右两类配对、任务分配	二分图最大匹配	图是否确实二分；增广路访问标记
每条边恰走一次	欧拉路径/回路	度数条件、连通性、有向/无向版本
布尔选择与“至少一个”的二元约束	2-SAT	每条约束转为蕴含边；变量与否定编号
有向图互相可达、缩点	强连通分量	Tarjan 栈内标记与低链更新来源
删点/删边使无向图断开	割点与桥	返祖边、重边、DFS 根的特殊规则
子树区间、树上离线统计	DFS 序/子树展平	进入/退出时间及区间端点
子树颜色/频次统计、保留重儿子信息	DSU on Tree	子树遍历次序、清空轻儿子贡献与复杂度
祖先、两点树上距离	倍增 LCA	深度对齐；预处理根与倍增层数
树上最远点、最长路径	树的直径	负边权时两次最远点法需重新确认
容量、可同时输送的最大量	Dinic 最大流	反向边下标、残量网络、`long long` 容量
单位费用、在最大流下最小成本	最小费用最大流	负费用、势能/最短路与溢出
任意子树/路径修改查询	树链剖分	DFS 序方向与线段树区间一致

10.4 数学、字符串与几何

题意关键词	候选模板	使用前核对
最大公约数、线性丢番图	GCD / 扩展 GCD	系数与中间乘积范围
大指数取模	快速幂	模数、负底数与乘法溢出
多次判素、列素数或查询 `[1,n]` 正因数个数	素数筛	上界内存；线性筛或埃氏筛；计数查询限定在筛表范围
单个数分解、约数个数/和	质因数分解与约数公式	试除到 $p^2 \le n$；剩余大质因子
$1..n$ 中与 n 互质个数	欧拉函数	分解质因子后乘法顺序
除法取模、解同余	模逆元 / 线性同余	逆元存在当且仅当互质（一般模数）
多个余数条件合并	CRT / 扩展 CRT	模数可否不互质；合并是否无解
组合数、计数取模	阶乘逆元 / Pascal	模数是否质数，n 与模数大小
至少一个、排除重复计数	容斥原理	交集项符号和子集规模
线性递推、走固定步数	矩阵快速幂	状态向量方向、乘法次序、模数
实数线性方程组	高斯消元	浮点 EPS；检查无解与多解
必胜/必败、若干独立游戏	SG 函数 / Nim	是否公平组合游戏，mex 的可达集合

题意关键词	候选模板	使用前核对
子串相等、回文快速判定	字符串哈希	碰撞风险、下标与模数
单模式匹配、前后缀	KMP / 前缀函数	失配跳转和匹配位置换算
前缀与每个后缀的 LCP	Z 函数	Z-box 右端点约定
最长回文子串	Manacher	插入分隔符后的下标换算
小写字母串前缀查询/匹配	Trie	本章模板字符集为 `a–z`；节点内存与字符集大小
多模式匹配	AC 自动机	fail 指针、输出累计与重复模式
所有后缀排序、重复子串	后缀数组	rank/LCP 定义与 0/1-based
左右方向、共线、线段相交	叉积与方向判断	整数叉积溢出；浮点 EPS
直线交点、点到线距离/投影	直线与投影	平行、重合和退化线段
多边形面积、点在多边形内	多边形	顶点顺序；边界点如何定义
最小凸多边形包住所有点	凸包	共线点保留策略、重复点
凸包最远点对	旋转卡壳	输入须为按序凸包；退化点数
圆与直线/圆与圆交点	圆基础	相切误差、无穷多交点与退化半径

10.5 五秒模型核对

1. 数据范围允许这个复杂度吗？多测看总规模。
2. 模板前提满足吗：有序、单调、非负、无环、二分、连通？
3. 题目要的是值、方案、数量，还是字典序最小方案？
4. 下标、区间、无解状态、重边、自环、负权、取模是否对齐？
5. 复制模板后，哪三个位置必须按本题修改？